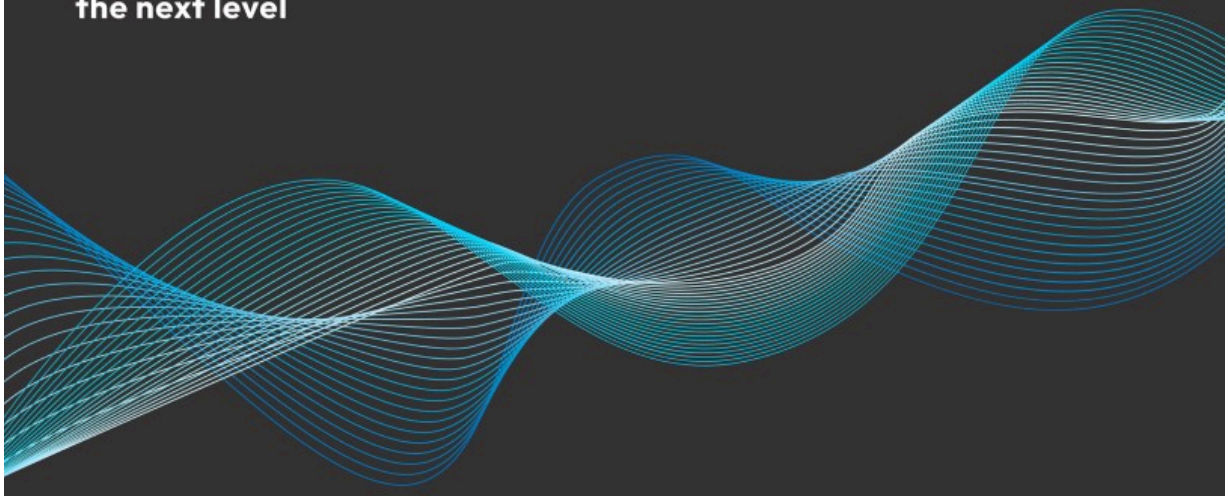
EXPERT INSIGHT

Early Access

Extreme DAX

Take your Power BI and Fabric analytics skills to the next level



Second Edition

Michiel Rozema
Henk Vlootman
Madzy Stikkelorum

‹packt›

Extreme DAX

Second Edition

Take your Power BI and Fabric analytics skills to the next level

Michiel Rozema
Madzy Stikkelorum
Henk Vlootman

‹packt›

Extreme DAX

Second Edition

Early Access Publication: Extreme DAX

Early Access Production reference: B31583

Published by Packt Publishing Ltd.
Grosvenor House
11 St Paul's Square
Birmingham
B3 1RB, UK.
ISBN 978-1-83664-763-8

www.packtpub.com

Table of Contents

Chapter 2: Model Design

Chapter 4: Context and Filtering

Welcome to Packt Early Access

Extreme DAX, Second Edition: Take your Power BI and Fabric analytics skills to the next level

We're giving you an exclusive preview of this book before it goes on sale. It can take many months to write a book, but our authors have cutting-edge information to share with you today. Early Access gives you an insight into the latest developments by making chapter drafts available. The chapters may be a little rough around the edges right now, but our authors will update them over time.

You can dip in and out of this book or follow along from start to finish; Early Access is designed to be flexible. We hope you enjoy getting to know more about the process of writing a Packt book.

Table of Contents

Chapter 3: Using DAX

Welcome to Packt Early Access

Extreme DAX, Second Edition: Take your Power BI and Fabric analytics skills to the next level

We're giving you an exclusive preview of this book before it goes on sale. It can take many months to write a book, but our authors have cutting-edge information to share with you today. Early Access gives you an insight into the latest developments by making chapter drafts available. The chapters may be a little rough around the edges right now, but our authors will update them over time.

You can dip in and out of this book or follow along from start to finish; Early Access is designed to be flexible. We hope you enjoy getting to know more about the process of writing a Packt book.

1

Analyzing Data with DAX

Join our book community on Discord

https://packt.link/EarlyAccessCommunity

Without a doubt, information nowadays is one of the most valuable assets of any organization. We all know this as consumers: companies are lining up to get our personal data. Not because any of us are so interesting individually (though we're sure *you* are a very interesting person!), but the combination of data from many consumers enables companies to get valuable insights to drive their business forward.

And this is not only true for commercial companies. Public institutions, hospitals, and universities also benefit from information to better run their core processes. In any case, information is the foundation of progress and innovation.

But to get from data to information to insights can be a tedious process. It involves combining data from different sources, discovering hidden structures and correlations, and considering the context of data. This is why the field of business intelligence, or data analytics, has traditionally been exclusive to IT professionals. This is not optimal, as it is really about *business* insights.

This book is all about one of the greatest languages for data analytics in existence today: **Data Analysis eXpressions** (**DAX**). As a reader, we assume that you have some experience with DAX and that you are hoping to improve your skills. First, it is important to understand what DAX is for and when *not* to use it. Additionally, you must learn to avoid doing things elsewhere that can be done better with DAX.

This chapter covers some general concepts that help you lay this foundation. We cover the following topics:

- The five-layer model for business intelligence

- Enterprise BI and end-user BI
- Power BI and Fabric
- Where DAX fits in, and where to find it
- Tools to develop models and DAX
- Powered by DAX: visual, interactive reports
- How to approach solution development
- The digital transformation cycle

The five-layer model for business intelligence

To be able to discuss analytics in a structured and comprehensive way, we have developed a simple framework that describes the main components and responsibilities in an analytics solution. We gave it an equally simple name: the **five-layer model**.

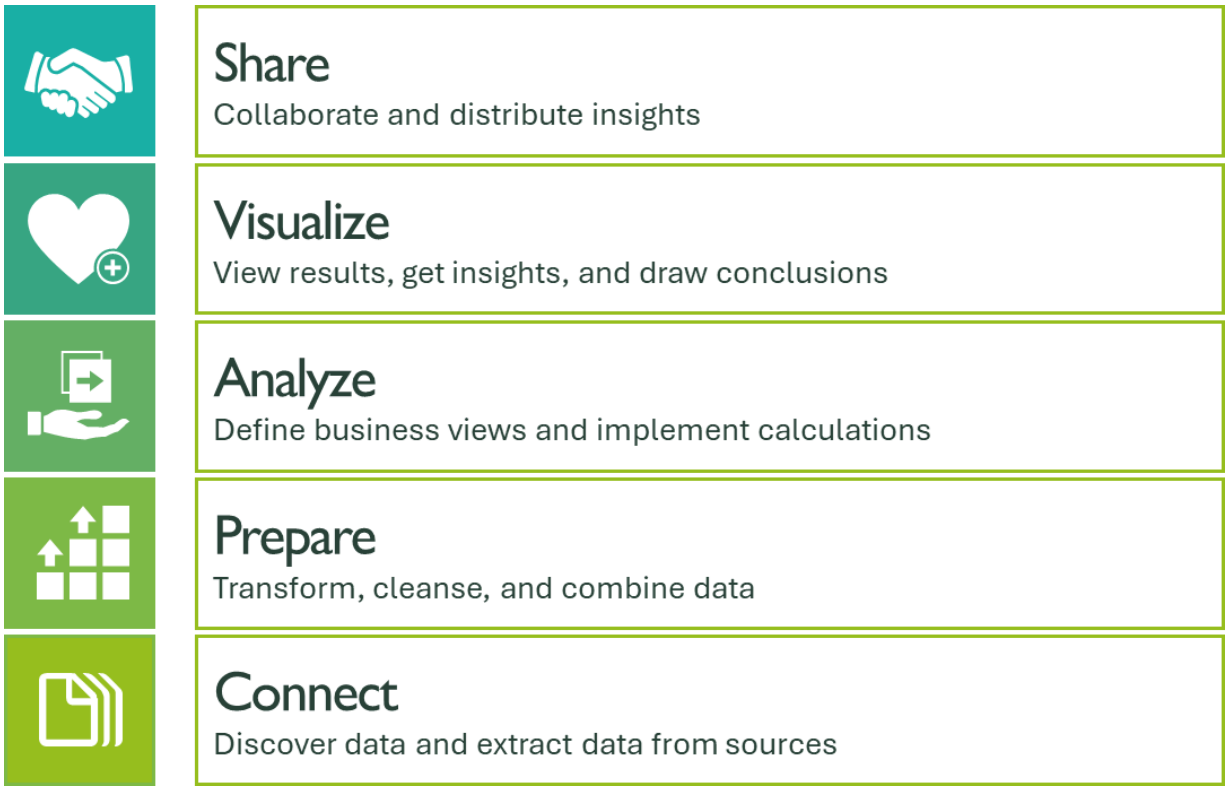


Figure 1.1: The five-layer model

The lowest layer, **Connect**, is the starting point of analytics: if you want to analyze data, that data must come from somewhere. It could reside in Excel sheets, text files, large business databases, or somewhere on the internet.

This raw data isn't typically in the right format to be able to analyze it, especially when it comes from different sources. Therefore, you need to **prepare** your data. Preparation can have many forms, such as imposing certain data types, transforming data, building data history, or matching data based on key attributes.

Creating nice and clean data sets in the *Connect* and *Prepare* layers can take a lot of effort. A lot! Building a data warehouse, a typical IT component in the *Prepare* layer, often leads to development projects spanning multiple years. The sad thing about this is that when the data warehouse is complete, the world has changed, and the data warehouse does not comply anymore.

When data has been put in a workable format, you can start with proper analysis of the data in the **Analyze** layer. This is really the heart of the analytics solution. Building analytical models allows you to slice and filter data, compute aggregations of all sorts, and add calculations to provide specific insights.

The **Visualize** layer is all about creating reports and dashboards visualizing the results of analytical models. We have called this layer *Visualize* and not simply *Output*, as a user needs help to focus on the important results. Visualizing turns output into insights, which is what BI is all about. Classic reports consisting of pages and pages of details are not very insightful. They force the user to export everything to Excel, where the user can aggregate the data. Providing real insight usually involves effective visualization of the most important outcomes of the analysis, while giving the user the option to answer new questions, arising from the insight, by viewing additional visuals on a lower detail level.

The top layer, **Share**, consists of platforms and processes aimed at providing access to reports and dashboards to the right people, while shielding them from those who shouldn't have access.

Whether you build your analytics solution in Excel, use Power BI or Fabric, develop a corporate business intelligence system, or use no automated system at all, you will have to cover the five layers in some way. In a good analytics solution, the layers are clearly separated, and the processing of data is done in the right layer. Doing this has a lot of advantages, such as avoiding implementing the same logic multiple times. Proper implementation of the

five-layer model makes it relatively easy to cope with changes in source systems, for example.

As you will see later in this chapter, DAX lives in the *Analyze* layer with strong ties to both the *Prepare* and *Visualize* layers. We will talk about visualizations in a separate section, but first, let us discuss the question: who does what in BI?

Enterprise BI and end-user BI

Organizations are becoming ever more driven by data. An organization that assesses performance on **key performance indicators** (**KPIs**) uses dashboards reflecting the status of each KPI. These dashboards are typically highly standardized: the organization has reflected on the business strategy, business processes, how to measure the KPIs, and how to report on them. An automated dashboard with KPIs is relatively stable and will not change much. It is typically built and maintained by a central IT department or BI competence center.

The next level of being a data-driven organization is that each decision can be based on insights from relevant data. This means that the need arises for a more dynamic form of analytics, one that can answer *ad hoc* questions. This form of analytics has been marketed as self-service BI, which promises that everyone can build a BI solution without the help of a central IT department. From the five-layer model, it is clear why this is not realistic: few end users have the time and capabilities to solve the complexities of all the layers. The ideal situation is when the central IT department and end users complement each other, each focusing on their specific strengths.

We use the terms *enterprise BI* and *end-user BI* to distinguish between the two forms of analytics. **Enterprise BI** is the form of analytics built and maintained by IT. It uses large-scale server or cloud platforms and serves many users at the same time. Solutions are built in professionally managed projects that focus on data quality, availability of the platform, and well-defined processes.

End-user BI is done by business users. They have a direct need for insights in their daily job and look for fast and accessible ways to get these insights. Many of them start building their analytics solutions in Excel, but the complexity of the Excel documents leads to an ever-increasing time

investment for keeping the data up to date and maintaining and extending the solution. Also, Excel is not capable of coping with growing data volumes. To solve this, Microsoft has extended the features of Excel to turn it into a real end-user BI platform, with Power Query and Power Pivot as powerful tools to prepare and analyze data. These tools have evolved into what is now Power BI.

The beauty of the Microsoft platform for analytics is that the tools for enterprise BI and end-user BI work seamlessly together. This started with Power BI back in 2015 and was deepened and widened with the introduction of the Microsoft Fabric platform in 2023. The technology underlying Power BI is one of the driving forces behind this blend of worlds.

The five-layer model is a useful framework to discuss how to combine self-service capabilities and corporate data platforms. In an ideal world, different users can tap into self-service analytics on each level of the model, while leveraging corporate assets where available. This is illustrated in Figure 1.2, where dark blocks denote corporate assets and light blocks show where self-service analytics can be done.

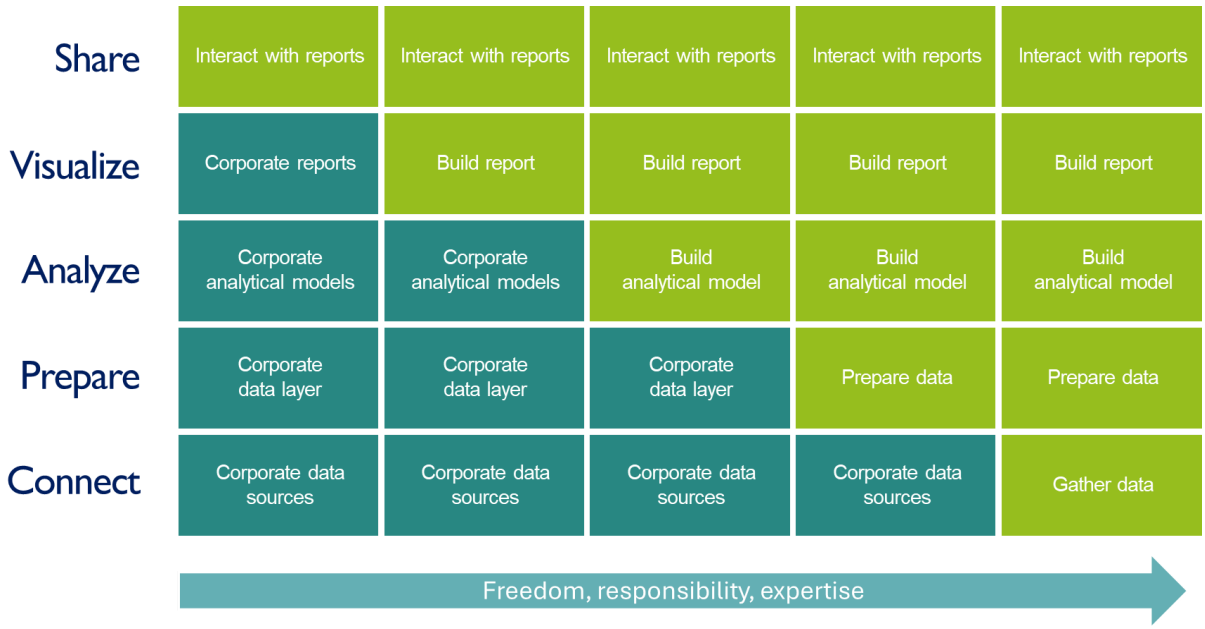


Figure 1.2: Self-service analytics

The most accessible form of self-service analytics is for users to interact with a report. They can do this with a well-designed, corporate report that has been developed in close alignment with the needs of the organization. In much the

same way, they can do this with a report that was made by themselves or a coworker in a self-service setting.

For some users, existing reports will not be enough. They can be given access to existing analytical models (whether corporate or created by business users) and build their own reports on top of these. With Power BI, the report can even expand on the model using additional DAX calculations!

The occasional business data analyst may have analytical needs that go beyond existing analytical models. They may leverage well-defined sets of data, prepared by either a corporate team or other business users, to design and develop their own, specific analytical model.

There may even be some business users who wish to do analytics with data that have not (yet) been incorporated into the enterprise BI platform, or that are only available in a "raw" state. These users may gather and prepare their own data to analyze.

It should be clear that when moving to the right in *Figure 1.2*, the complexity of the work done tends to increase. Therefore, this not only requires more advanced expertise, but it also adds to the responsibility of complying with corporate guidelines and standards. Central IT or BI needs to facilitate selected business users to create their own solutions. When implemented correctly, this leads to a situation where all people in the organization benefit from insights optimally.

The Microsoft platform for analytics offers several features that enable the blend of enterprise and end-user analytics outlined previously. One of these features is DAX, which is instrumental in empowering business users to not only create their own analytic solutions but also be involved in developing enterprise solutions. Before we dive into this in more detail, it is worthwhile to discuss the Microsoft analytics platform in a broader sense.

Fabric and Power BI

Since its introduction, Power BI has quickly become the leading platform for business intelligence. One of the reasons for this is, undoubtedly, that the platform allows business users to start doing BI for themselves without having to wait for a central department. All one needs to do is go to the Power BI website, `powerbi.com`, and register with a work or school account.

Almost everything you would want to do with Power BI comes in the form of items in a workspace. For instance, you can have dataflow items (which contain data connection and transformation logic), semantic model items (which encapsulate the actual analytics), report items (which contain the visualizations), or dashboard items (which provide an overview of insights from one or multiple reports).

When it comes to analytics, Power BI is not all there is. For example, large-scale data ingestion and transformation (although there have been many investments in making Power BI fit for Enterprise use), real-time analytics, and data science and AI are all things that require other tools and platforms.

With Fabric, Microsoft has envisioned a single platform for any data and analytics scenario. Based on the success of Power BI, Fabric takes the same core principles and expands on Power BI to address additional workloads. This means that in Fabric, you still have workspaces, but now with many more types of items that implement different things.

But the true revolutionary innovation in Fabric is the *separation ofshared storage* for different workloads. Before Fabric, choosing a technology meant choosing the way data is stored. Data in a data warehouse was not directly used by a seamlessly available to Power BI model, just as data and calculations in a Power BI model could not be directly used by a data scientist who wanted to train an AI model. In Fabric, this is different. All data in Fabric is stored in a single data store, **OneLake**, in a common format (in fact, any data file can be stored in OneLake, but once data is transformed into tables, these tables use the common format). On top of the data, Fabric offers several compute engines to do useful things with data. Think of a database engine to process queries in the SQL language, a Spark engine to do data science in Python or related languages, and an analytical engine that works with DAX.

This means that with Fabric, it is now easier for Power BI analysts to extend their work to other capabilities, such as large-scale *Connect* and *Prepare* work. But it also means that, for example, data scientists can leverage Power BI models for their work on AI. We'll discuss this in more detail later in this chapter.

Where DAX fits in, and where to find it

In an analytics solution based on the Microsoft platform, DAX is mostly used in the *Analyze* layer. DAX lives inside analytical models as the formula language to define calculations and other logic. We said *mostly*, as Power BI reports allow us to define DAX calculations, but still, these are merely extensions of the semantic model powering the report.

In fact, models and DAX are really two sides of the same coin: the design of the model impacts the complexity of the DAX code, and your skills in DAX determine your model designs (we will elaborate on the core concepts of semantic models in *Chapter 2, Model Design*).

The power of DAX is in its strong data aggregation capabilities. The DAX language contains many functions and constructs to define a variety of aggregations to generate results that lead to the insights needed. In the past, many types of aggregations could not be created directly but had to be implemented through specifically preparing data. For instance, computing a year-to-date sales total can be done in DAX with a single function, while with Excel or traditional reporting tools, separate indicators are needed to denote which sales transactions belong to the year-to-date period. The result, while more complex to implement, is still less dynamic than the DAX solution, which provides historical year-to-date figures as well.

This means that with DAX, the effort in data preparation is much lower than in traditional BI solutions. As the DAX syntax and many core concepts are similar to the use of formulas in Excel, the DAX *language* is relatively easy to learn for Excel users. This doesn't mean, though, that DAX is easy to master it requires a different kind of thinking than either Excel functions or database queries with a language like SQL. Moreover, when working with DAX, you will find that the complexity of questions that you'll want to answer with DAX calculations increases steadily. And with that, you will need to create ever more sophisticated DAX code. This book will give you many examples of advanced applications of DAX, which will hopefully help you solve your own DAX challenges.

All core Microsoft data products now offer the capability to build an analytical model, including DAX. It is confusing, however, that in each product, the naming of the model is different. Next, we will walk through an overview of models and DAX in different Microsoft products.

Excel

Since version 2010, Microsoft Excel has offered analytical data modeling capabilities in the form of **Power Pivot**. Power Pivot, also called **Data Model** in Excel, is the analytical model based on DAX.

Power BI and Fabric

The shining star in the Microsoft data platform is Power BI. It was introduced first as an add-on to Office 365 but was turned into a separate service in 2015. The analytical model in Power BI is called a semantic model and is the home of DAX. Now that Power BI has become one of the central components of the Microsoft Fabric platform, you may encounter semantic models in Fabric without an explicit reference to Power BI.

Semantic models are run within the Fabric cloud service, accessible through the Power BI website, `powerbi.com`, or through `fabric.com`. Some other access points to the service are available, such as the Power BI mobile app, Microsoft Teams, and even custom applications that can embed Power BI reports running on Fabric. Alternatively, Power BI Report Server is server software designed for customers who cannot or do not want to use the cloud service. You can run semantic models on your own PC as well, using Power BI Desktop.

SQL Server Analysis Services

Microsoft's data server platform, SQL Server, contains an analytics component called **SQL Server Analysis Services** (**SSAS**). SSAS offers a DAX-based analytical model capability called **Tabular** (which is mainly to distinguish from the older **multidimensional** analytical model capability).

Azure Analysis Services

Azure Analysis Services (**AAS**) is a fully managed analytics cloud service based on the same Tabular engine as SSAS. The difference, obviously, is that AAS runs in the cloud. The result is that your organization doesn't have to worry about the maintenance of the hardware and databases. It is also a flexible solution, since the resources can be scaled to meet the needs of the moment.

Going forward, Microsoft's investments in analytics are clearly focused on the Fabric platform, in which the features of (Power BI) semantic models go

beyond what is available in SSAS or AAS.

NOTE

The analytical model with DAX clearly exists under many different names: Power Pivot, Data Model, semantic model, or Tabular model. As Microsoft's investments in analytics are clearly focused on the Fabric platform, in which the features of (Power BI) semantic models go beyond what is available in SSAS or AAS, we will use the term **semantic model** throughout this book, or simply **model** when there is no risk for confusion. The term **analytical model** is used only conceptually in the context of the five-layer model.

Tools to develop semantic models and DAX

You have multiple tools to choose from if you want to work with DAX, depending on the target platform for the model:

- For a Power Pivot model, you use Excel with the Power Pivot editor.
- For a Tabular model in SSAS or AAS, you use Visual Studio, which offers many features for professional development, such as integration with version control systems, scripting, and compatibility.
- For a semantic model, you use Power BI Desktop. Alternatively, you can develop models directly in the Fabric cloud service. Fabric allows you to use Visual Studio as well.

NOTE

It is interesting to note that there are, in fact, three versions of Power BI Desktop. One can be downloaded from the Power BI website, `powerbi.com`. Another is installed from the Windows store and is automatically updated just like any other app from the store. When you realize that Power BI Desktop gets a new release nearly every month, these automatic updates are a real plus, although it can sometimes be annoying when a new release changes things you do not expect. If you want to, you can install both versions on the same computer. The third version of Power BI Desktop, which can also be

> downloaded from the Power BI website, is a special edition for use with Power BI Report Server.

- In addition to these Microsoft tools, there are several community-based tools available to edit semantic models, such as Tabular Editor and DAX Studio. These can even be integrated into Power BI Desktop.

For this book, we will use "plain" Power BI Desktop, as this is a free app that you probably already have. Every reader of this book can easily download Power BI Desktop and use the example files provided in the book's GitHub repo.

Now that we know where to find DAX and which tools are available to work with DAX, let's look at what we can do with it.

Powered by DAX: visual, interactive reports

In discussing the five-layer model, we already briefly touched upon the importance of visual reporting, but effective visual reporting is only possible through the power of DAX-based models.

Creating visual output is paramount to generating real insights versus just a lot of information. Let's take the following example:

Product	Jan	Feb	Mar	Apr
VZ1714IN HELMET VISOR XRR/XRS TRANSPARENT 04 NORM	1,037,535.98	1,212,067.58	624,573.60	779,
VENTURI S500 BLACK	666,929.16	512,042.79	716,421.69	462,
TOP VENTILATION RSF2 RACE SLM	14,249,186.22	166,407.97		
TOP VENTILATION LATERAAL S500R SLM	6,350,391.55	14,066,895.02	8,010,202.22	7,976,
TOP VENT FRONT + VENTURI S500R SLM	82,408,846.32	142,459,033.08	106,023,022.16	87,592,
TOP VENT FRONT + VENTURI S500R BLK	47,244,316.50	50,395,903.12	42,521,083.83	44,585,
TOBY 15.SU12.01 SUZ GSF1200BANDIT N 96-00	23,885,091.99	32,893,481.25	24,675,594.08	25,261,
TOBY 15.KA07.09 KAW ZX-7R/-RR RAC 96-02	4,084,937.31	6,927,763.88	4,814,627.67	3,675,
TOBY 15.DU09.09 DUC 900 S4R 03-04	50,947,610.25	92,611,499.27	84,294,006.91	57,071,
SluitCHAIN LINK DC420D CL	12,480,623.57	33,707,922.13	21,313,689.05	17,389,
SHARK S800 FASHION SILVER/RED/BLACK S	5,588,614.73	8,209,797.12	3,942,769.32	8,617,
SHARK S800 BUTTERFLY BL/WHITE/GOLD M	13,513,204.21	19,457,351.56	16,775,535.52	11,735,
SHARK S500 SHARKY'S 2 GREY/RED/WHITE XS	11,857,347.72	17,712,513.60	10,287,387.60	9,434,
SHARK S500 SHARKY'S 2 GREY/RED/WHITE M	692,520.64	1,499,143.27	815,452.24	693,
SHARK S500 SHARKY'S 2 GREY/RED/WHITE L	1,356,232.38	3,085,298.53	2,260,571.78	1,551,
SHARK S500 POWDER BLACK/SILVER/ANTRACITE XS	3,514,377.33	5,645,034.59	3,183,241.84	2,664,
SHARK S500 POWDER BLACK/SILVER/ANTRACITE S	592,804.80	1,435,307.95	1,141,364.56	749,
SHARK S500 POWDER BLACK/SILVER/ANTRACITE M	4,390,268.14	7,698,559.90	4,561,018.80	3,785,
SHARK S500 FULL MAT DARK GREY XS	2,493,130.27	3,212,519.94	1,706,710.45	2,791,
SHARK S500 FULL MAT DARK GREY XL	1,631,584.19			
SHARK S500 FULL MAT DARK GREY S	1,155,890.25	3,580,362.00	1,953,656.06	1,591,
SHARK S500 FULL MAT DARK GREY M	4,481,103.31	8,419,326.44	4,818,707.77	4,115,
SHARK S500 FULL MAT DARK GREY L	6,761,979.23	12,483,755.84	6,702,361.80	5,738,
SHARK S500 FRACTAL BLACK/YELLOW/WHITE S	4,976,091.12	7,353,569.06	4,423,645.99	3,430,
SHARK S500 FRACTAL BLACK/YELLOW/WHITE M	2,395,952.79	2,963,570.33	2,157,560.82	1,284,
Total	**4,487,160,995.04**	**7,533,291,979.74**	**5,037,199,851.84**	**4,318,206,**

Figure 1.3: Some sales data in a table visual

Can you spot the problem or opportunity this company has? If so, you are very good at numbers! Most people, though, would have trouble with this. Not only are you flooded with numbers, but you would even need to use the scrollbars to see more information.

The following figure presents the same information in a visual way:

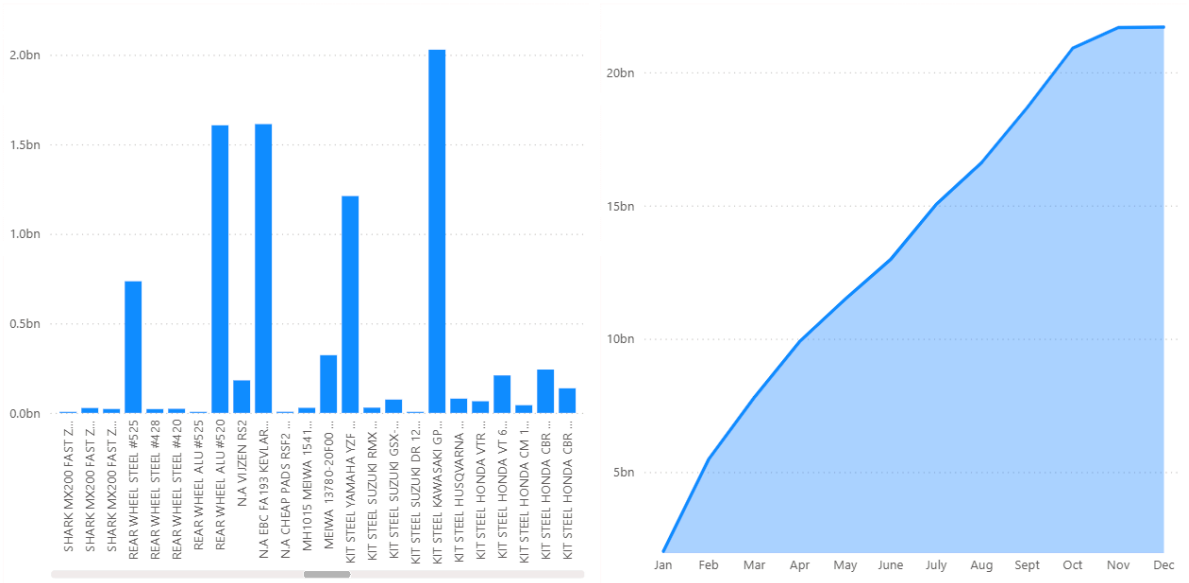


Figure 1.4: Sales data in a visual report

It is obvious now that some products do significantly better than others. That is an interesting and valuable insight: if we can make the other products reach similar performance, the overall results of the company will dramatically increase. Note that this report also *decreases* the amount of information presented: we don't see the numbers by month for every product, but instead, we have a total view of cumulative sales by month.

These insights also lead to something else: you want to know what it is about the large products that makes them so great. Is there a single large customer that ordered these products? Are there specific geographies where the products are sold? And what about the margin on the products; perhaps they sell well because we priced them too low?

This is a fundamental cause-and-effect that happens all the time when you create deep insights through visual reports. *Insights lead to new questions.* The answers to these questions are new insights, which, in turn, lead to other questions.

How can this effect be addressed? The traditional way has been to provide as much information as possible in a single report. The reason for this was that it takes time to render a report. The report user is overwhelmed by the amount of information and has no choice but to export data to Excel to try to make

sense of it all. Power BI reports approach this in a fundamentally different way, made possible by the power of DAX, by adding interaction to reports:

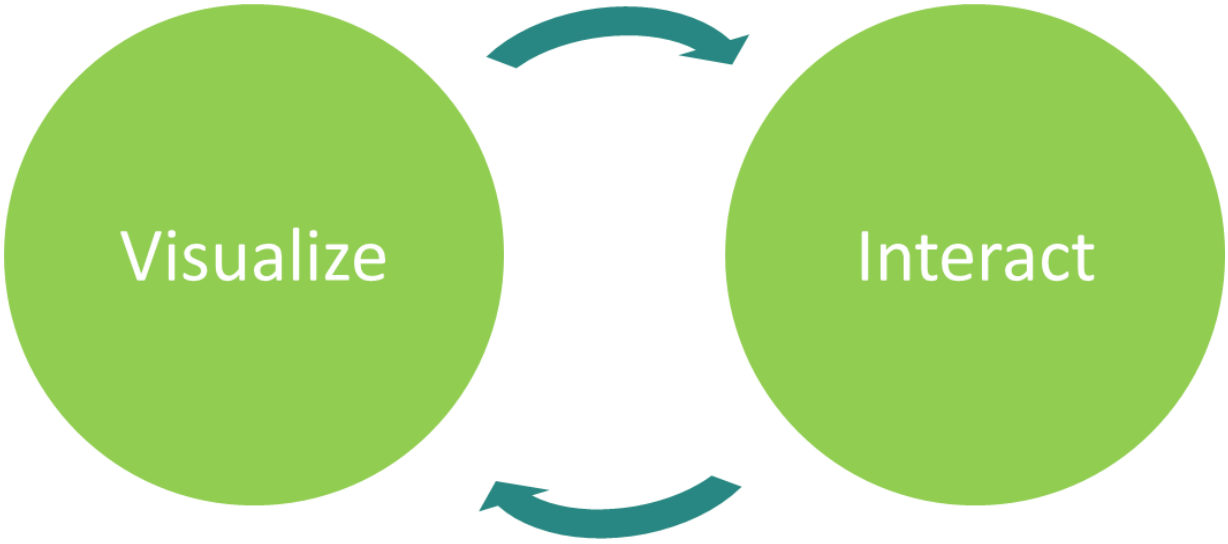


Figure 1.5: The Visualize-Interact cycle

Interaction allows the consumer of a report to dig into the initial insights and find the answers to their follow-up questions, making the move from straightforward reporting to analyzing in an organic way. As a simple example, in the visual report shown previously, the user can click on a product, and the chart with cumulative sales by month will change and show the results for that product only.

For this to work, the report needs to be able to provide new visualizations at the blink of an eye. For a Power BI visual report, which retrieves all content from a semantic model, this means that the model must be equally fast in providing results to the report. The performance of the model is a result of its structure and of the DAX code that you implement. So, there is a direct link between your DAX code and the user experience of your reports!

The data-driven transformation cycle

So far, we have focused on what is needed to go from *raw* data to insights, and the role of DAX-powered semantic models in this process. We have seen that business value does not come from merely connecting and preparing data, but that it is the insights from visual, interactive reports that provide value.

However, no organization will get better from just staring at nice-looking reports. What really matters, of course, is what you *do* with the insights. In other words, you need to take action to reap the benefits of a BI solution. You will also want to measure the effects of these actions, either in an automated way or by letting users enter feedback into some kind of system. The result of that is, again, data.

This leads to a cycle of data-driven transformation or business improvement.

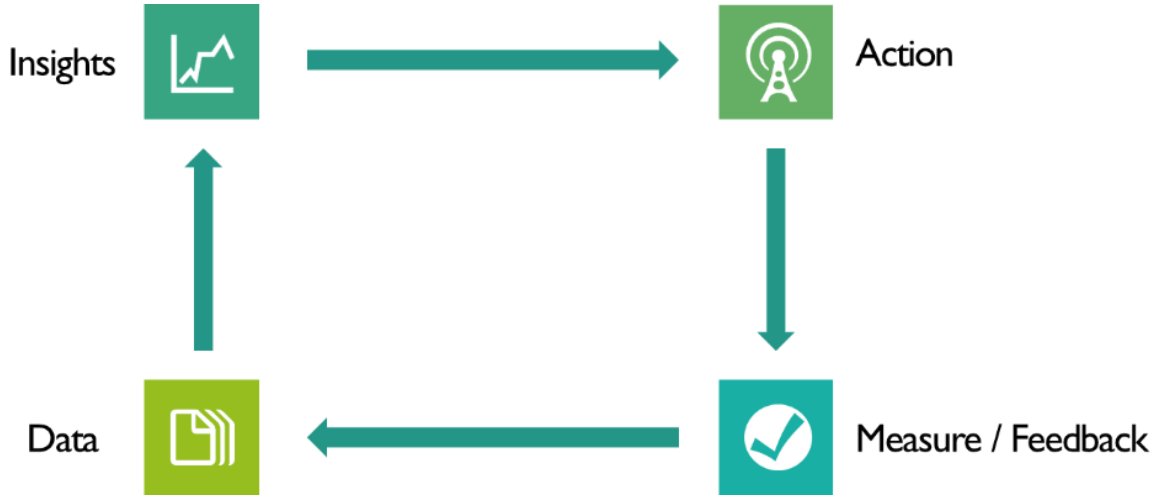


Figure 1.6: The data-driven transformation cycle

While semantic models and Power BI visual reports shine in the process of going from data to insights, they are not designed to cover the other half of the transformation cycle. That part requires different capabilities, such as facilities to enter or update data, and technology to connect systems and people.

One of the options to implement these capabilities is the Microsoft Power Platform. Power Platform offers components such as Power Apps, which provide an environment to develop business applications for adding or editing data, and Power Automate, which allows for automating process flow between a variety of systems, services, and user-oriented applications.

Power Apps and Power Automate can be integrated into Power BI reports. For Power Apps, this means that an app can be embedded in a visual report through a visual that supports interactivity, just like any other visual. For Power Automate, a flow can be triggered through a button in a visual report.

With Power BI being part of the Fabric platform, however, better options arise:

- **User data functions** in Fabric can implement any functionality, such as adding or changing data, triggering a process, or whatever you can imagine (and code).
- Visual reports can call a user data function through a button, while passing through parameter values for the function based on what the report user selected or manually entered. The combination of these report features and user data functions is currently known as **translytical task flows**.
- **Activator** can automatically trigger an event when the results in a visual report are updated and meet predefined conditions. As a response to the event, several things can happen, such as sending an email or Teams message, executing an action in Fabric, or triggering a Power Automate workflow.
- With **Semantic Link**, data scientists and business analysts can leverage the capabilities of semantic models, such as model structure and DAX aggregations, to enrich data that they use for training AI models, for example.
- With **Copilot**, intelligent agents can be developed that reason over semantic models to provide smarter answers to questions from business users.

All these Fabric capabilities can provide new data that is made available to semantic models for further analysis and reporting, thus closing the cycle on data-driven transformation.

In this book, we focus on the DAX language and capabilities specifically, and we will not go deeper into the components discussed previously. However, they are an indication of how important DAX-based semantic models are in the modern Microsoft platform for analytics. So, learning DAX and deepening your understanding of DAX concepts is a valuable time investment!

How to approach solution development

Semantic models and DAX enable a different way to develop BI solutions with a much deeper involvement of businesspeople. This leads to solutions that

deliver insights that optimally add value to the business.

The traditional, IT-centric way to create BI solutions is to start by connecting to data sources and preparing data. At first glance, this sounds like the right thing to do; after all, we need good data to have good and valuable insights.

Figure 1.7: The traditional BI solution development approach

This approach typically materializes in the development of an enterprise data warehouse. The idea of having a data warehouse is to store all the organization's data in a single place and to use that as the foundation for all reporting. It should be clear that this can be a major undertaking, as organizations have many different systems with lots of different data in them. The data warehouse is traditionally implemented as a relational database system, which means that all enterprise data must fit in the database structure, or schema, of the data warehouse.

The variety of data causes the schema of a data warehouse to be highly complex. Moreover, whenever a source system changes or a new system is introduced, the data in the new system must match the schema of the data warehouse, or the data warehouse must be changed to accommodate the new data. Consequently, data warehouse projects are notorious for their duration and high costs. Many careers are built on data warehouses, and, unfortunately, many careers have stumbled on them as well.

There is a more fundamental flaw in the traditional approach. By starting with source systems and working upward through the five-layer model, you miss the invaluable business context about the insights that are actually needed. Even while most data warehouse projects aim to include business needs and contexts in the process, in reality, the business fades into the background in many, and perhaps most, projects that are approached this way. The technical complexity is just too much to let the business be deeply involved. Often, the result is that when the data warehouse is at last finished (or rather, taken into production for the first time), it is already behind the real business needs of that moment.

While waiting for the corporate reports to be available, the organization has already deployed a **shadow IT** tactic to retrieve the insights needed. People take any tool at hand (usually Excel) and any data that they can put their hands on, and build analytics solutions themselves. While this is obviously a solution to a problem (being the urgent need for insights), it doesn’t help the organization in the long run. The quality of the data, and therefore the validity of the insights generated, remains questionable. The risks of taking wrong decisions based on these insights are high.

Using semantic models for BI solution development

Instead of the approach just described, semantic models enable working both upward and downward through the five-layer model.

Figure 1.8: Semantic models enable a different solution development approach

With this approach, we use (Power BI) semantic models not only as the target platform for the solution, but also as a tool to streamline the project itself. By

doing this, you leverage the specific capabilities of Power BI: fast creation of reports providing tangible insights, based on wherever the data is. The nature of semantic models and DAX is foundational for these capabilities.

The approach is based on two basic principles:

- We do not know exactly what we need
- Our data is not correct

The consequence of these principles is that it is impossible to "get it right" the first time. Rather, you should deploy an iterative way of working to fail fast and improve fast.

We do not know exactly what we need

This principle means that you do not expect the business owner, or yourself for that matter, to be able to provide correct specifications of the reports. If you have ever undertaken a BI project, you will recognize this. Even those who claim they know exactly how things should be will overlook some details. And even if they do not, there will be a misunderstanding in how to implement the specifications in the solution, if only because the person developing the solution does not have the same business context.

As a consequence, it is not effective to spend a lot of time gathering requirements or writing down specifications and getting them approved. You just know that the first version of a report will be wrong, so the better way is to do it wrong as fast as possible. As it turns out, it is much easier to point out the errors and flaws (and the good things!) in something that actually exists than to write down all the details in an abstract way.

You can formalize this approach by working in multiple iterations with a joint session at the end of each iteration to discuss ever-richer versions of a report and gather feedback. We call these **business design sessions** to highlight their nature: they are not feedback sessions nor demos, but joint efforts to achieve the right results. Depending on your skills in Power BI and DAX, you can take two, three, or more days to work on an iteration. The results of a business design session are taken as input for the next iteration.

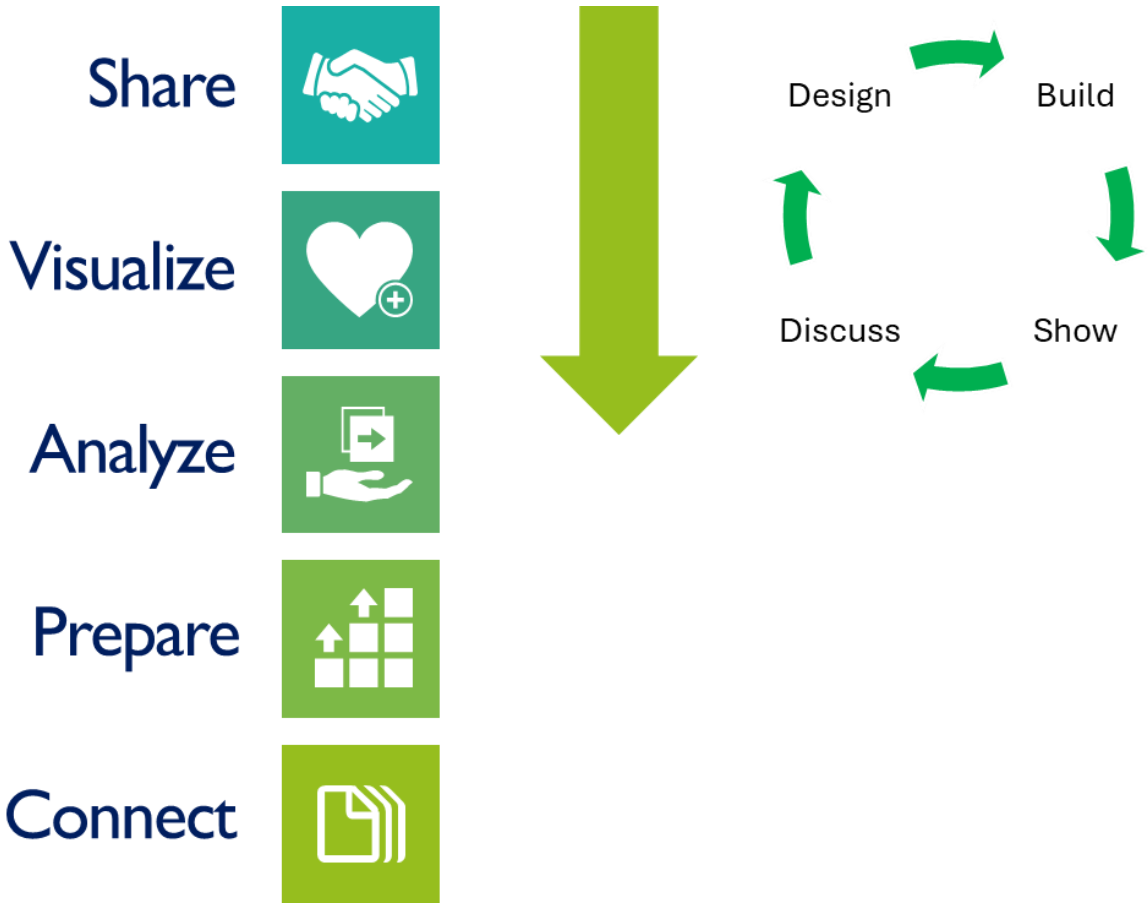


Figure 1.9: Iterative design through business design sessions

The result of this process is a report that is truly jointly developed and optimally aligned with business needs. Moreover, the report and underlying semantic model provide an exact specification of the data that the lower layers in the five-layer model should provide.

Our data is not correct

This second principle is most probably not new to you. Of course, our data is not correct! The reason for this is that real-world business processes are much messier and more complex than anyone could model. IT systems are designed to support specific business processes, based on what that process should look like. But this poses a fundamental dilemma. Either the system implements a strict data quality policy, in the sense that one can only enter data that conforms to the designed process (and therefore the system will fail to capture each and every instance of the process), or the system provides

flexibility to capture all instances of the business process, and necessarily allows for data that does not fit the designed ideal flow.

The latter option is what is chosen mostly. This means that a typical business system allows for exceptions, custom fields that users leverage to enter custom information, and different kinds of bypasses. As a consequence, data from business systems does not always conform to your expectations. And things become worse when your business data is in spreadsheets or other files!

In traditional BI solutions, the messiness of data is hard to detect and to solve. The reason is that either these BI systems only contain aggregated data, or the technology does not support exploring data on a detailed level by business users. This is where Power BI comes in: the technology of semantic models is so powerful that in many situations, data can be loaded into the model without being aggregated. The visual and interactive reports provide insights through sophisticated (DAX) aggregations, while allowing you to zoom in to the deepest levels of detail.

In an iterative approach to BI solution development, results after the first few iterations are typically full of errors. Knowledgeable businesspeople are generally capable of discovering these errors quite easily. At first, flaws in the implemented aggregations are discovered this way, but in later iterations, the quality of the data comes to light. Being able to see detailed data in a Power BI report is a huge help in driving adoption of and trust in new BI solutions.

Summary

In this chapter, we discussed the field of business intelligence and the central role of analytical models in modern BI solutions. Semantic models are ideally suited for use as such models, not least because of the power of DAX.

You have learned about two capabilities of DAX that have a profound impact on the way BI solutions can be designed and developed. One, DAX enables sophisticated calculations for a variety of aggregations of data, aggregations that in the past needed a lot of preparation of customized data. Therefore, DAX allows for shifting focus from data (with all the tedious work involved) to the logic to generate business insights.

Two, DAX, as a language, is set up in a way that allows businesspeople, who are mostly familiar with Excel, to work on the BI solution themselves, to varying degrees. This means that much more alignment with business priorities can be achieved.

As semantic models and DAX are two sides of the same coin, it is important to know how to balance between the two to achieve optimal results. Simply put, you should do with DAX whatever DAX is particularly good at and not solve these things in data. And vice versa, you should not use DAX to prepare or generate data.

The next two chapters elaborate on these topics. In *Chapter 2*, we discuss the dos and don'ts in designing semantic models. *Chapter 3* focuses on how to use DAX for the best results. Later in this book, you will find many examples, mostly based on real-life projects, that demonstrate the power of DAX and the balance between DAX and semantic models.

2

Model Design

Join our book community on Discord

https://packt.link/EarlyAccessCommunity

If you aspire to be effective with DAX, it all starts with designing a good semantic model. In this chapter, we address some modeling-related topics that are important to understand to create strong model designs.

The topics covered in this chapter include the following:

- The way the Power BI engine stores data
- Choosing the right data types
- Relationships
- What structure to strive for in your models
- Architectural options in semantic models

To achieve good models, you will need to adapt to the appropriate way of thinking. This change is needed when you have a background in Excel, but some adaptation is required when you have a background in relational databases as well. When you are used to working in Excel specifically, the concept of relationships in a semantic model takes time to comprehend, but even when you have a database background, there are many things that are different as well. One of the difficulties in designing for Power BI is that the concepts seem familiar to database professionals, when in reality, they are fundamentally different. We will, therefore, discuss many of these differences in this chapter.

Columnar data storage

The power of the semantic model comes primarily from a smart data storage mechanism in combination with in-memory processing. Semantic models are, in fact, databases in the sense that they organize and store data. But the

internals are very different from other database technologies you may be familiar with.

Relational databases

The traditional, corporate way of working with data is with a **relational database management system** (**RDBMS**) such as Microsoft SQL Server. In an RDBMS, tables of data are defined with a fixed number of data columns per table. Each column must have a data type, such as integer, text, or decimal number. From this information, the RDBMS can derive how much space is needed to store a single row of data, or record, and how many rows can be stored in a disk-based data file. This concept makes an RDBMS an effective choice for applications that *process transactions*, such as sales transactions from a web shop or transactions in a company's financial ledger.

The concept of an **index** in an RDBMS makes finding a specific record fast and efficient, which means that processing transactions on existing records can also be implemented effectively with an RDBMS. Tables can be linked through relationships (hence the name RDBMS), which makes sure that data in these tables is consistent; for example, an RDBMS will block the insertion of a sales transaction involving an unknown customer.

The RDBMS is a mature concept, and many optimizations for it have been invented and implemented. As a result, most traditional analytics platforms rely on RDBMS technology as well. However, the needs of an analytical solution are completely different from those of a transactional system. In analytics, you are typically not interested in retrieving all data from a single row, but in retrieving data from many rows at the same time, and only from one or a few columns. To retrieve information from a single column, the RDBMS still needs to read complete rows from storage. Also, aggregating data from many rows is something an RDBMS is not designed to do, and so it will be relatively slow.

Figure 2.1 visualizes this: storing data by row (identified by the numbers) leaves no possibility of efficiently retrieving all the required column's values.

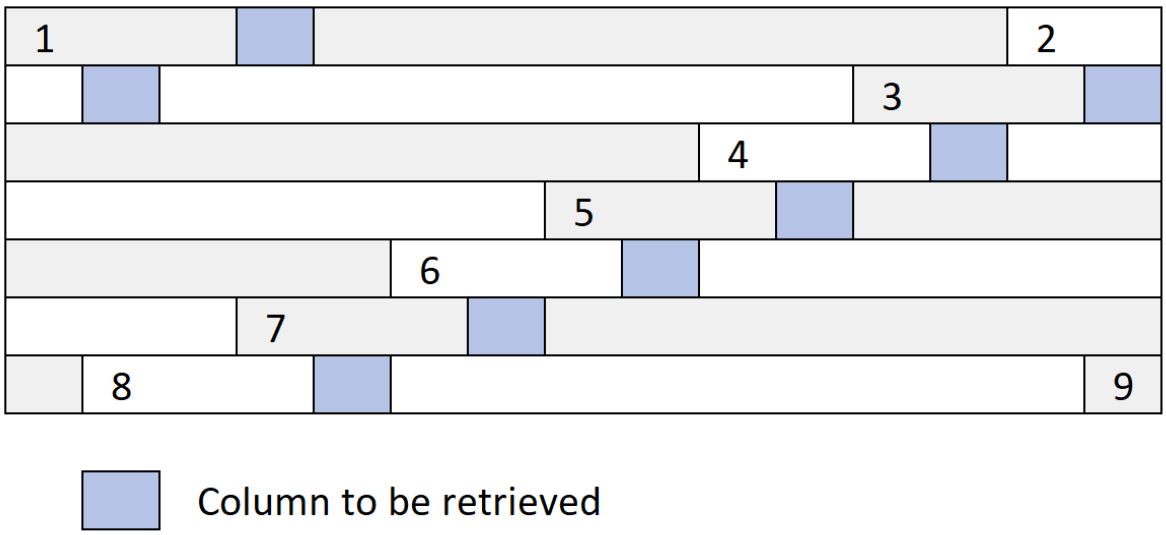


Figure 2.1: Retrieving column values from row-based storage is inefficient

Columnar databases

The semantic model in Power BI transposes the concept of an RDBMS by storing data not on a per-row basis but per column. The rationale behind this is that in an analytical solution, often only a couple of columns need to be read from storage, but for all available rows. This is most efficient when all data in the same column is stored near each other.

Another reason is that in real-life situations, many values in a single column are the same; for example, millions of sales transactions are for only thousands, or tens of thousands, of products. A columnar database can, therefore, highly compress data by only storing a specific value once and keeping track of which rows it belongs to.

The high compression rate that columnar databases achieve opens up the possibility of keeping the whole database in memory. This means that all data resides in the internal memory of the computer or server that the database runs on, instead of being stored in files on disk. Keeping data in memory accelerates data retrieval even more.

The columnar model means data aggregation is done very efficiently. Instead of, for instance, summing all separate values in a column, the columnar database engine can simply take each distinct value and multiply that by the number of rows the value appears in. In short, the semantic model's database

engine is designed from scratch to support the typical workloads that come with data analytics: handling large amounts of data with specific characteristics and performing aggregations and calculations across these.

There are, however, some caveats here. For example, you still need to know which values in different columns go together in one row. It is not enough that product 103 has been sold; you need to know how much it was sold for, to which customer, and on what day. To enable this, the model must keep lists of pointers that keep track of which value goes in which row, for any column. The overhead from this obviously grows when more columns are added to a table. "Narrow" tables are therefore more efficient than "broad" tables in a semantic model.

Later in this chapter, we will discuss more considerations for designing efficient semantic models.

Data types and encoding

The Power BI semantic model works with a limited number of data types. Choosing the right data type for your data is important, as it determines the way your data is stored, or encoded, and how efficiently the model can process your data. The following is a list of all the data types recognized by Power BI:

- **Text**: The most generic data type is `Text`. Virtually all data can be stored using this data type. When loading data through Power Query, the generic Power Query data type `Any` is converted to `Text` in the Power BI model. This causes numerical columns to be stored as `Text` when you forget to explicitly make the type conversion in Power Query. (You can, of course, change the data type in the model, which will automatically add a type change step in Power Query.)
- **Whole Number**: The `Whole Number` data type is used, as you would guess, to store whole numbers. Because of the way the semantic model stores and compresses data, this is one of the most efficient data types available.
- **Decimal Number**: This data type is the most versatile of the number data types. It can store almost anything, from very small to very large numbers, and with fractional values. You can store numbers with up to

15 digits. Due to the nature of this data type, arithmetic calculations can come with rounding errors.

- **Fixed Decimal Number**: The `Fixed Decimal Number` type, sometimes called `Currency`, is a data type for storing fractional number values with four decimals. You can store numbers with up to 19 digits, including the 4 decimals. This means that this data type has a smaller reach than the `Decimal Number` type. The `Fixed Decimal Number` type is typically used to store currency amounts, but it can be used for any value that doesn't need many decimals. Fixed decimals are suitable for precise arithmetic calculations within the limits of the data type.
- **Date/Time**, **Date**, **Time**: The semantic model stores date and time values with a similar structure to Excel. This means that values are decimal numbers, with the whole number part representing the date, and the decimals representing the time.
- The difference compared to Excel is in the base reference date: in a semantic model, the number 1 corresponds to December 31, 1899, while in Excel, the number 1 corresponds to January 1, 1900 (both at midnight). The decimal adds the time as a fraction of a 24-hour day; for example, the value 2.5 represents January 1, 1900, at noon. You have three choices regarding your date/time data. The `Date/Time` data type stores both dates and times. The `Date` data type stores only dates. The `Time` data type only stores the time part. Note that these differences are mainly about the formatting of the values stored, although data is changed when you change the data type. For example, changing a date/time value to the Date data type removes the decimal part of the value representing the time. Likewise, changing the same value to the Time data type removes the whole number part (or replaces it by 0), leaving only the decimals.
- **True/False**: The `True/False` or `Boolean` data type can only store two values: `True` or `False`. Though of limited use, data is stored very efficiently using this type simply because of the small number of unique values.
- **Binary**: The `Binary` data type is used to store data that cannot be represented as text, such as image data or documents. It is not possible

to perform aggregations or calculations with this data type, but it may be used, for example, for storing images that need to be included in reports.

Selecting the best data types for your data is paramount for an efficient model. The semantic model aims to store the unique values in a column as efficiently as possible. We have long passed the time when we needed to care about bits and bytes when working with computers, but when designing a model, it still helps to consider the individual 0s and 1s that computers work with. The model will determine how many bits are needed to store your values. After all, as all data is kept in memory, every bit saved helps—even more so when you deal with millions of rows of data.

As an example, suppose you have a column with whole number values between 0 and 10. In digital notation, the number 10 is represented as 1010, or 4 bits. The set of values can thus be encoded in words of 4 bits, directly representing the values. This approach is called **value encoding**. In our modern computers with 64-bit processors, using 4 bits is already much more efficient than storing the values as 64-bit numbers.

Value encoding is possible for data types that are whole numbers "under the hood": `Whole Number`, `True/False`, and also `Fixed Decimal`. `Fixed Decimal` benefits from the *fixed* number of four decimals: it is basically a whole number divided by 10,000. The DAX engine is, in fact, capable of doing basic transformations before doing value encoding, such as subtracting the same number from all values encountered.

Other data types cannot be directly represented as whole numbers, and the database still needs to find a way to store values in a minimum number of bits. This is done by keeping a numbered list of values and storing the numbers instead of the values. This is called **hash encoding**. Hash-encoded columns work less efficiently than value-encoded columns, as the database needs to do the translation between numbers and values each time the column is used.

It is important to keep in mind that the semantic model chooses the best encoding given the data type and values in a column. This means that even when you use a whole number-based data type, you could still end up with hash encoding. To give an extreme example, when you have a column

containing not only numbers between 0 and 10, but also the number 1,000,000, the number of bits needed to store these values directly is so much larger that the engine will decide to use hash encoding instead.

We see this happen in real projects quite regularly, in particular when it comes to storing dates. Suppose you have a table with employees, containing the date they entered the company but also the date they quit. For all current employees, the end date is, of course, empty; though, often, a specific future date is set instead of keeping the field empty. This is a valid approach, but if you choose a date such as December 31, 9999, you will certainly lose the benefits of value encoding for your date columns. Instead, use a value that is not too far into the future, such as December 31, 2029 (depending on your scenario, of course).

Relationships

One of the most misunderstood elements in Power BI semantic models is the concept of relationships. Whether you are working with Power BI coming from an Excel background or you have been educated in relational databases, relationships in semantic models require an approach that is different from what you are familiar with.

Data in Excel

Let us zoom in on data in Excel first. The concept in Excel that is closest to a database is that of Excel tables. You could consider an Excel table as a "flat" database. This way of storing data has many disadvantages.

As an example, *Figure 2.2* shows data as it may be stored in an Excel worksheet:

Full Name	Job Role	Date of Birth	Order Date	Amount
Taylor, Giuliana	Sales Manager Finance	20 May 1975	2-4-2018	120
Campbell, Aki	Business Development manager	28 October 1980	3-4-2018	244
Brown, Arun	Senior Consultant	16 February 1968	5-4-2018	3154
Stewart, Amaris	System Developer	2 May 1982	31-3-2018	355
Stewart, Amaris	System Developer	2 May 1982	8-5-2018	521
Stewart, Amaris	System Developer	2 May 1982	12-4-2018	78
Taylor, Giuliana	Consultant	20 May 1975	11-5-2018	224
Collins, Chet	Finance Manager	22 November 1965	28-3-2018	1528
Clark, Elke	Consultant	5 September 1993	9-4-2018	1205

Figure 2.2: A table in Excel

This table contains order amounts and dates for sales orders, which are sold by employees. There are multiple issues with flat databases:

- Clearly, all information about an employee (such as job role and date of birth) is duplicated with each order sold by that employee. So, a lot of information is stored redundantly, which takes up a lot of storage space.
- Storing information multiple times increases the risk of errors in the data.
- When some attributes of an employee change, such as their job role, the changes must be effectuated in all rows associated with that employee.
- Things get worse when there are multiple attributes of the same kind for an entity. In our example, Giuliana seems to have two job roles. Each of Giuliana's sales orders is associated with only one job role, which may or may not make sense from a business point of view; when we aggregate sales by job role, the result for "Consultant" will only contain one of Giuliana's orders.
- One of the biggest issues arises when data is loaded from multiple sources. Let's imagine that we have not only sales data but also targets. It takes a lot of effort to combine data from these sources into one flat table of data. In fact, Excel users spend most of their time setting up the single flat data table they need to work with pivot tables.

In Excel, there is really no workaround for these issues; that is, except when you start using Power Pivot, the semantic model implementation in Excel.

Before discussing the Power BI approach, let's look at how data is handled in a relational database.

Data in relational databases

In a relational database, the data is separated into multiple tables. Typically, these tables are associated with entities that are relevant to the organization, such as `Customer`, `Employee`, and `Product`. Each row in a table has an identifier, or **key**, that allows for consistent referral to the row from other tables. In a table with sales orders, for instance, the keys for `Customer` and `Product` can be included instead of all the attributes for the customers and products involved; see *Figure 2.3*.

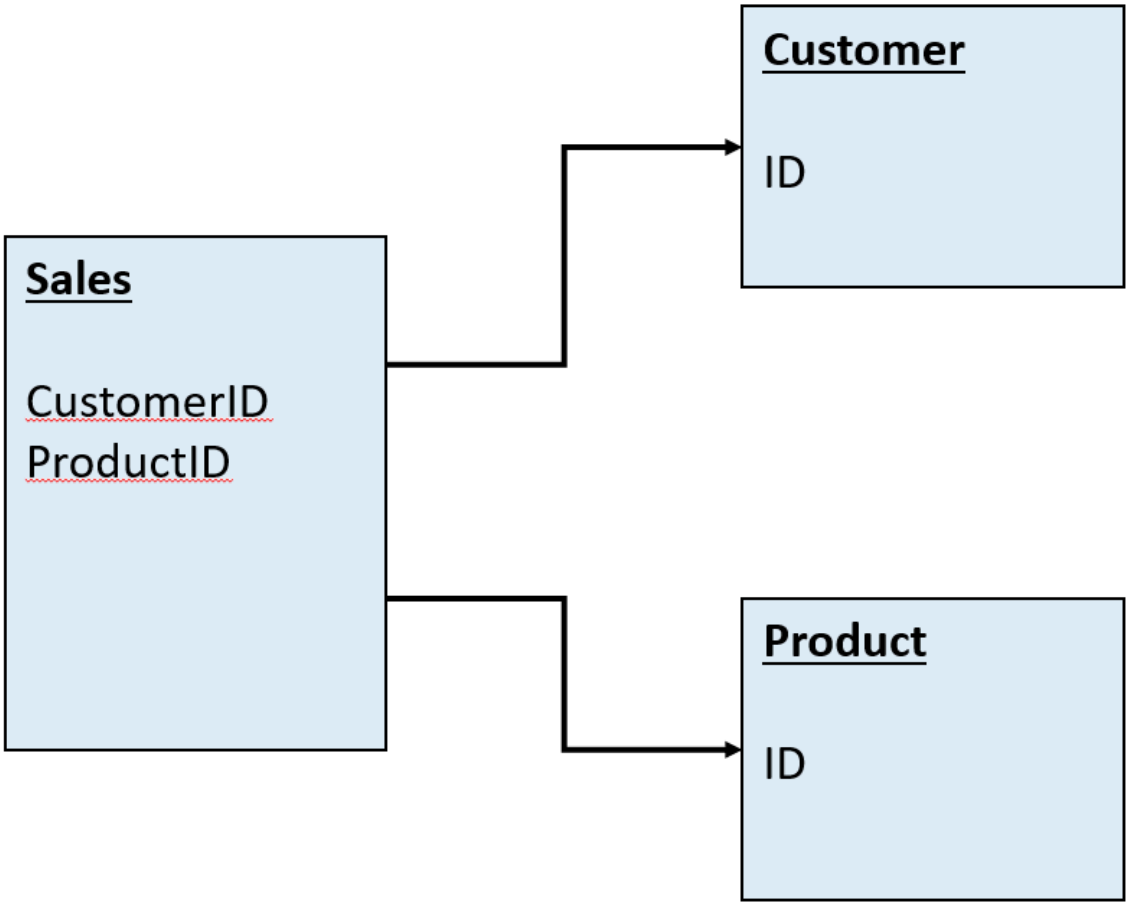


Figure 2.3: Relationships in an RDBMS

Obviously, it doesn't make sense to register a sales order without a customer key or with an unknown key. That is why, in an RDBMS, you can define **relationships** between tables to denote which columns in a table refer to the keys in other tables. The RDBMS then enforces that a column on which a relationship is defined only contains known keys in the related table. It blocks the insertion or mutation of a record with a value that doesn't appear in the related table. In other words, an RDBMS relationship acts as a *constraint* on the data stored and is used to enforce **referential integrity**.

Power BI's relational model

In a Power BI semantic model, relationships are links between tables. At first sight, they are comparable to relationships in an RDBMS, but in reality, they are fundamentally different.

The basis of a relationship in a semantic model is a data table with a unique key. Another table with the same key values can be related to it, but in this table, the key values need not be unique. This type of relationship is called **one-to-many**, meaning that there is one table where a key appears exactly once, and another table where the same key can appear many times. You may call the column with unique keys a **primary key** column, and the column with non-unique keys a **foreign key** column, just like with relational databases.

The structure of a semantic model, with its tables and relationships, can be viewed in a diagram view, such as the one from Power BI Desktop in *Figure 2.4*.

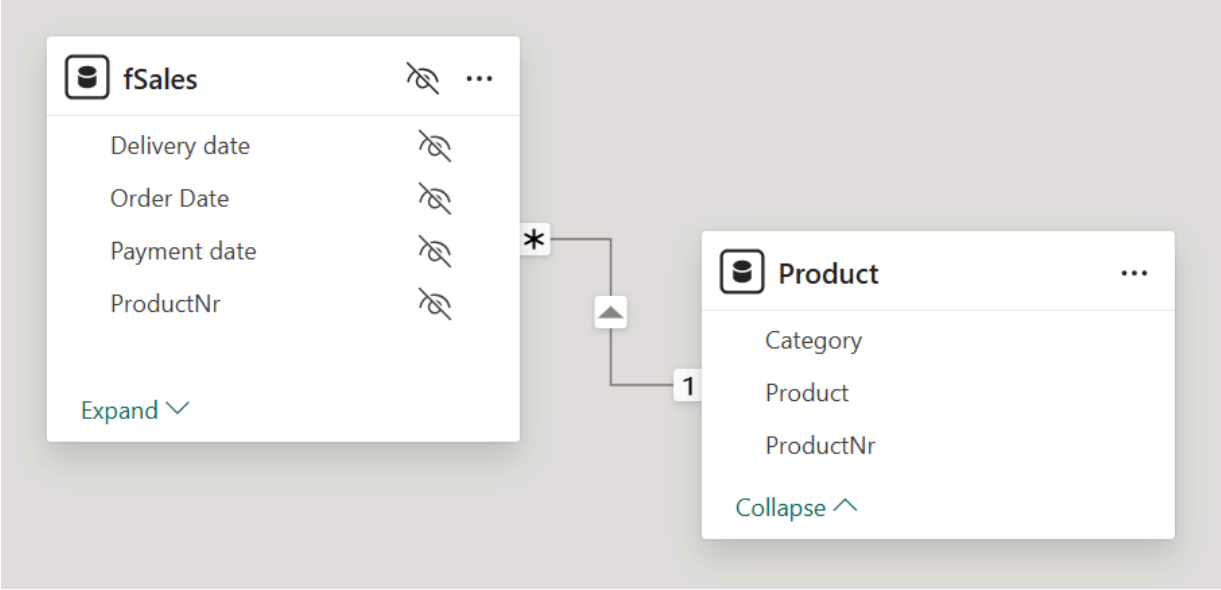


Figure 2.4: A relationship between two tables in a Power BI model

There are two fundamental differences between relationships in a semantic model and relationships in an RDBMS. The first is referential integrity. While a relationship in an RDBMS acts as a constraint on the data, a relationship in a semantic model does not do this. Frankly, the model couldn't care less whether or not your data is consistent. When the foreign key column contains values that do not appear in the primary key column, the relationship can still be defined.

The model will link each unknown foreign key value to a blank row (as in *Figure 2.5*). This row will not be visible in the model but can become visible in a report.

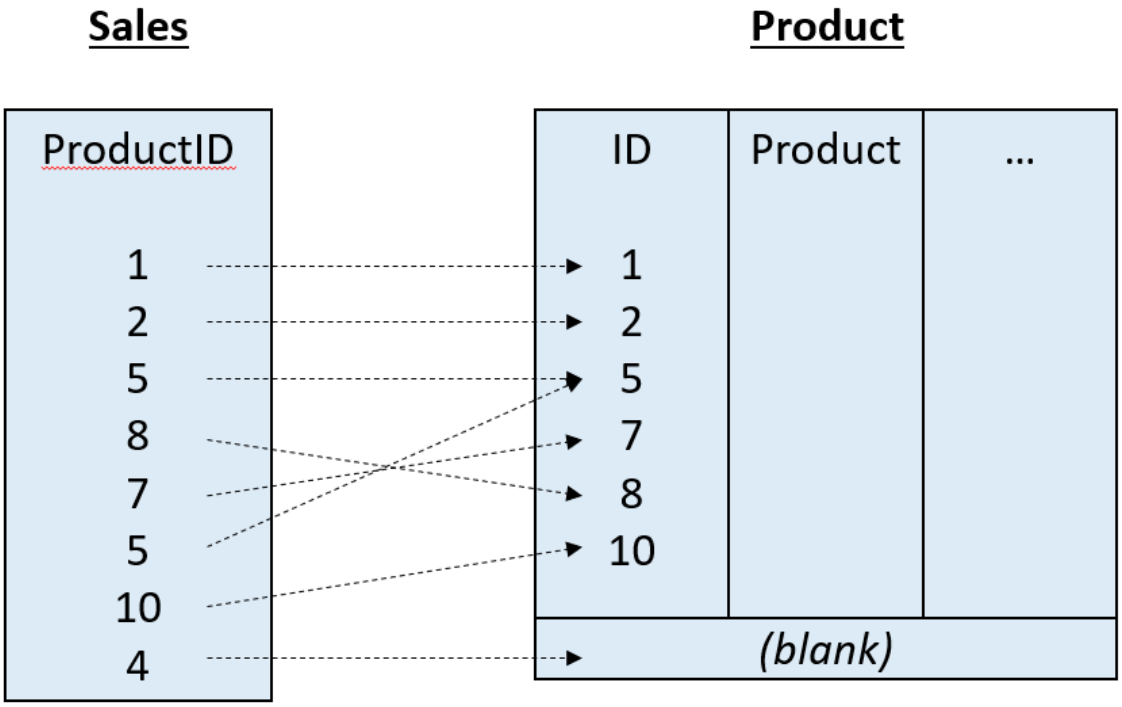


Figure 2.5: Unknown values are related to a blank row

One advantage of this is that you don't have to worry about the order in which data tables are loaded or refreshed, whereas in an RDBMS, this is something to consider carefully. The drawback, of course, is that you must be careful when creating relationships, specifically if you do that through dragging and dropping fields in the diagram view. Power BI will not complain when you drop a field on the wrong target, creating a relationship that doesn't make sense whatsoever.

The other fundamental difference between relationships in semantic models and in relational databases is **filter propagation**. A relationship in a semantic model actively filters data. More specifically, when some rows in one table are selected, the related rows in the other table are automatically selected as well (in the direction of the arrow on the relationship line; see again *Figure 2.4*). This is a core design principle of the semantic model, with many implications for designing DAX calculations.

In an RDBMS, a relationship does not have this behavior. When querying an RDBMS, the user must specify which tables to combine, or *join*, on which (primary key and foreign key) columns. This makes querying an RDBMS very

flexible, but it also forces the RDBMS to do a lot of work for each query. As a result, retrieving data from many tables and therefore processing a lot of relationships can become slow.

Relationship properties

When you create a relationship between tables in a semantic model, you can set several properties on the relationship that drive its behavior. These properties are directly related to a relationship's main purpose, filter propagation.

Active and inactive relationships

To enable relationships to do filter propagation, there must be an unambiguous connection between tables. Suppose that for a sales transaction, both the order date and the payment date are registered. When relationships exist from both columns to a `Calendar` table, and a row is selected in the `Calendar` table, it would not be clear which sales transactions should be selected: the transactions that were ordered on that date, those that were paid, or both?

To deal with this, the model allows only one active relationship between two tables. This also applies when tables are connected via other tables: only a single active **relationship path** is allowed. In *Figure 2.6*, it is the relationship between the `fSales[Order Date]` and `Calendar[Date]` columns. When you define a relationship that creates a *second* path, that relationship is made inactive.

Inactive relationships are shown in the diagram view with a dashed line. In *Figure 2.6*, the relationship between the `fSales[Delivery date]` and `Calendar[Date]` columns, as well as the relationship between `fSales[Payment date]` and `Calendar[Date]`, is inactive. This is not clear from a static image, but when you hover over the relationships, the columns involved in that relationship are highlighted.

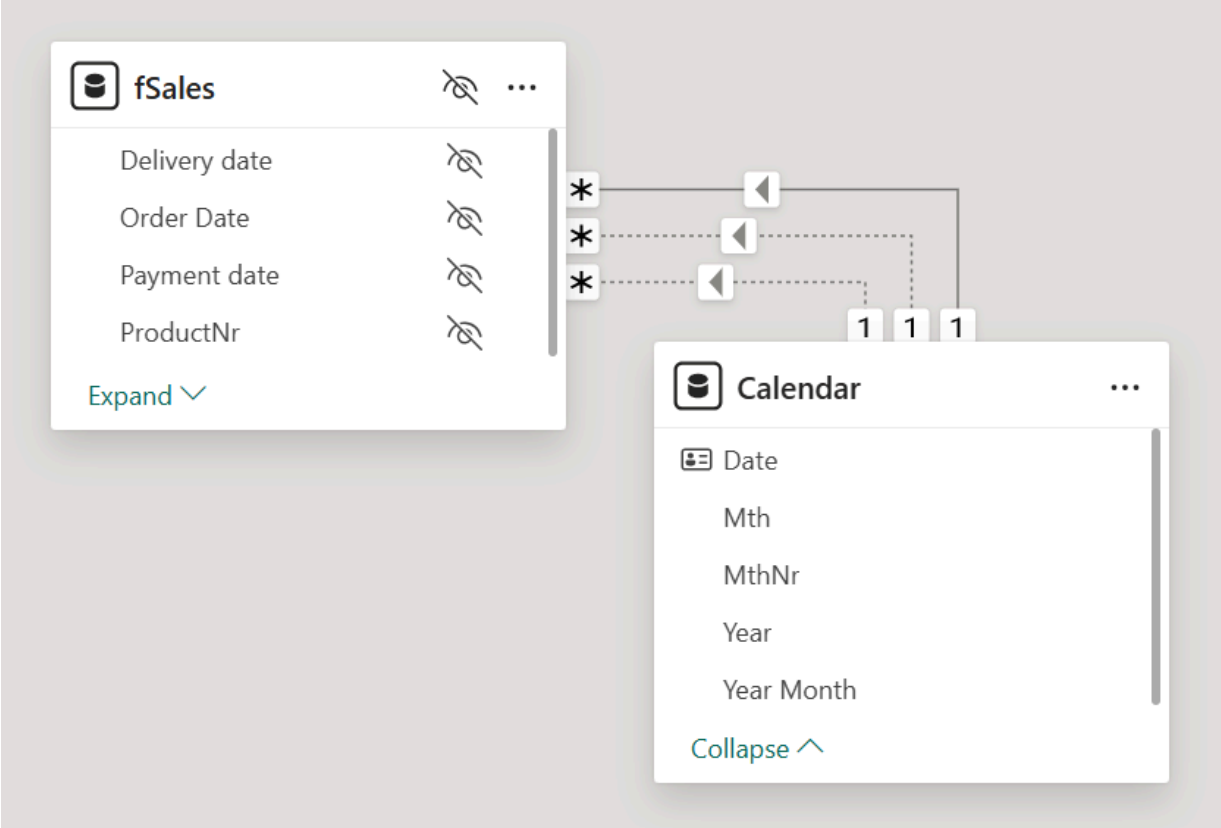


Figure 2.6: One active and two inactive relationships

An inactive relationship can be activated for a specific calculation using the `USERELATIONSHIP` DAX function.

Cross filter direction

Filter propagation through relationships is normally done only from the primary table to the foreign table. In the diagram view, the direction of filter propagation, or cross filtering, is indicated with a small arrow in the middle of the relationship line. You've seen that in *Figure 2.4* already.

It is possible to change the **cross filter direction** so that filter propagation happens in both directions. This is done in the **Edit relationship** dialog or the **Properties** pane by setting the cross filter direction to **Both**. This may seem like a convenient setting to apply by default, but do not do this! In fact, double, or two-way, cross filtering relationships should be used only in specific scenarios. Try to avoid them otherwise, or you will end up with strange behavior in reports, many inactive relationships, and highly complex DAX calculations.

One specific scenario in which to use relationships with double cross filter direction is when implementing many-to-many relationships. For instance, assume a model with customers and branch offices (*Figure 2.7*). Customers are serviced from one or more branch offices and, conversely, branch offices service multiple customers:

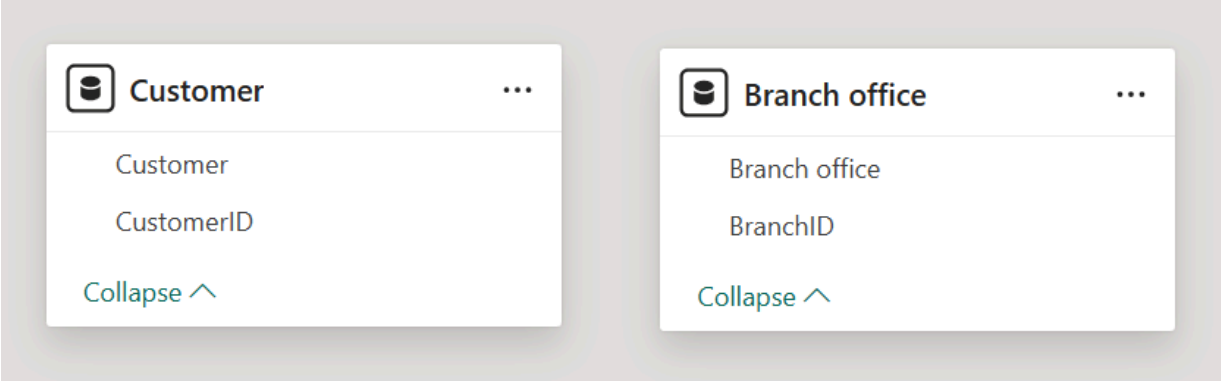


Figure 2.7: Customer and Branch office

Both the `Customer` and `Branch office` tables have unique key columns, but neither of them can have a column with a foreign key: each row must be linked to multiple rows in the other table. The solution to this is to have an intermediate or **bridge** table with all relevant combinations of customer keys and branch office keys. Now, two relationships can be created from the combination table to the `Customer` and `Branch office` tables, respectively; see *Figure 2.8*.

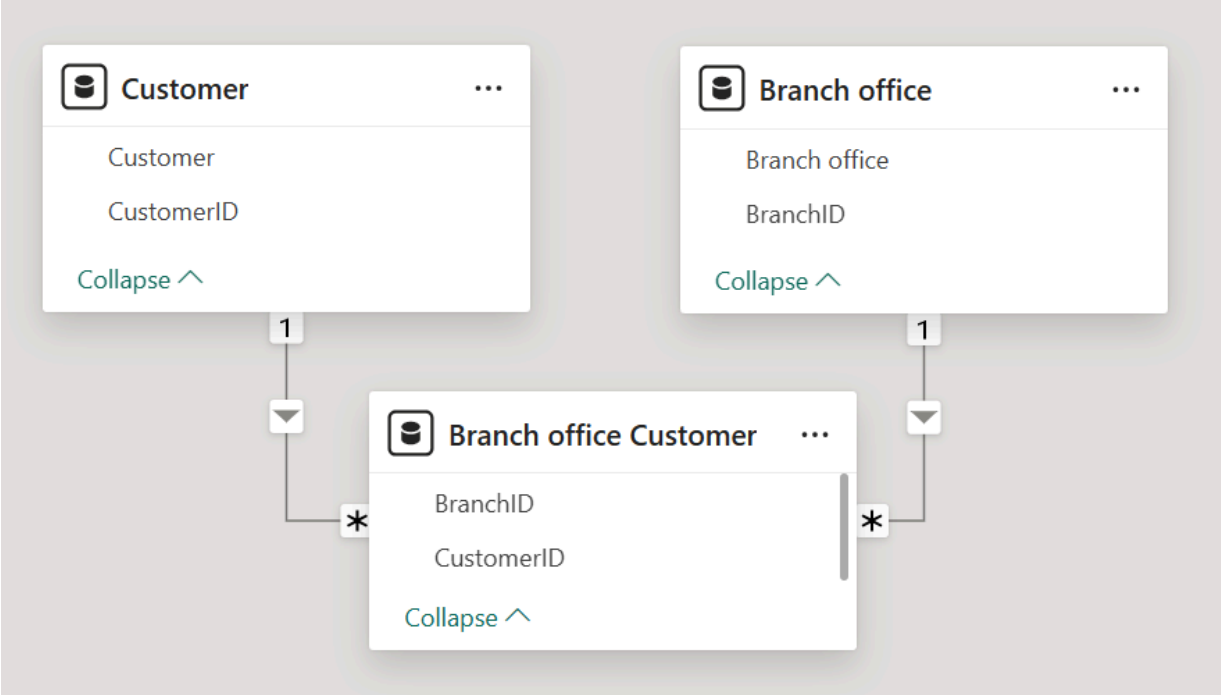


Figure 2.8: A bridge table

However, the relationships do not yet correctly cross filter from `Customer` to `Branch office` and vice versa; you can select a row in the `Customer` table and the relationship will propagate the selection to the bridge table. But this selection is not propagated to the `Branch office` table.

The solution is to set both relationships to the **Both** cross filter direction. Now, when selecting a row in the `Customer` table, the relationship propagates this selection in the default direction to the bridge table, and the other relationship propagates this to the `Branch office` table. Conversely, when selecting a row in the `Branch office` table, the relationship propagates the selection to the bridge table, and the selection is then propagated to the `Customer` table; see *Figure 2.9*.

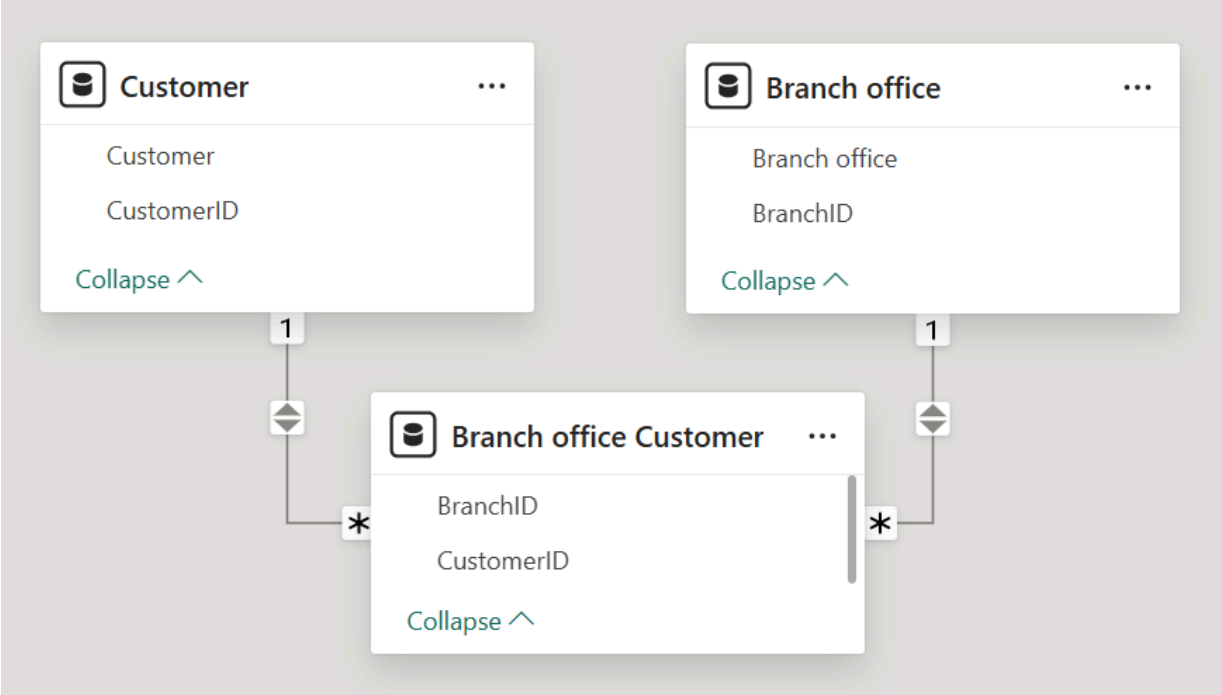


Figure 2.9: Implementing a many-to-many relationship through a bridge table

The caveat here appears when you relate one of the tables, say the `Customer` table, to a table with sales transactions, `fSales` (see *Figure 2.10*). Necessarily, with each sales transaction, the `Customer` key is recorded. The setup allows for selecting a customer—say, `MegaBike`—and viewing the total sales for them. However, it is not possible to view sales for `MegaBike` from a specific branch office: when you select a branch office, the total sales for `MegaBike` are returned whenever they (`MegaBike`, that is) are linked to the branch office. When you report on total sales by branch office, the `MegaBike` sales will be part of each branch office that `MegaBike` is related to.

In this scenario, the better modeling option is almost always to relate both the `Customer` and `Branch office` tables to the `fSales` table directly, without using the bridge table to link both tables together.

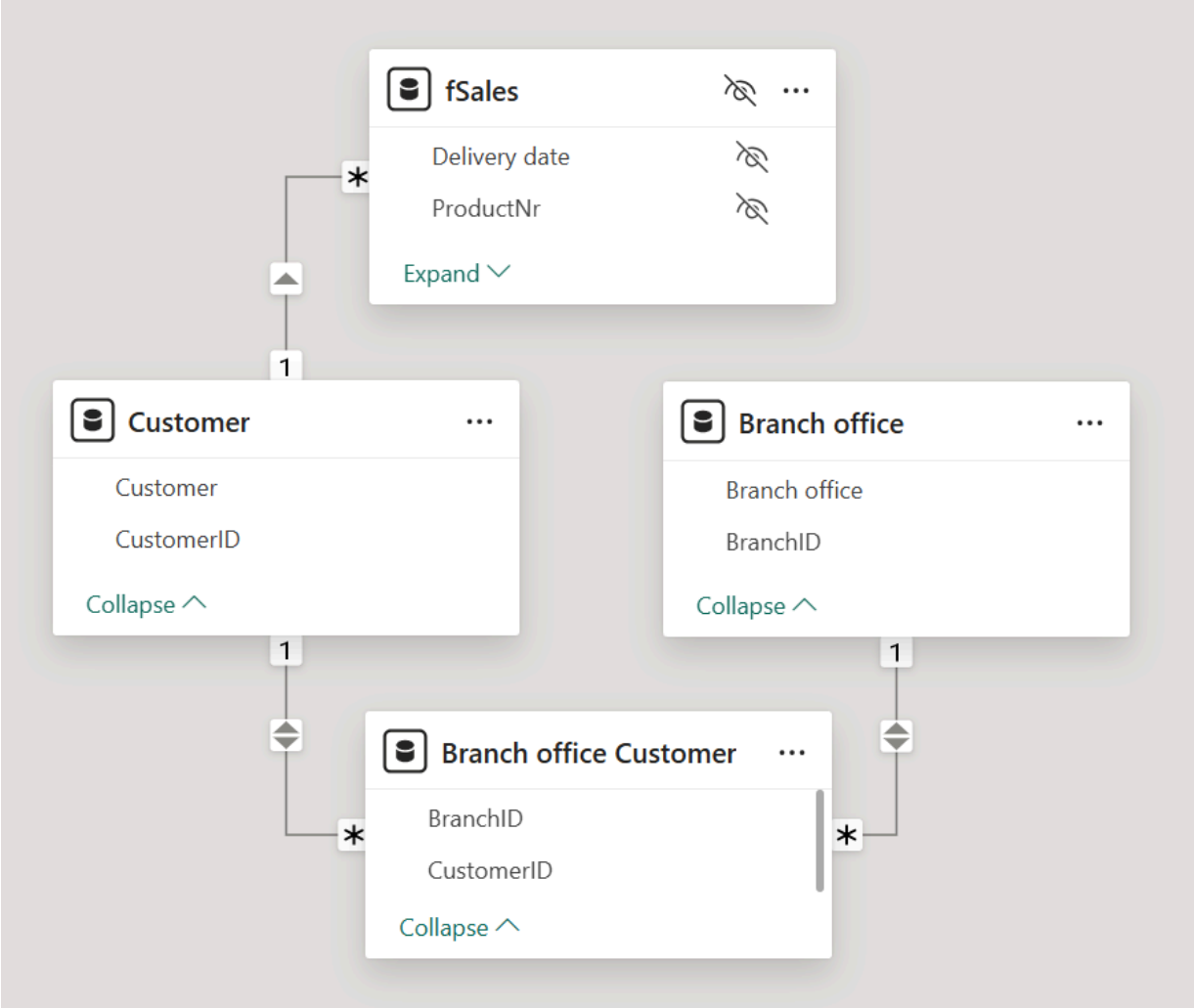


Figure 2.10: Using cross filtering between the Branch office Customer and Customer and Branch office tables

Cardinality

The default relationship in a model is **one-to-many**, with a table containing a (unique) primary key, and another table containing the same values as foreign keys, which are not unique. The official name for this relationship property is the relationship **cardinality**.

Relationships can have other cardinalities as well. A trivial one is **many-to-one**, which is really the same as one-to-many but with the position of the two tables reversed.

A relationship can have **one-to-one** cardinality, meaning that the keys are unique on both sides of the relationship. One-to-one relationships have a

double cross filter direction by default. Consequently, the two tables act as a single table in almost all circumstances. You should avoid one-to-one relationships in your model: instead, combine the two related tables into a single table unless you have specific reasons to keep them separated (see *Chapter 9*, *Working with Auto-Exist*, to learn what these reasons could be).

The last option for relationship cardinality is **many-to-many**. In this case, neither of the two related tables contains unique keys. Again, you may have specific reasons to use this kind of relationship. We strongly advise against using many-to-many relationships, as these will easily lead to bad modeling. The *RDBMS principles to avoid in Power BI models* section later in this chapter further elaborates on many-to-many relationships.

Limited relationships

When you create a many-to-many relationship, you will find that the diagram for this relationship looks slightly different, like in *Figure 2.11*. The "gaps" in the relationship are an indication that the relationship is *limited*. This means that the semantic model cannot implement the relationship using the column store, but instead, through a filter that enumerates all the selected values in the relationship's column.

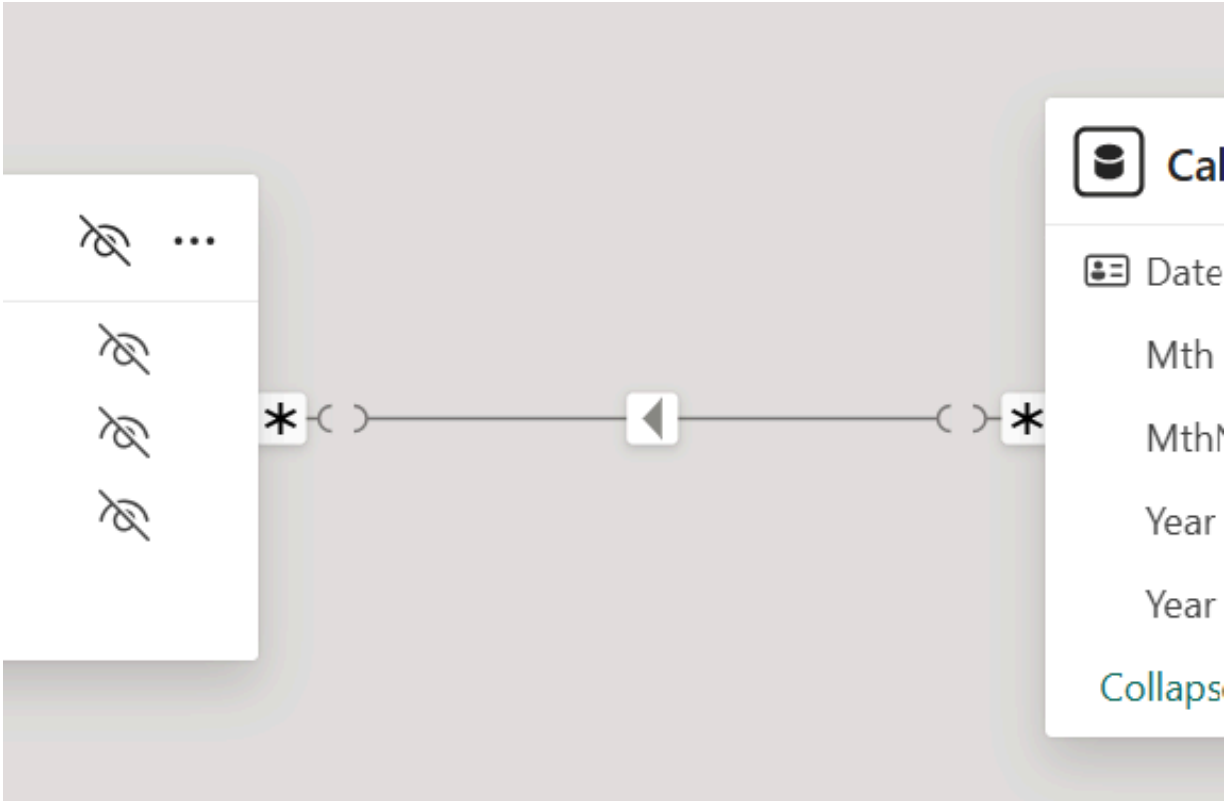


Figure 2.11: A limited relationship

Although you won't have to do this yourself (it is a relationship, after all), it is a computationally expensive operation that will negatively affect your model. Moreover, DAX functions that rely on traversing relationships generally don't work well with limited relationships. You will encounter some examples of this later on.

There are situations where limited relationships are inevitable. More on this later in this chapter, in the section titled *Architectural options in semantic models*.

Effective model design

The concepts of relationships and filter propagation enable powerful analytics in Power BI semantic models. It is important to think about the design of a model: which tables should the model contain, and which columns need to be in those tables? Which relationships are needed? In short, what is the overall structure of the model? The choices you make in model design will determine what results the model will be able to deliver.

Star schemas and snowflakes

A best practice for analytics using relational databases is to work with a specific database structure, known as a **star schema**. The basic ideas of star schemas apply to semantic models as well. *Figure 2.12* shows the structure of a star schema.

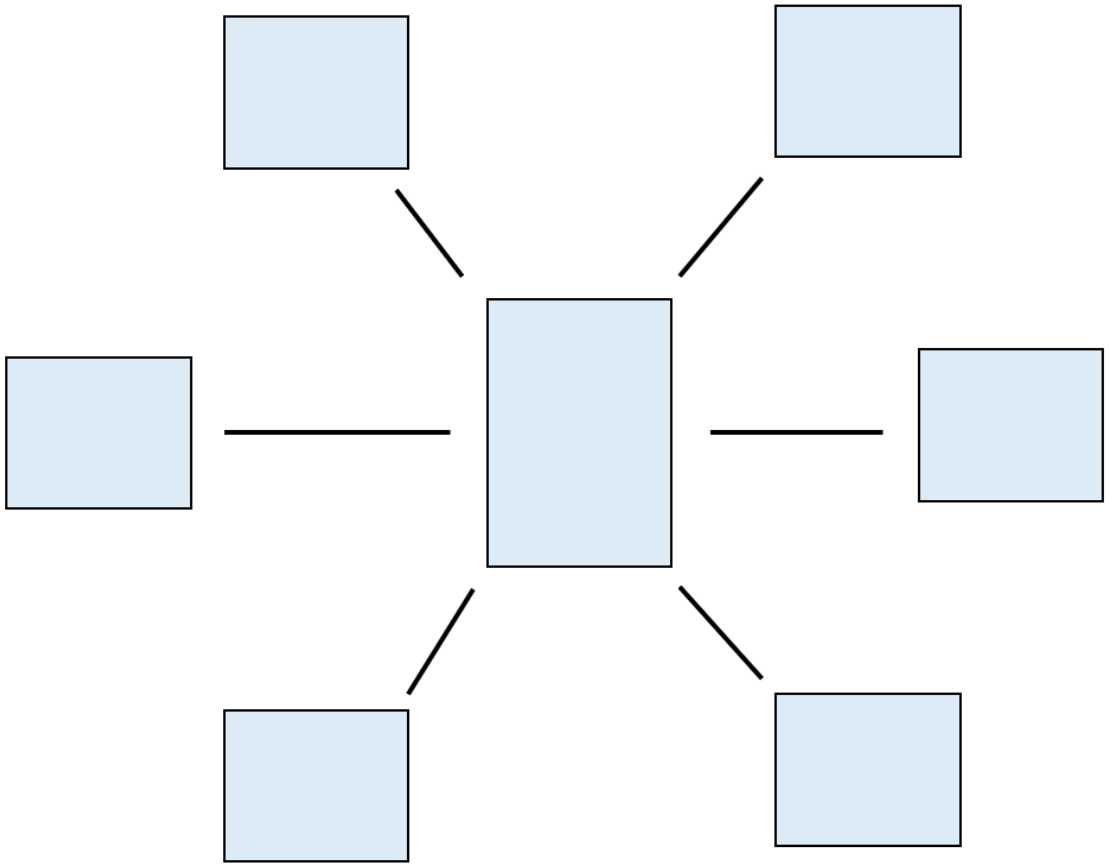


Figure 2.12: Generic star schema structure

The central tables in a star schema model are the **fact tables**. These tables contain things that have happened, will happen, or should happen, such as sales transactions, financial ledger transactions, customer inquiries, student enrollments, and sales opportunities.

Through foreign key columns, the fact tables are related to tables describing different entities relating to the facts, such as customers, products, cost centers, students, and dates. In the star schema concept, these tables are called **dimensions**; in a semantic model, however, we prefer to call them **filter tables**, for the following reasons explained.

The columns in filter tables are used to filter the results in a report, by using them as row labels in a matrix or table visual, on a chart axis, or as a slicer field. A fact table contains the data that is aggregated in the report. Each key

value can appear multiple times in a fact table, corresponding to multiple facts that happened on the same day, or for the same customer, and so on.

In a pure star schema model, there are no relationships between filter tables. When filter tables are related to other filter tables, the resulting model structure is called a **snowflake**; see *Figure 2.13*.

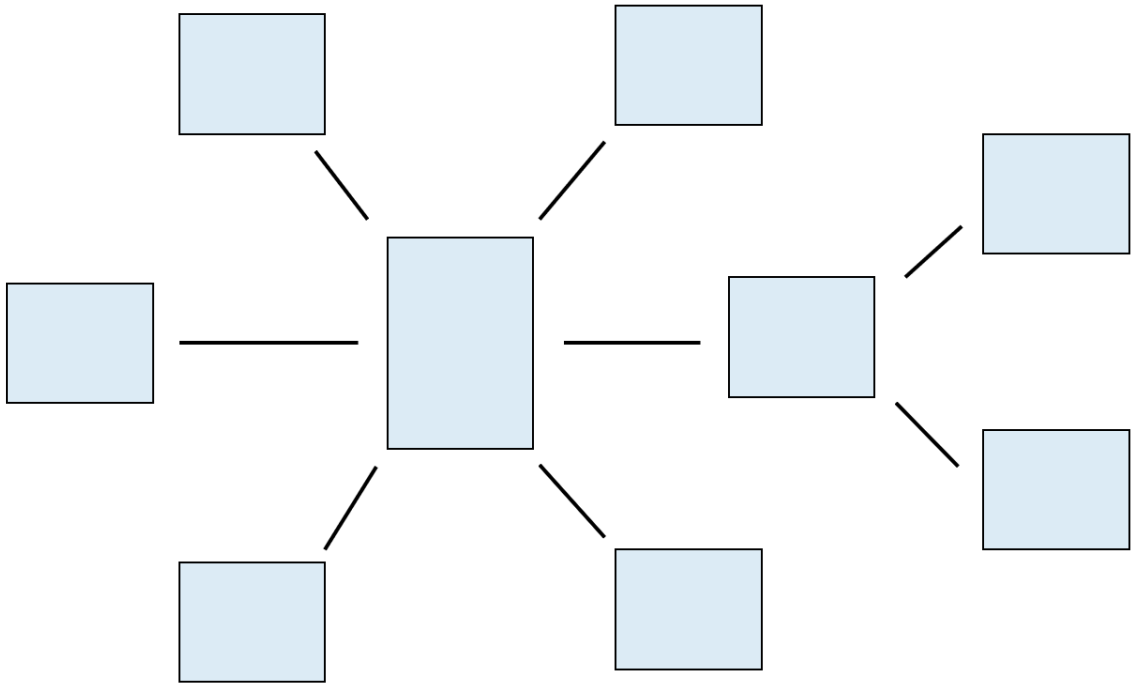


Figure 2.13: A snowflake schema

Note that both star schemas and snowflakes are structures that vastly differ from the database schemas used for transactional databases! It is never a good idea to simply copy the database structure from your transactional system into a semantic model. After all, a transactional database is optimized for exactly that: processing transactions. This is a totally different workload when compared to analyzing data.

The issue with star schemas

Among RDBMS professionals, snowflake schemas are generally considered to be an inferior design. A lot of design effort is put into creating a pure star schema. Many Power BI consultants come from an RDBMS background and apply what they learned in relational databases to Power BI semantic models.

Consequently, you will hear "*Build a star schema!*" all the time when gathering information on semantic modeling.

The somewhat provocative title of this section is meant to spark a discussion about what is really important in designing a semantic model. The fact is, a semantic model is not a relational database; what is confusing, however, is that many concepts and terms used look similar to the RDBMS world. This invites the Power BI model designer to apply learnings from relational databases to Power BI models. The problem is that this generally leads to suboptimal models.

It is important to realize that the concept of star schemas was developed when there was no such thing as a columnar database. An RDBMS star schema minimizes the number of joins to be solved when querying the database, which is important as relational databases tend to get into trouble when needing to join many tables at once, on a lot of rows of data. This is the typical workload of a data warehouse, which was traditionally used as the source for reporting. By "traditionally," we mean before the time of Power BI models—nowadays, the data warehouse, when available, is just the source of data for Power BI models, and, in importing data to a model, hardly any join will be needed.

The question "*why a star schema?*" is typically answered in the context of "*as opposed to a normalized, transactional schema*." Indeed, business intelligence, with its need for aggregations on many, many rows of data, is a totally different workload from transaction processing, with its need for inserting or updating individual rows of data while guarding the consistency of the data. For analytics, a star schema-based model is an absolute necessity.

Many people, however, translate this, in a rather purist way, to "*do not use a snowflake model*." Or, put differently, each dimension table should be directly related to a fact table. While there may be reasons for this in a data warehouse that is queried directly for reporting, it cannot be stated generically for Power BI models. The technology of the Power BI model works far better with relationships, and, on top of that, the compression of data is so strong that using a snowflake model does not need to be a problem. As a rule of thumb, a star schema is a good starting point when designing your model, but don't bother spending a lot of effort to avoid having a snowflake model.

Why this long exposition on star schemas versus snowflake schemas? This is because it is the primary indication of cluelessly applying traditional data warehouse ideas to Power BI. In our consultancy work, we regularly have to deal with IT departments doing this, and it takes a lot of time and effort to explain that, indeed, Power BI is fundamentally different. We deliberately use different terminology for some elements of a Power BI solution to underline the differences and to make things easier to understand for business users.

In the next section, we discuss several ways in which suboptimal Power BI solutions appear from applying traditional RDBMS and data warehouse principles.

RDBMS principles to avoid in Power BI models

In the previous section, we warned against applying learnings from the RDBMS world to Power BI models mindlessly. In this section, we discuss several specific examples.

Interdependent dimensions

In data warehousing, a **dimension** is a table containing descriptive attributes about facts stored in a fact table. The name *dimension* is derived from the concept in mathematics and physics; here, a dimension is an independent parameter that describes an object, such as height or width.

This, as well as the term **multidimensional modeling** to describe the old way of doing analytics (before column store solutions existed), suggests that dimensions in a data warehouse are meant to be *independent entities*. We encounter a lot of data warehouse schemas, however, with dimensions that are closely related. For example, you may encounter a `Customer` dimension table as well as a `Market Segment` dimension table. If a customer can belong to multiple market segments, the dimensions are indeed independent; but in many organizations, a customer belongs to a single segment.

This is not an issue in a data warehouse, but in a Power BI model, it is. Imagine you have a Power BI report with slicers for market segment and customer. Your users will rightfully expect that when they select a segment, the customer slicer will only show the customers for the segment chosen. In other words, your model needs to propagate a filter on `Market Segment` to the `Customer` table, and vice versa. With a default one-to-many relationship

with a single cross filter direction, this is not going to happen; the solution is to enable both-ways cross filtering on the relationships, resulting in a model like the one in *Figure 2.14*.

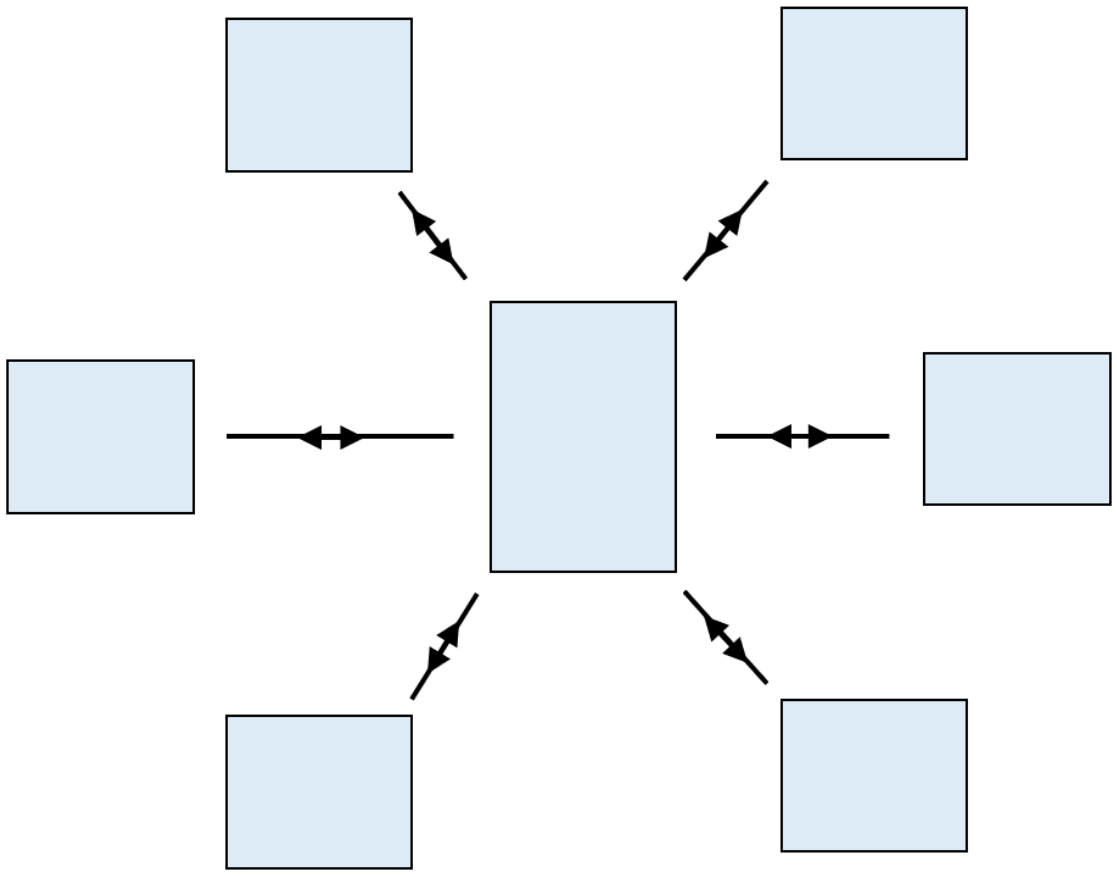


Figure 2.14: Interdependent dimensions require both-ways cross filtering

However, this is a severe case of bad modeling in Power BI, and you will find out why when you try to add a second fact table; it is impossible to do that while keeping all relationships active. A better design would be to cluster filter tables that belong together and let only one of them have a relationship with the fact table (with a single cross filter direction). If needed, the relationships between the filter tables in a cluster can be implemented with a double cross filter direction, as in *Figure 2.15*.

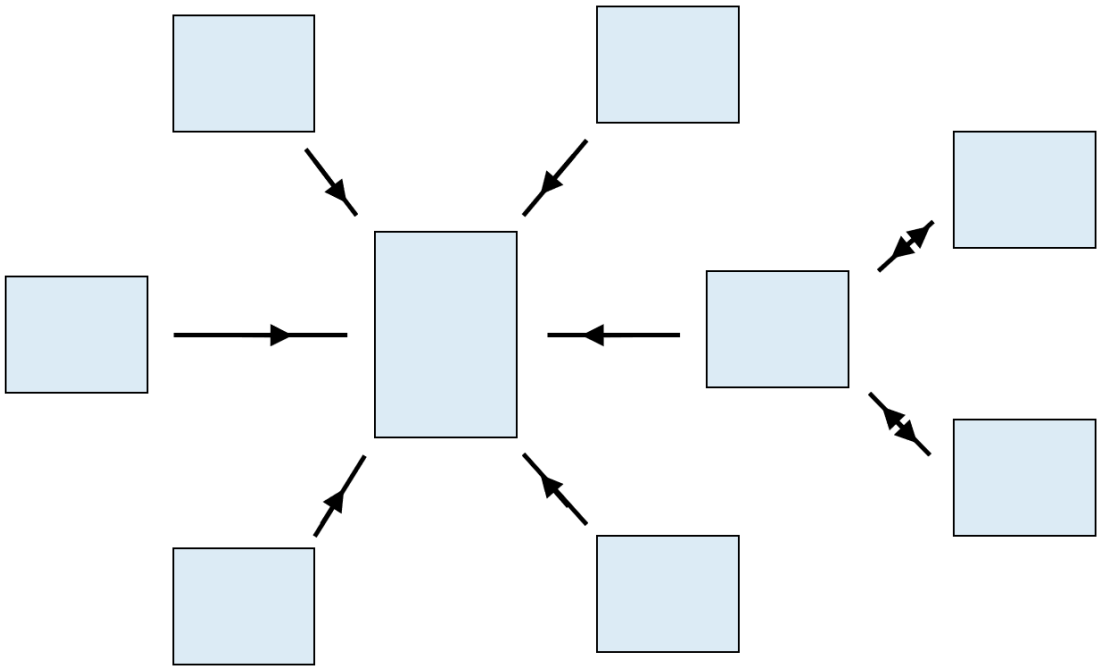


Figure 2.15: Clustered filter tables

A big advantage of this is that multiple key columns in the fact table can be eliminated.

NOTE

In this book, we use the name *filter table*, and not *dimension table*, for three reasons: first, to avoid the suggestion that we are doing traditional RDBMS-based modeling; second, because multiple filter tables may belong to a single cluster and thus are not independent; and third, to use a name that makes sense for the business user and better aligns with the core concept of filters in Power BI.

You could, of course, argue that the filter tables in a cluster could be combined into one large table, which would turn the model into a proper star schema. Indeed, you could, but you do not necessarily need to do this. Moreover, there are reasons not to do it: you can spend your time better by addressing business questions, you may have other fact tables with a different granularity needing to relate to one of the filter tables specifically (such as a target per

segment), and the entities may be more easily recognizable to business users than when combined in a single large table.

One fact table only

This one is quite basic: sometimes, we meet people who have heard about the star schema requirement and interpret it as "*you should have a single fact table*." While this would solve the problem with many double cross filter relationships, it requires a lot of work creating that single fact table and leads to a fact table with a lot of columns. There is no problem with having multiple fact tables in your model!

Data warehouse as the single source of truth

From the preceding discussion, it should be clear that there is no technical necessity for Power BI to have a (relational) data warehouse in place with a thoroughly designed database schema. As the data warehouse would only serve tables of data to semantic models, you do not really need the **schema-first** architecture of a relational data warehouse. Indeed, you could work with a simpler **data-first** architecture, such as a data lake. This is exactly the approach in the Microsoft Fabric platform with its strong emphasis on data-first data management in OneLake.

In many situations, however, you will find that a data warehouse is available. This usually comes with the requirement that "all business logic must be implemented in the data warehouse." From the point of view of Power BI, this is not the best approach.

The main flaw in this policy is that a data warehouse has only one way of communicating with the outside world: in the form of data. A report typically contains data that is aggregated in either a basic or highly sophisticated way (later chapters in this book are mostly about advanced ways of aggregating data). The fact is that many results needed in a report cannot be established by standard aggregations such as a sum or average of values in a column. It is, therefore, impossible to implement all business logic in the data warehouse.

Many data warehouse implementations at least aspire to implement as much business logic as possible. The problem here is, again, that the data warehouse can only communicate in the form of data. This leads to fact tables with many columns, each aimed at a specific business rule or aggregation. But

what you do not want to have in a semantic model is fact tables with many columns!

If you want to have a single source of truth, a semantic model may actually be the better option as it is capable of providing both data and aggregation logic. Specifically in the Microsoft Fabric platform, semantic models can be used as a central hub for data and business logic. Not only can these models be used to power visual reports, but other services can tap into their data and business logic through DirectQuery from other semantic models, semantic link, and programmatically through APIs. More on that later in this chapter.

Using many-to-many relationships

One thing you should definitely avoid at all costs is creating a direct relationship between two fact tables. As fact tables seldom contain columns with unique values, this relationship will be of many-to-many cardinality. (If your fact table does contain a column with unique, or nearly unique, values, you should ask yourself whether your model needs this column.)

Not only will such a relationship lead to unexpected results, as filter propagation is hard to track this way, but the model will struggle with performance as well. This is because there are probably far too many rows being related. The performance impact of a relationship is highly correlated with the number of unique values in the primary key, or one side of the relationship. Do not let this number become too big; we use a maximum of about 100,000 rows as a rule of thumb.

Another, barely better, use case for many-to-many relationships is to relate fact tables with filter tables that have a different granularity. For instance, if your model contains a `Product` table with a `Category` column that groups multiple products, sales facts are probably stored at the product level. Targets, or sales forecasts, may be given at the level of product category. The model allows creating a many-to-many relationship between the target fact table and the `Category` column in the `Product` table (*Figure 2.16*).

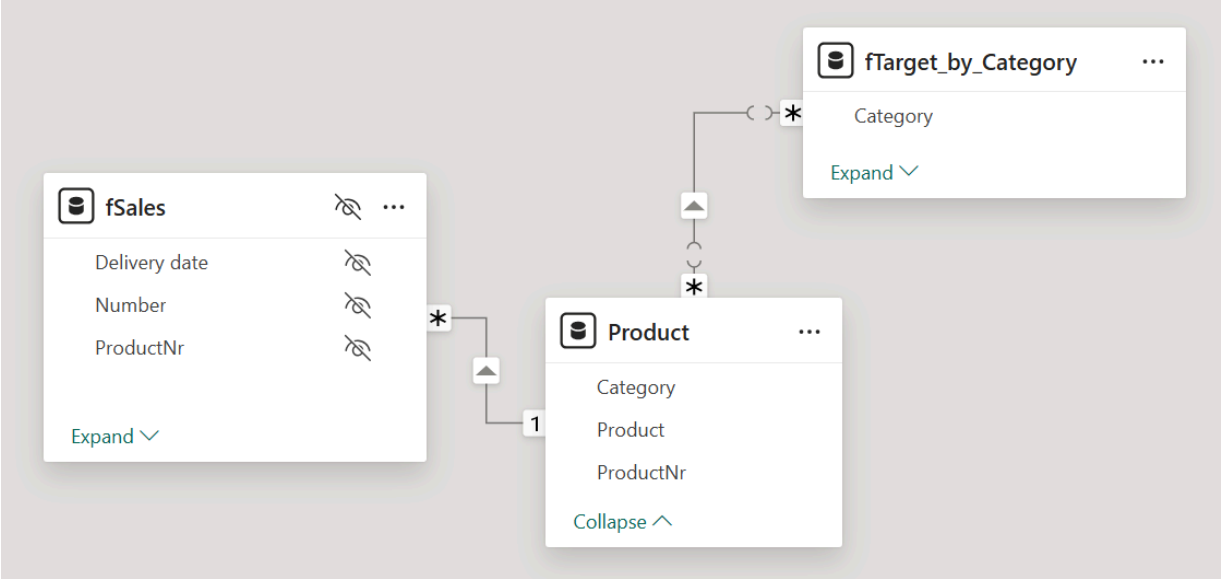


Figure 2.16: Using a many-to-many relationship

While this works without problems, we do prefer implementing this using an intermediate or bridge filter table containing categories as unique rows (*Figure 2.17*).

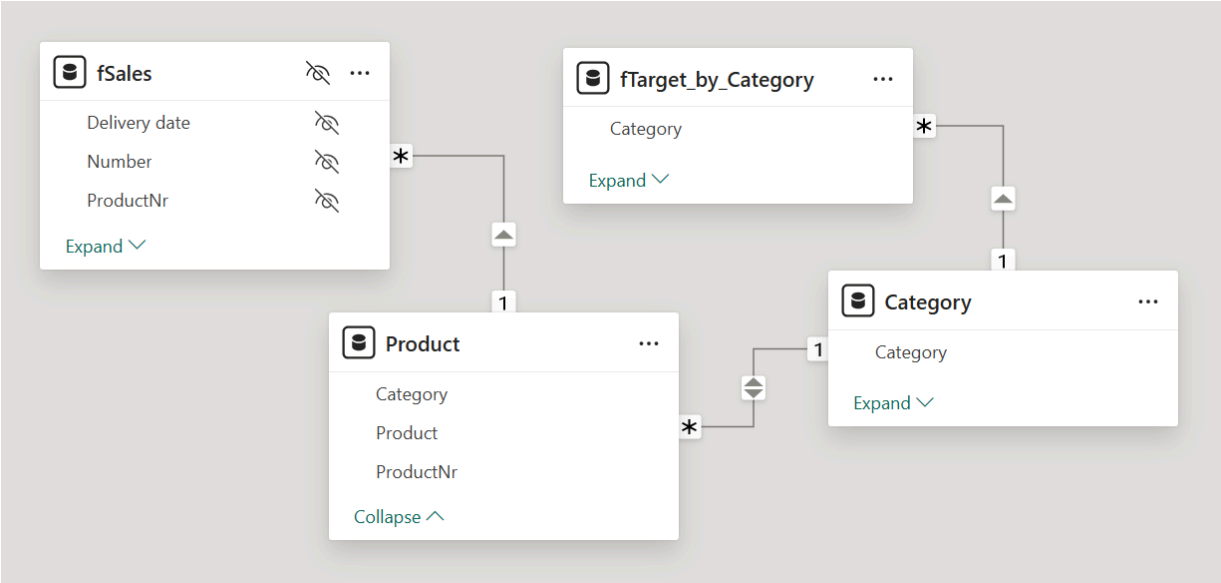


Figure 2.17: Using a bridge table

By using a bridge table, all structure is implemented through regular, one-to-many relationships that have consistent behavior and for which the DAX engine is optimized. One difference, for instance, is that many-to-many relationships do not cause a blank row to be added to the filter table when an

unknown value is encountered; this can lead to unexpected results. After all, inconsistency in data is generally visible in the output of a Power BI model through blank labels, which are caused by blank rows. Without these blanks, inconsistencies remain hidden.

The use of clusters of filter tables, as discussed before, is a natural way to deal with granularity differences in fact tables using regular relationships.

We have mentioned a lot of different topics and considerations to create a solid model. In the next section, we'll take a step back to summarize some of these tips and best practices.

Memory and performance considerations

The design of a semantic model highly impacts its size, and size is highly correlated with performance. In this section, we share some best practices to optimize the performance of your model, as a recap of the topics discussed in this chapter. As a rule of thumb, smaller models with respect to size are faster. You can use the file size of the Power BI model, when saved as a `.pbix` file, as an indication; you can also get a more detailed view of size and performance by using `INFO` functions in DAX query view or specific community-driven tools such as DAX Studio.

Keep in mind the following guidelines while designing your Power BI model:

- **Having fewer columns is better**: Semantic models achieve a high compression rate of data due to the columnar database concept. However, a model still needs to keep track of which values belong together in a row. The more columns a table has, the more overhead the model needs to know what goes where. So, keep the number of columns per table as small as possible. What we see a lot of people do is just load all columns of a source table into the model out of convenience (or laziness, you could say). Consider that it's easier to add a column when you need it than to find out which columns aren't being used and remove them later. A model never shrinks organically!
- **Use efficient data types**: **when possible.** Internally, the model optimizes data storage at the level of bits. All optimizations of the columnar database are based on this. It means that any data type that is not a whole number has to be treated differently using a dictionary of values.

This doesn't mean that only the `Whole Number` data type is efficient to use; multiple data types are treated as whole numbers internally, such as `Date`, `Fixed Decimal Number` and `True/False`. The data type consideration also applies to relationships, so use one of the whole number types for relationships when possible.

- **Having many rows is no issue, but beware of many values**: Again, due to the columnar database concept, the model can store a lot of rows efficiently. It will determine the best way to store the values in a column, but more distinct values need more space. The number of unique values in a column is by far the most important thing to keep an eye on! Often, an obvious way to save memory is to remove unique keys from fact tables. Many transactional systems provide a unique identifier for each transaction and, when loaded, these are among the most expensive (in terms of memory usage) columns in a semantic model. We have had cases where simply removing one unique column from the largest fact table reduced the size of the model by over 90%!
- As with data types, the number of distinct values has an impact on relationships as well. The number of primary key values of a relationship should be relatively small. If you have a relationship with hundreds of thousands or even millions of unique key values, you should be looking for a different structure in your model.
- **Avoid outliers**: In many source systems, developers use special values for denoting the planned absence of real data, or other reasons. The special values are often outliers to ensure they are not confused with real data, such as *December 31, 9999*. These outliers can cause the model to store a column using a dictionary as with the less efficient data types, even if the values themselves are whole number values. This is because when storing values as whole numbers, the model must account for all possible values between the smallest and largest values in the column, and it may be more efficient to go for a dictionary instead. To avoid this, leave these values blank or choose a special value that is close to the actual values.
- **Do you really need all that history?** An obvious way to have a smaller model is to load less data. We have encountered many cases in which a source system keeps a long history of data. This is especially true when

using a data warehouse as a data source for a Power BI model. You may load sales transactions from the year 2000 onward, but ask yourself: who is going to analyze the sales data from before 2010 or 2015? It may be better to have a separate model for the few historians who want to carve out wisdom from the past, and have a richer, more elaborate model for daily use containing only a few years of history.

- **In some cases, split columns**: In extreme situations, it may be useful to split columns to end up with two columns, each having fewer distinct values. This could happen with composite keys, for example, a product code consisting of a category code and a sequence number: "A82.019." Separate category code and sequence number columns would have far fewer distinct values each and may be stored more efficiently. This approach has obvious drawbacks in more complex handling and, also, the composite column may be needed for a relationship; so, only do this when you really, really need it. A common scenario that benefits from column splitting is when you have date/time values. Most often, splitting these in separate date and time columns drastically reduces the number of unique values.

Many of these considerations have more impact when the data volume of your model gets bigger. But it is good to keep them in mind even with a small model; after all, it is harder to change things when problems arise.

Architectural options in semantic models

It should be clear now that the structure of your semantic model is very important for both the performance and functionality of the model, and the amount of complexity in DAX calculations.

In a Power BI solution, typically other considerations are in play as well. We do not just have to deal with data that is in the model. We also must accommodate new data. After all, one of the values of a proper BI solution is that it provides insights based on the latest data. The frequency of new data coming in, and the volume of the data in general, are important factors that drive the technical setup, or architecture, of the semantic model. In this section, we discuss several ways, called **storage modes**, in which semantic models can handle data, each with their strengths and weaknesses.

Import models

The common way for a Power BI semantic model to work with data is to import data through Power Query. The data tables are defined through a query definition and stored in the model's `.pbix` file or in a folder structure containing files of the `.pbip` format. When the model is loaded, either in Power BI Desktop or the Power BI or Fabric service, the table data is read from the storage and loaded into memory.

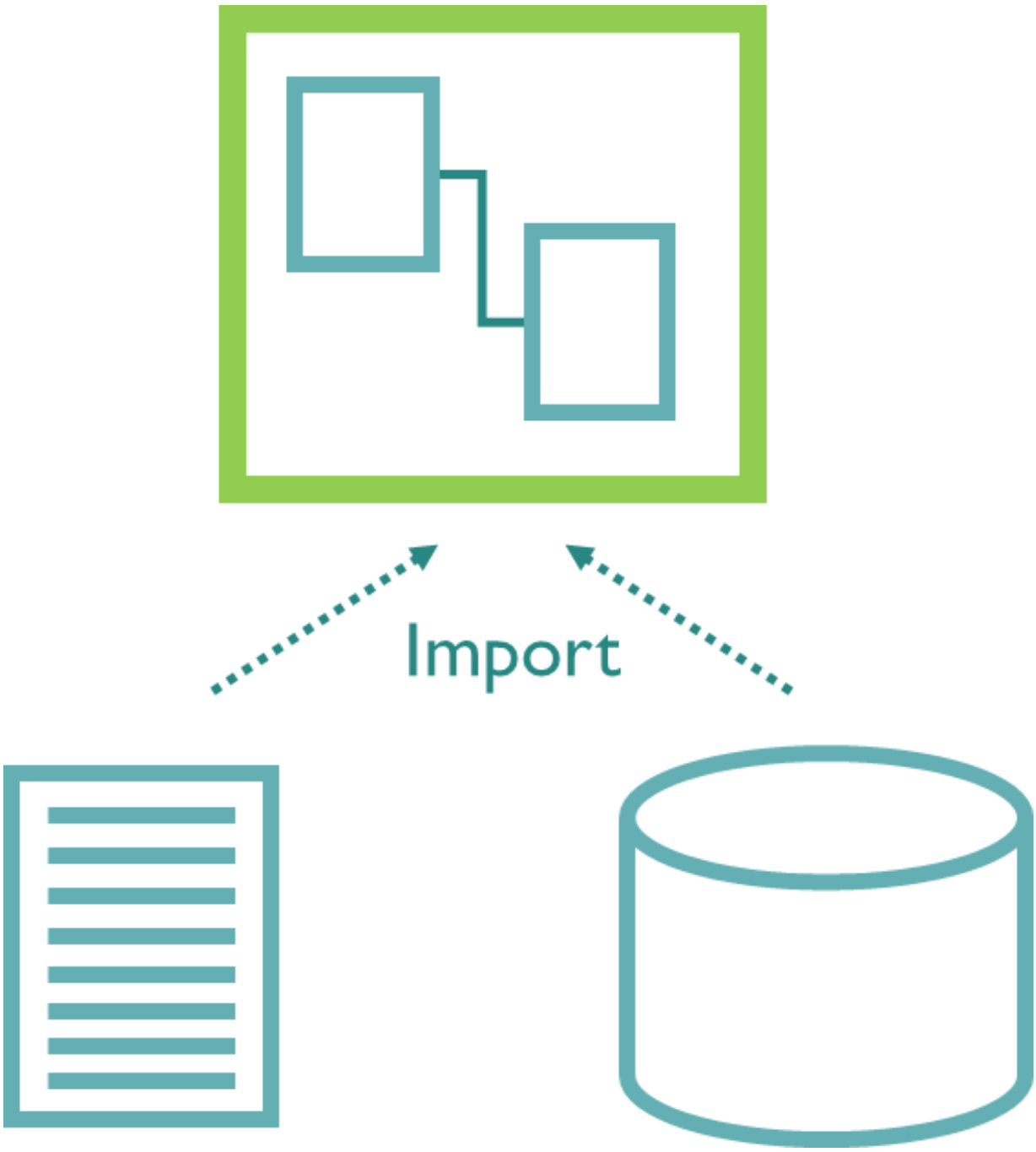


Figure 2.18: Import mode

The advantage of the import storage mode is that all of the data is readily available for the model to work with, in the proper column store format. The drawback is that some work needs to be done to convert the data from the original source to tables in the semantic model. This is done through *refreshing* the model. In Power BI Desktop, you can do this manually (and even for single

tables). In the service, you have several options: do a manual refresh, set a refresh schedule to automatically refresh the model at specific intervals, or refresh programmatically through an API. It depends on your license how often refreshes can happen:

- With Power BI Pro, refreshes can be scheduled eight times a day
- With Power BI Premium or Fabric, up to 48 scheduled refreshes can be set per day
- Refreshes through the Power BI or Fabric API count toward the eight refreshes per day limit for Power BI Pro but have no limit on Power BI Premium or Fabric

The need for refreshing the model means that there is always some delay between new data being available and becoming visible in a report. An import model is, therefore, less convenient for real-time solutions, while near-real-time solutions can only be achieved with premium capabilities.

DirectQuery

The second storage mode to discuss here is called **DirectQuery**. Here, no data is loaded into the model; instead, the data stays where it is. This leads to the question of how visuals in a Power BI report are populated (which is normally through a DAX query to the semantic model), and how DAX calculations are executed. The answer to this question is that each retrieval of data is translated into a query to the original data source. The most common situation is to have a SQL database as the source, and all DAX is translated to SQL queries.

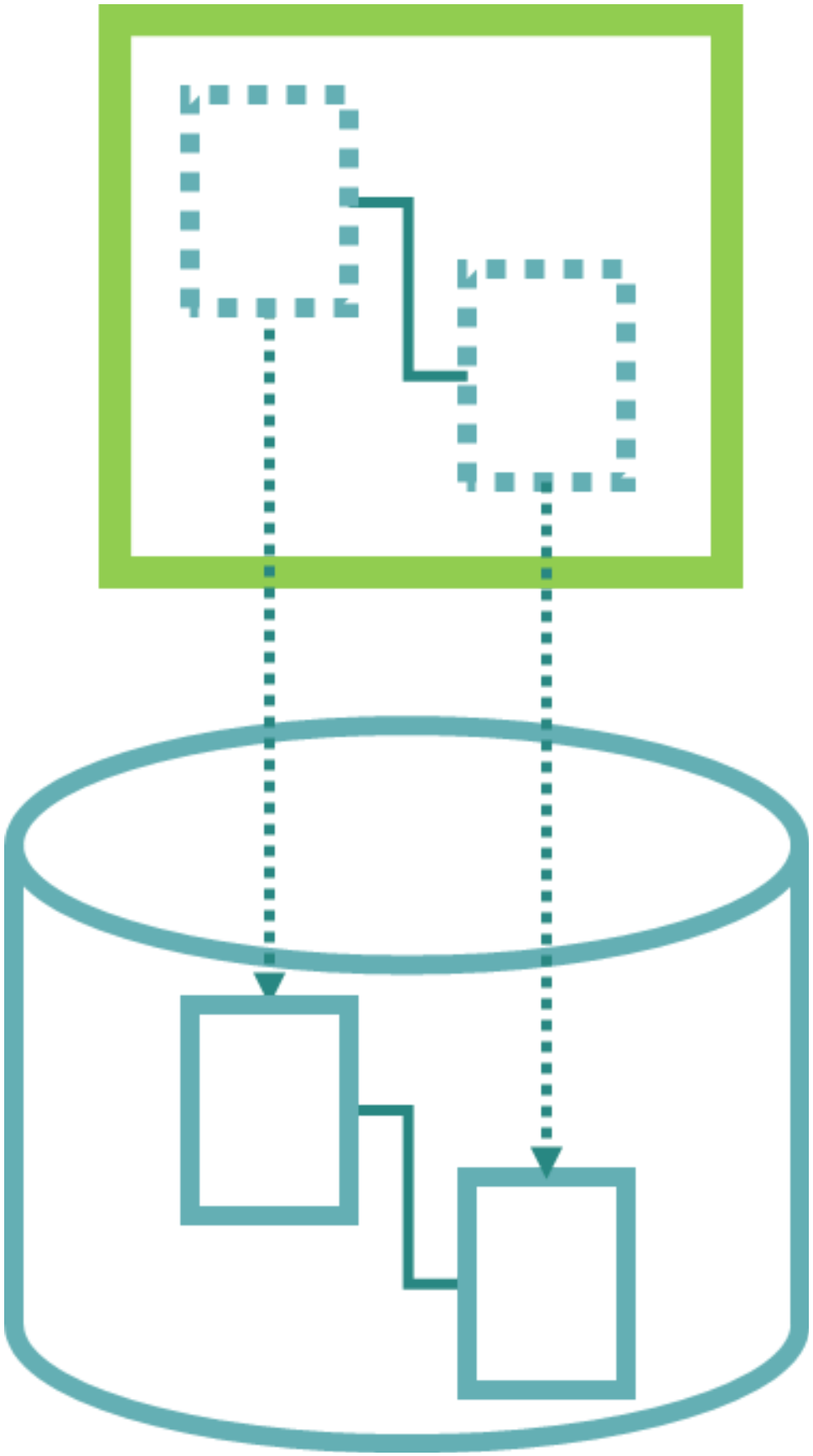


Figure 2.19: DirectQuery mode

Consequently, you'll have a few things to consider:

- The data source needs to be capable of processing the queries received from the semantic model. This means that, for instance, a SQL database can be used as a DirectQuery source, but a folder containing text files cannot.
- The data source is responsible for the performance needed to provide proper interactivity in the visual report. Whenever a report user clicks an item in a report, visuals send new queries to the semantic model, which are sent through to the data source. Results must not take too long to be returned, or the user will experience unacceptable response times.
- As all DAX code must be translated to SQL, not all DAX functions can be used as they cannot be translated to SQL properly.

The primary benefit of DirectQuery is that new data is immediately visible in a report, as no separate import or model refresh is needed. A common problem is that many data sources are not particularly suitable for the query performance that import mode can provide; after all, the semantic model was designed exactly to provide the performance needed for interactive reports, but other database technologies are not.

DirectLake

With the Fabric platform comes another storage mode: **DirectLake**. This is not the equivalent of Direct Query, as you may think from the name. Instead, DirectLake directly taps into the core concepts of Fabric as discussed in *Chapter 1*: separation of storage and compute and a uniform, column-based approach to data storage in the form of Delta Parquet tables.

Where semantic models in import mode process (DAX) queries in a way that optimally aligns with the model's column store, semantic models in DirectLake mode do the exact same thing but using Fabric's column store instead. While import mode needs to refresh a semantic model to load data into the column store, with Direct Lake, the data is already available in a column store (and data is loaded into that using any of the technologies available in Fabric). The semantic model adds relationships, as well as all things DAX-related.

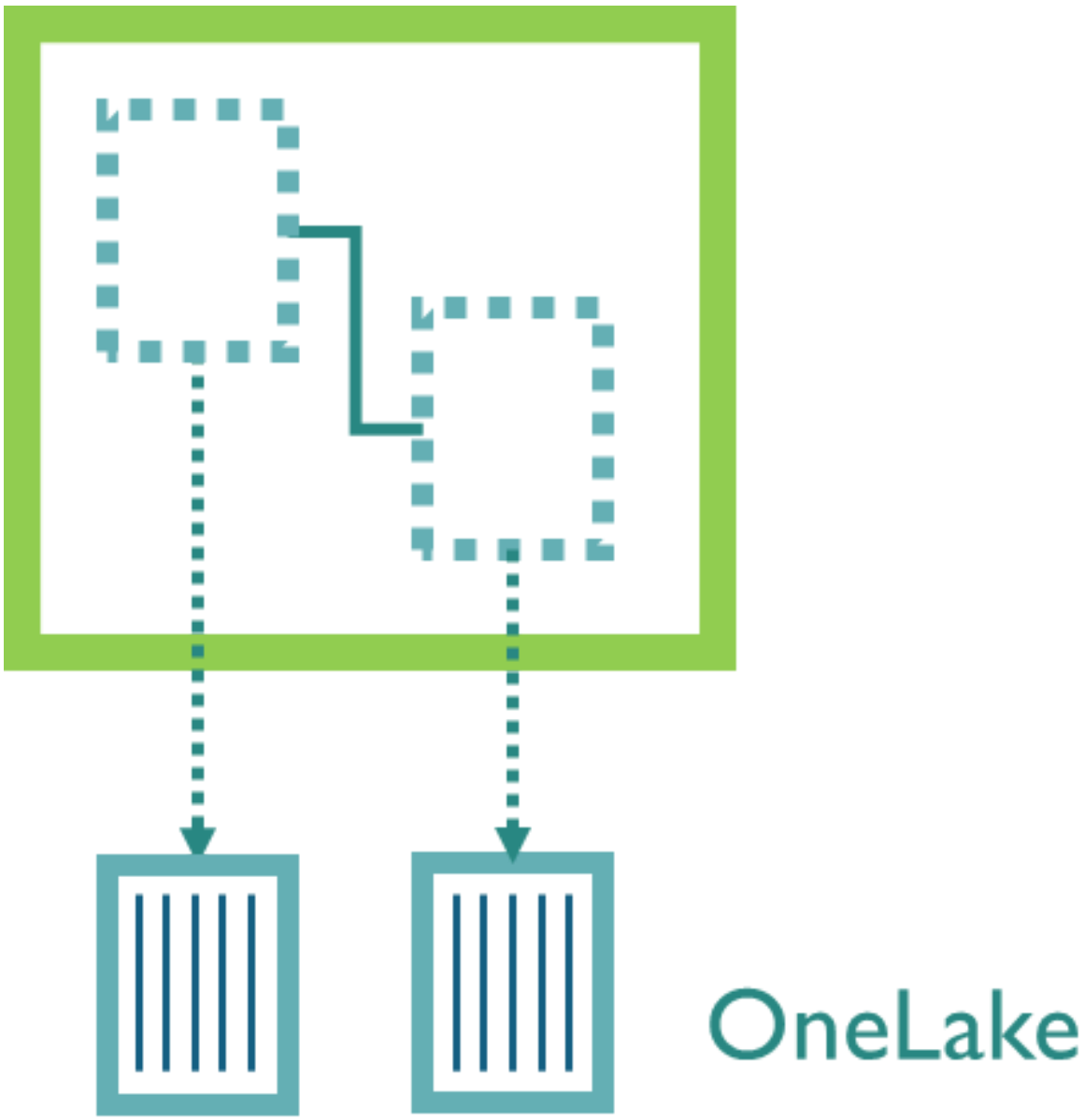


Figure 2.20: DirectLake mode

Direct Lake, therefore, combines the advantages of both import models and Direct Query—hyper-performance and no data import—while avoiding the disadvantages of both storage modes. This means that, among other things, DirectLake can achieve high performance on huge data volumes with hundreds of millions, or billions, of rows of data.

Composite models

The three main storage modes, import, DirectQuery, and Direct Lake, can each be applied to semantic models as a whole. Additionally, you can design semantic models in which storage modes are set on a per-table basis. These models are called **composite models**.

With a composite model, you can choose the best configuration to serve your specific needs. We will go through a few examples.

Import and DirectQuery

You may have an import model with a single table in DirectQuery mode that contains a huge number of rows, or that contains frequently updated rows. Especially when this table is only referenced in some specific cases, most of the model will be fast, and only when using a DAX calculation that references the DirectQuery table may it be slower.

Be careful, however, with relationships to this table; these relationships would have to do filter propagation across the import/DirectQuery boundary, which is computationally expensive. These relationships are, therefore, *limited*. Semantic models offer a specific storage mode for filter tables that are related to both import tables and DirectQuery tables: **Dual mode**. With Dual mode, filter propagation is done in the DirectQuery source when the filter table needs to filter the DirectQuery table. When an import table needs to be filtered from the filter table, filter propagation is done in the semantic model. The obvious requirement for this is that the filter table is available in the DirectQuery source too!

DirectQuery from different sources

A composite model allows tables from multiple sources, each connected with DirectQuery. In a model like this, you will notice that in the model view, tables from different sources are distinguished by color. Here again, relationships between different **data source groups** (including the group of import tables) are limited.

DirectLake and import

With Fabric, it is possible to create a composite model that combines tables in DirectLake mode with imported tables. Just like with the combination of import and DirectQuery, this allows avoiding frequent loading of large tables or frequently updated tables. The huge advantage over DirectQuery is that the

source of the data is a table in OneLake, which is stored in a format the semantic model can work with directly. Relationships between DirectLake tables and imported tables are, therefore, not limited but fully functional, native relationships.

When considering this type of composite model, you should, of course, consider whether to have all tables in OneLake and have a full DirectLake model, or to create a composite model.

DirectQuery to semantic models

As mentioned earlier, you can use a semantic model (which has been published to the service) as a DirectQuery source. Not only does this give you access to the data that the model contains, but it allows you to use the DAX logic in that model, in the form of DAX measures, as well.

A DirectQuery connection to a semantic model can be combined with imported tables, or tables in DirectQuery mode that connect to another source; in other words, you can create a composite model.

The support for DirectQuery to semantic models provides interesting possibilities, and even more so when combined with DirectLake. You could have one or more central models in which core analytic capabilities and common data aggregations are implemented. For instance, you could have a central sales model that not only contains sales data but also provides common aggregations, such as measures for year-to-date sales, sales growth, or hit rate for sales opportunities. Other models can tap into the central model without needing to load data or copy data transformation logic, while using the corporate definitions of what "sales" is, how to compute hit rates, and so on.

Semantic link

DirectQuery to semantic models allow you to reuse data and logic from a semantic model in another semantic model. This is a great feature when your endpoint is a Power BI visual report. But a report is not the goal in all situations. If you have implemented valuable business logic in a semantic model, it only makes sense to reuse that in other ways.

On the Microsoft Fabric platform, reusing logic in semantic models is made possible through a feature called **semantic link**. Semantic link allows you to

connect to a semantic model through Python code or any language supported by Fabric notebooks. This way, a data scientist can work with the result of DAX measures, for example, in training AI models.

Summary

In this chapter, we have discussed the foundational concepts of the Power BI model. You have learned what makes a Power BI model fundamentally different from other data management products (the in-memory column store) and what an optimal design looks like.

A good Power BI model has an effective structure with fact tables, filter tables, and relationships between them. Making good design choices when it comes to structure and data types, as well as carefully considering what your data looks like from the perspective of granularity, unique values, and value distribution, leads to a model that performs well. And, perhaps more importantly, such a model forms a good basis for rich calculations in DAX.

Finally, different architectural options, called storage modes, were discussed, each with its strengths and weaknesses. The choice of storage mode impacts the frequency with which your model can handle new data. In the next chapter, we'll take a closer look at where you can find and use DAX.

3

Using DAX

Join our book community on Discord

https://packt.link/EarlyAccessCommunity

The real power of Power BI models is in calculations using the DAX language. While many Power BI users focus on the model and try to avoid DAX entirely, anything that goes beyond simple, basic aggregations of data requires DAX calculations. You will surely encounter the need for more sophisticated calculations in Power BI sooner rather than later. The typical way things go is that the first well-designed Power BI report leads to more and ever more complicated questions to ask about your data.

Chapter 5, *Security with DAX*, and later chapters aim to give you inspiration on what can be achieved using DAX and how to approach a business problem with DAX. Before we dive into these scenarios, we still have some fundamentals to cover. In this chapter, we briefly touch upon the different uses of DAX in Power BI.

The uses we will discuss here are as follows:

- Calculated columns
- Calculated tables
- Measures
- Visual calculations
- DAX security filters
- Field parameters
- Calculation groups
- DAX queries
- User-defined functions

We also discuss how to create date tables with DAX. The chapter concludes with some general best practices for the use of DAX.

NOTE

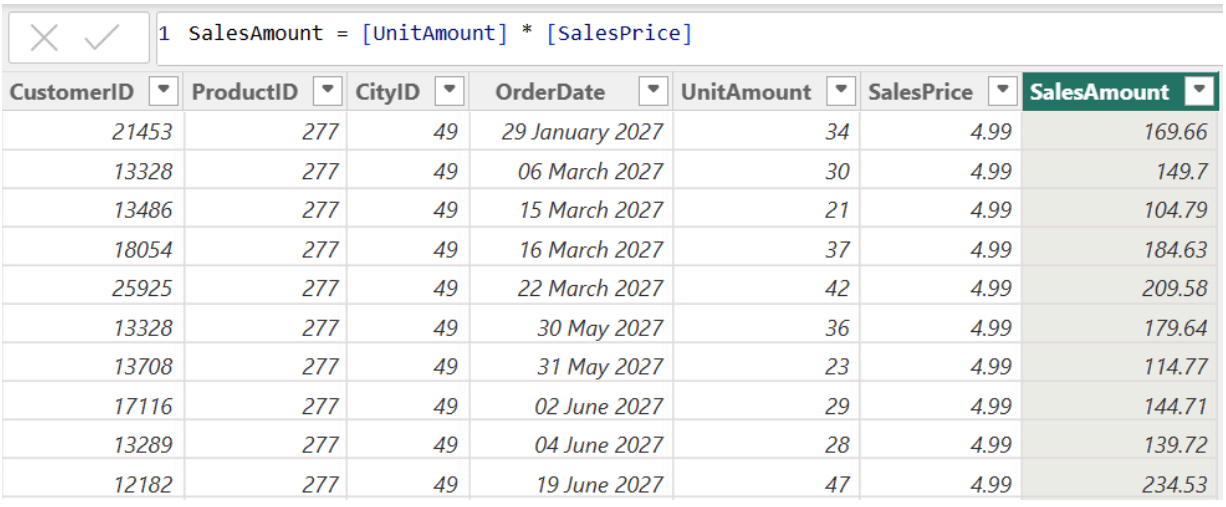
A PBIX file with examples accompanies this chapter. The file, `Using DAX.pbix`, can be downloaded from this link: https://github.com/PacktPublishing/Extreme-DAX-Second-Edition/tree/main/Ch03.

Calculated columns

A **calculated column** is a column of data that is added to a table in the Power BI model by performing a DAX calculation. You can create a calculated column with the **New column** option when the table is selected in Power BI Desktop. A basic example is to calculate the value of a sales transaction by multiplying the number of products sold by the price per product (note that column names are written between square brackets in DAX):

```
SalesAmount = [UnitAmount] * [SalesPrice]
```

The following is the resulting calculated column:



CustomerID	ProductID	CityID	OrderDate	UnitAmount	SalesPrice	SalesAmount
21453	277	49	29 January 2027	34	4.99	169.66
13328	277	49	06 March 2027	30	4.99	149.7
13486	277	49	15 March 2027	21	4.99	104.79
18054	277	49	16 March 2027	37	4.99	184.63
25925	277	49	22 March 2027	42	4.99	209.58
13328	277	49	30 May 2027	36	4.99	179.64
13708	277	49	31 May 2027	23	4.99	114.77
17116	277	49	02 June 2027	29	4.99	144.71
13289	277	49	04 June 2027	28	4.99	139.72
12182	277	49	19 June 2027	47	4.99	234.53

Figure 3.1: A calculated column.

Calculated columns are a straightforward way to add some intelligence to the Power BI model. If you come from an Excel background, this probably has the most natural feel to it, as putting formulas in columns is the way most Excel

users have learned to work in Excel. But our strong recommendation is: *DO NOT* use calculated columns, unless you have very good reasons to do so. There are a few reasons why:

- A calculated column creates new data, which takes up space in the model. As was discussed in *Chapter 2*, *Model Design*, in the *Memory and performance considerations* section, more columns make the model larger and slower. Furthermore, at a model refresh, all calculated columns are recalculated (regardless of whether they are actually used or not), which can cause longer refresh times.
- If, for whatever reason, you need to delete a table from your model and add it again in some changed way (and you will definitely find yourself in this situation at some point), you will lose all calculated columns and will have to rebuild them.
- The columns used for a calculation, such as the `[UnitAmount]` and `[SalesPrice]` columns in the preceding example, need to stay in the model but would possibly have no other use. In the example, ask yourself what you would do with the `[SalesPrice]` column. Compute the average price per product, perhaps? The answer is no: the average price should be weighted by the number of products sold, so a straightforward average of the `[SalesPrice]` column would be incorrect. Instead, you would divide the total sales amount by the total number of products sold, and you don't need the `[SalesPrice]` column for that.
- The results of the calculations in a calculated column are static: they are computed only when defining the column and when the Power BI model is refreshed. This is in opposition to the dynamic nature of DAX and Power BI reports in general.

The issue with calculated columns is that, most often, the things you do with them fall into the category of data preparation, or the *Prepare* layer in the Five-Layer model discussed in *Chapter 1*, *Analyzing Data with DAX*. There are better tools for doing data preparation than the Power BI model, such as Power Query. An optimal model would contain the columns `[UnitAmount]` and `[SalesAmount]` in a sales transaction table, not `[SalesPrice]`.

There are some exceptions to the rule of not using calculated columns, which you may encounter when you work on more advanced scenarios with DAX:

- Sometimes, when creating a complex DAX calculation, you will find that part of it is, in fact, static, in that it could indeed be implemented as a calculated column. If this is a complex calculation that needs to be done over and over again, you may gain significant performance by implementing it as a calculated column. However, you should first consider creating the column in the *Prepare* layer!
- Some calculations that should result in a model column are very hard to do in the *Prepare* layer with, for instance, Power Query, while being straightforward using an appropriate DAX function. In this case, you would save on both development time and data refresh performance by implementing these calculations as a calculated column. This situation typically occurs when the column values needed are the result of some sophisticated aggregation. However, if you run into this situation, first ask yourself if the problem can be solved without a calculated column!

In short, you should not use a calculated column unless you have very good reasons to, as an exception.

Calculated tables

Calculated tables are comparable to calculated columns: they add data to a Power BI model, but now in the form of a complete table. To create a calculated table, you most often need special DAX table functions. You will encounter many DAX table functions in *Chapter 5*, *Security with DAX*, and later chapters of this book; for a general introduction to table functions, see *Chapter 4*, *Context and Filtering*.

To create a simple calculated table in a Power BI model, you can use the table constructor. The following expression creates a table with one column:

```
Example = {1, 2, 3}
```

The result of this formula is a table named `Example`, with a single column of `Value`:

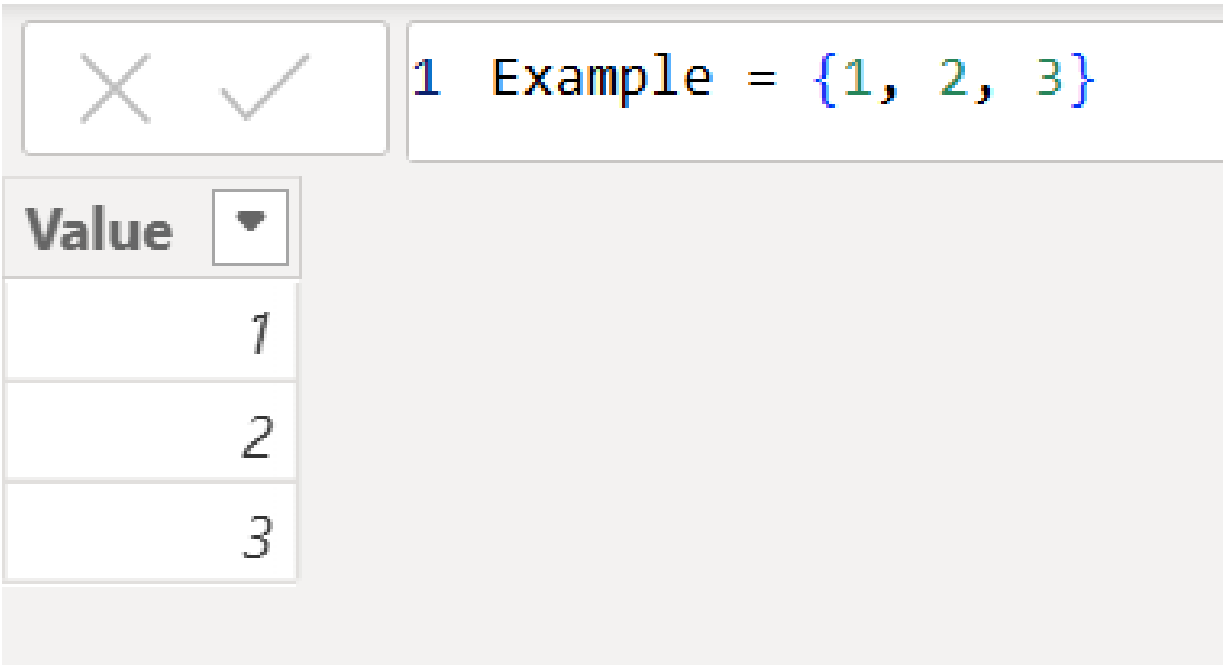


Figure 3.2: A calculated table

Note that the table constructor does not give much control over the table that is created. The column is named `Value`, and the data type of the `Value` column is derived from the values provided (which is, of course, fairly accurate most of the time). When the values provided are different types of data, a data type is chosen that can store all the values. Here's an example:

```
Example2 = {1, 2, "3"}
```

This formula produces a table in which the `Value` column has the `Text` data type.

The table constructor does allow the creation of tables with multiple columns by providing lists of values by row, between parentheses:

```
Example3 = { (1, "Red"), (2, "Green"), (3, "Blue") }
```

This is the resulting table:

Figure 3.3: A calculated table with two columns

A table with properly named columns and tightly controlled data types can be obtained using the `DATATABLE` function. The arguments to this function are a series of pairs of column names and data types, and a list of values for the rows of the table. `DATATABLE` has two quirks: first, the data types have different names than the ones used in the Power BI model (`INTEGER` for whole number, `STRING` for text, and so on), and the values in one row must be contained in braces instead of parentheses, like in the table constructor. The table in *Figure 3.3*, but with nicer names, can be constructed using `DATATABLE` with the following formula:

```
Example4 =
DATATABLE(
    "Number", INTEGER,
    "Color", STRING,
    {
        {1, "Red"},
        {2, "Green"},
        {3, "Blue"}
    }
)
```

Calculated tables are often used to create `Date` or `Calendar` tables in a Power BI model. These are discussed in the *Date tables* section later in this chapter.

Some of the issues with calculated columns also apply to calculated tables: a calculated table increases the Power BI model's size, and you are probably doing something that is really data preparation. However, contrary to a calculated column, a calculated table is not tightly coupled with other elements of the model. When you remove columns or tables that the

calculated table uses to compute, you will get an error, but adding the tables or columns again will solve this.

In general, our recommendation is not to use calculated tables when they can be taken care of in the *Prepare* layer of the Five-Layer model. When you don't have access to *Prepare* capabilities, for example, when you are working with a centrally managed data warehouse, you can use calculated tables for tables that are not offered by the data warehouse, or that would not be practical to store in the data warehouse.

Measures

Measures are, without a doubt, the most powerful element of Power BI models. In fact, most of the work we do on Power BI models comes down to designing and implementing DAX measures.

When using a numeric column from a fact table in a Power BI report, the column values will be aggregated. The basic aggregations available are *sum*, *average*, *minimum*, *maximum*, *count*, *distinct count*, and some statistical aggregations such as *standard deviation*, *variance*, and *median*. The basic aggregations differ depending on the data type; for a `Date` column, for instance, you can choose *earliest* and *latest*, *count*, and *distinct count* only. When you use columns this way, the Power BI model creates an **implicit measure** in the background: an aggregation function that returns the selected aggregation of the values in the column.

In real life, many of the required insights come down to aggregations that cannot be expressed in terms of basic aggregations. Many people, both Excel users and data warehouse developers, build (calculated) columns to try to create data that will provide the results they need, while still only needing one of the basic aggregations. For example, in both Excel models and data warehouses, you may encounter an indicator that denotes whether or not a row of data falls in the "current year-to-date" category. Again, this is a static solution that will not let you, say, retrieve the year-to-date results of two months ago.

DAX allows you to define your own *custom aggregations* by writing DAX formulas to create **explicit measures**. As an example, the weighted average

price discussed previously, in the *Calculated columns* section, can be implemented as a DAX measure with the following formula:

```
Average Price = SUM(fSales[SalesAmount]) / SUM(fSales[UnitAmount])
```

In this formula, the assumption is made that the fact table with sales transactions is called `fSales`.

NOTE

The division operator `/` is normally replaced by another DAX function, `DIVIDE`:

```
Average Price =
DIVIDE(
    SUM(fSales[SalesAmount]),
    SUM(fSales[UnitAmount])
),
```

The advantage of `DIVIDE` is that division by zero is handled elegantly. This is a simple example of the added value of DAX measures using appropriate DAX functions over basic aggregation of columns.

DAX measures should be your default option for adding intelligence to your Power BI model. A measure does not add data to the model and, therefore, keeps the model lean and fast. However, as the calculation is done on demand when a user views a report, it is important to put effort into creating the most efficient calculations. Other options include visual calculations, field parameters, calculation groups, and user-defined functions, which are all created with the DAX language. These will be discussed in the next sections of this chapter. Every option has its own benefits and applications. In *Chapter 5*, *Security with DAX*, and later chapters, some applications of these options will be used to solve complex business scenarios.

For working with DAX in general, but with DAX measures in particular, there are some fundamental concepts that are critical to understand. Among these are the concepts of DAX context, DAX filtering through context transformation, and DAX table functions. These concepts are discussed in *Chapter 4*, *Context and Filtering*.

Visual calculations

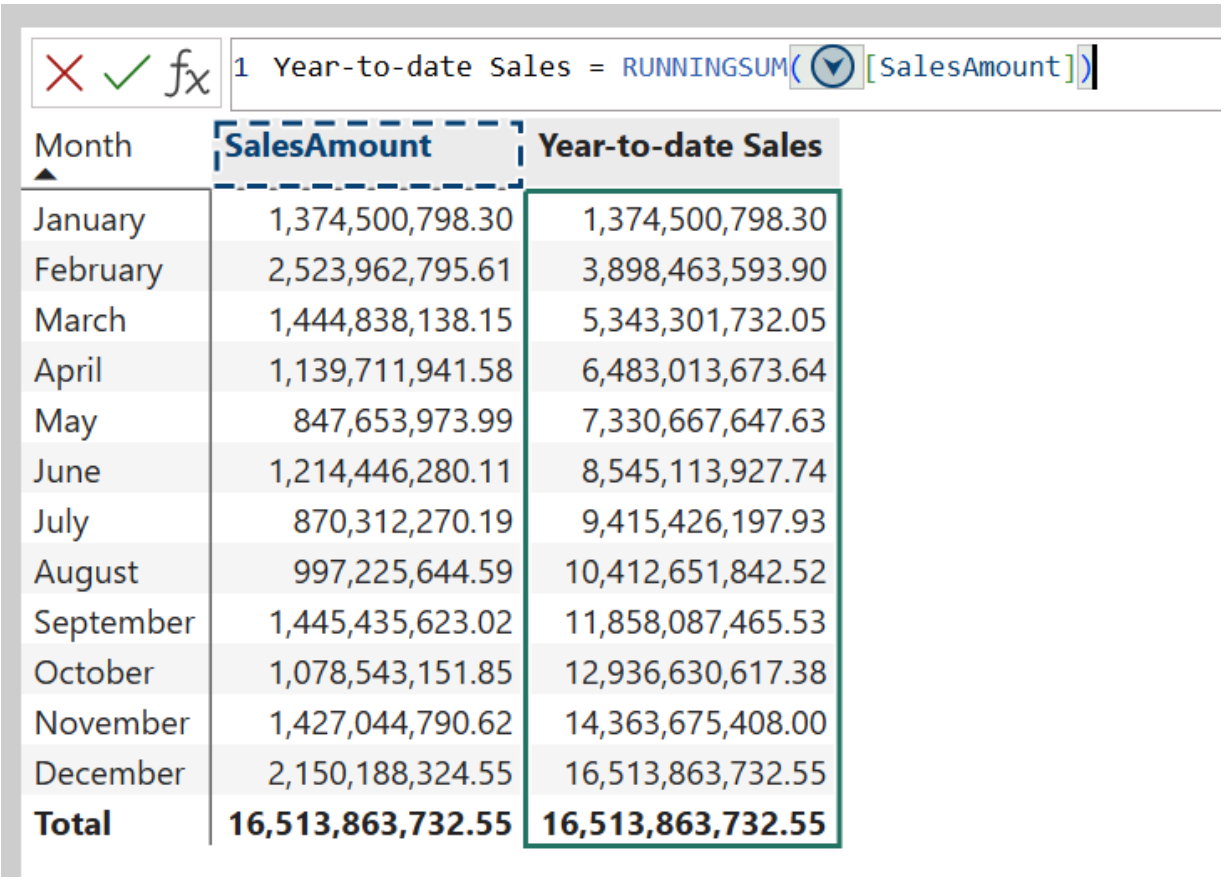
Visual calculations are DAX expressions that are defined on a specific visual in a Power BI report. They work as regular DAX measures, in the sense that they do not add data to the model, and that they provide the freedom to create custom aggregations. However, there is one big difference with measures as well: measures are defined to work on columns of tables in your model, and they will compute over all the rows of these columns (unless otherwise specified). Visual calculations work only on the subset of data that is visible in the visual on which you create the visual calculation. Usually, you show aggregated data in a visual, and therefore, a visual calculation on this visual will compute over the aggregated data as well. Visual calculations, therefore, usually perform better than measures.

Most DAX functions can be used in visual calculations. On top of the regular DAX functions, there are also DAX functions that are specific to visual calculations. These are business functions that calculate a *running sum* or a *moving average* of the data in a column or row but also functions that allow you to easily move across the data in your visual and retrieve the *first*, *last*, *previous*, or *next* value.

As an example, suppose you have a clustered column chart in your Power BI report that shows the sales amount per month for a specific year, and you want to use a visual calculation to compute the year-to-date sales amount. This can easily be achieved with the following visual calculation:

```
Year-to-date Sales = RUNNINGSUM([SalesAmount])
```

The following is the resulting *visual matrix*, where all the values can be found that are computed to populate the visual itself:



Month	SalesAmount	Year-to-date Sales
January	1,374,500,798.30	1,374,500,798.30
February	2,523,962,795.61	3,898,463,593.90
March	1,444,838,138.15	5,343,301,732.05
April	1,139,711,941.58	6,483,013,673.64
May	847,653,973.99	7,330,667,647.63
June	1,214,446,280.11	8,545,113,927.74
July	870,312,270.19	9,415,426,197.93
August	997,225,644.59	10,412,651,842.52
September	1,445,435,623.02	11,858,087,465.53
October	1,078,543,151.85	12,936,630,617.38
November	1,427,044,790.62	14,363,675,408.00
December	2,150,188,324.55	16,513,863,732.55
Total	**16,513,863,732.55**	**16,513,863,732.55**

Figure 3.4: A visual calculation

The clustered column chart itself now contains two bars each month: one for `SalesAmount` and one for the newly added `Year-to-date Sales`:

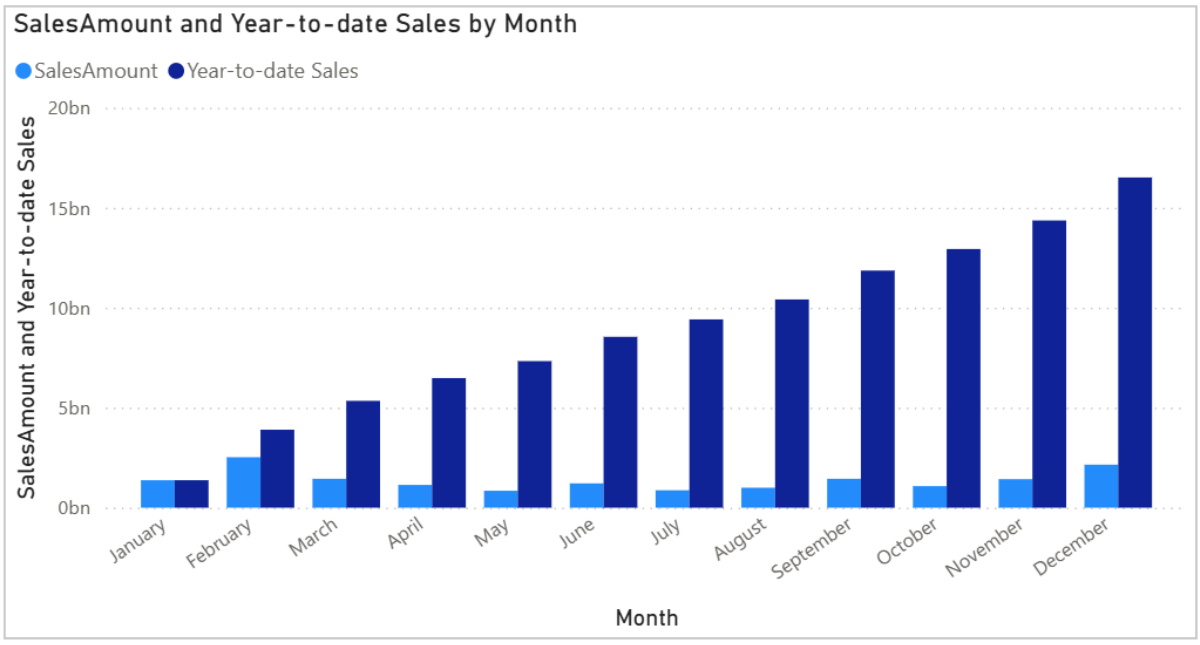


Figure 3.5: The visual calculation is directly visible in the column chart

Visual calculations are easier to use and understand than regular DAX measures, especially for users who are familiar with Excel, because they work on a subset of the data that is visible when you create the calculation. Creation of a visual calculation is less abstract than that of a measure, because you can immediately see the result in the visual matrix, and you refer directly to other columns that you want to use in a calculation.

One downside of visual calculations is that they are not easily reusable. They exist only on a visual and can be used inside that visual, but if you want to use them in another visual, you will have to recreate them there. Therefore, if the definition changes, you need to change the calculation in two places as well. In *Chapter 10, DAX-Driven Waterfalls*, we'll explore visual calculations further and also propose a way to overcome this reusability limitation.

DAX security filters

DAX can also be used for implementing security within a Power BI model. When a user retrieves a report, they will be able to see all results provided by the model through that report. In many cases, there is a need to restrict what the user sees, based on their role or identity. For instance, consider the following DAX **security expression**:

```
Customer[Region] = "Europe"
```

When set for a specific security role, this DAX security filter will cause users in that role to only see customers in the Europe region, together with data related to those customers.

Chapter 5, *Security with DAX*, further introduces security with DAX.

Field parameters

A **field parameter** in Power BI allows a user of the report to define which fields they want to select. These selections can make the values that are shown in a visual, or the axes on which those values are shown, dynamic, creating a new level of interactivity between the report and the user of the report.

You create a field parameter in Power BI by selecting which fields you want to include. You can include measures and columns, but it is advised not to mix them in a single field parameter, because the effect can be unpredictable.

You can use measures in a field parameter to allow the report user to dynamically select which fields they want to view in a visual. For example, you can use a field parameter that includes measures for costs, sales, and margin, and create a column chart with this field parameter on the values and a slicer on the report page that shows all the fields in the field parameter. The report user can then use the slicer to select whether they want to see only the costs, sales, margin, or any combination of these fields.

If you create a field parameter that includes columns, you can use this to dynamically change the axes of a visual. For example, you can create a field parameter that contains the columns `[Year/Month]`, `[Group]`, and `[RetailType]`, and use this field parameter on the *x* axis of a column chart. By adding a slicer on the report page, the user can dynamically select which breakdown they want to view in the column chart – the value over time, or a breakdown by product group or customer retail type – without the need to create multiple visuals.

In *Chapter 6*, *Dynamically Changing Visualizations*, field parameters are used in a business case and will be further explored.

Calculation groups

Literally, a **calculation group** in a Power BI model is a group of calculation items. It allows you to reuse DAX code for different measures without having to create multiple copies of the measures. For example, if you have sales data and you're analyzing it over time, you may also want to have year-to-date sales, last year's sales, and year-on-year growth for the sales. But if you also have cost or quantity data, you may also want to include the year-to-date, last year's, and year-on-year growth for this data. The logic to create year-to-date sales is the same as year-to-date costs – only with a different base measure, namely `[Costs]` instead of `[Sales]` (assuming you already have measures that calculate the costs and sales and that are named as such).

This is where calculation groups come in. You can create different calculation items to calculate the year-to-date, last year's, and year-on-year growth variants for a generic base measure and then apply these calculation items to your different measures to create all the possible flavors: year-to-date sales, last year's costs, year-on-year growth for quantity, and so on. Your model only needs to contain the three base measures and one calculation group with three calculation items to allow the nine different variants. There is no need to create nine different measures for this, with the added downside that if the logic for one of these variants' changes, you need to change it in all measures that use that logic. With a calculation group, you only have to change the calculation item that contains the logic.

Calculation groups will play an important role in the business case discussed in *Chapter 8*, *Alternative Calendars*. In that chapter, alternative ways to use calculation groups will also be explored.

DAX queries

Another place where you encounter the DAX language is in DAX queries. A **DAX query** is a question you can ask your model. You do this in the DAX language, hence the name DAX query.

A DAX query is a way to retrieve output from your Power BI model via an external tool. In this way, your Power BI model can be used, for example, as a data source for classic, RDBMS-oriented reporting tools.

If you use Power Pivot in Excel, you can use DAX queries as a way to retrieve data from the Power Pivot model, as an alternative to the default pivot table

output.

A specific use case for DAX queries is in Power BI paginated reports. These are developed using Power BI Report Builder (as opposed to Power BI Desktop, which is the tool for all other use cases) and can connect to a published Power BI model. When making the connection, you need to provide a DAX query to retrieve a set of data from the Power BI model that is published.

Like calculated tables, DAX queries need a table expression to evaluate. In the table expression, all DAX functions can be used, including DAX measures that can be used to retrieve specific aggregated results from the model. The only restriction is that the result of the formula must be a table.

A DAX query has a basic structure and can be divided into three parts. In the first part, you can define variables or measures that you would like to use in the DAX query. This part starts with the keyword `DEFINE` and is optional. The declaration of variables is the same as in regular DAX and will be further explained in *Chapter 4*, *Context and Filtering*.

The second part is the only mandatory part, and it is the core of the DAX query. It starts with the keyword `EVALUATE`. After the `EVALUATE` statement, you state the table that you would like to return, simple as that. You can simply return an existing (part of a) table that exists in the model, or you can use DAX to create custom tables based on your model and elaborate table expressions.

It's possible to include more than one `EVALUATE` statement in a single DAX query. Each `EVALUATE` statement results in a table, and the user can switch between the different output tables.

In the final part of a DAX query, we can influence how we want the resulting table to be returned. For example, you can control the sort order by adding an `ORDER BY` clause here for each of the `EVALUATE` statements. This clause is optional as well and is associated with an `EVALUATE` statement, not with the whole query if there are multiple `EVALUATE` statements.

All these keywords – `DEFINE`, `EVALUATE`, and `ORDER BY` – are not DAX functions. They are exclusively used in DAX queries.

It's possible to write and run DAX queries inside a Power BI model directly, in the DAX query view. In this view, you can write and run queries to explore your model. There are quick queries available so you don't even have to write the DAX code, but you can easily obtain information about tables and columns in your model, such as the number of rows, the number of distinct values in a column, or the minimum and maximum value of a column.

The DAX query view also identifies dependencies between measures, so it shows which measures are all used when you evaluate one specific measure. You can view the definition of these measures and change them directly in the DAX query view to see the results of the modified expression. With a single click, you can update your semantic model to reflect these changes in the measures.

The following is a simple DAX query:

```
EVALUATE
    Example4

ORDER BY [Color] ASC
```

The output of this query is the entire `Example4` table, which was defined as a calculated table in the *Calculated tables* section earlier in this chapter. The `ORDER BY` statement ensures that the table is sorted by the `[Color]` column in ascending order. Since the `[Color]` column contains textual values, the values are sorted alphabetically. The query and its output in the DAX query view are shown in *Figure 3.6*.

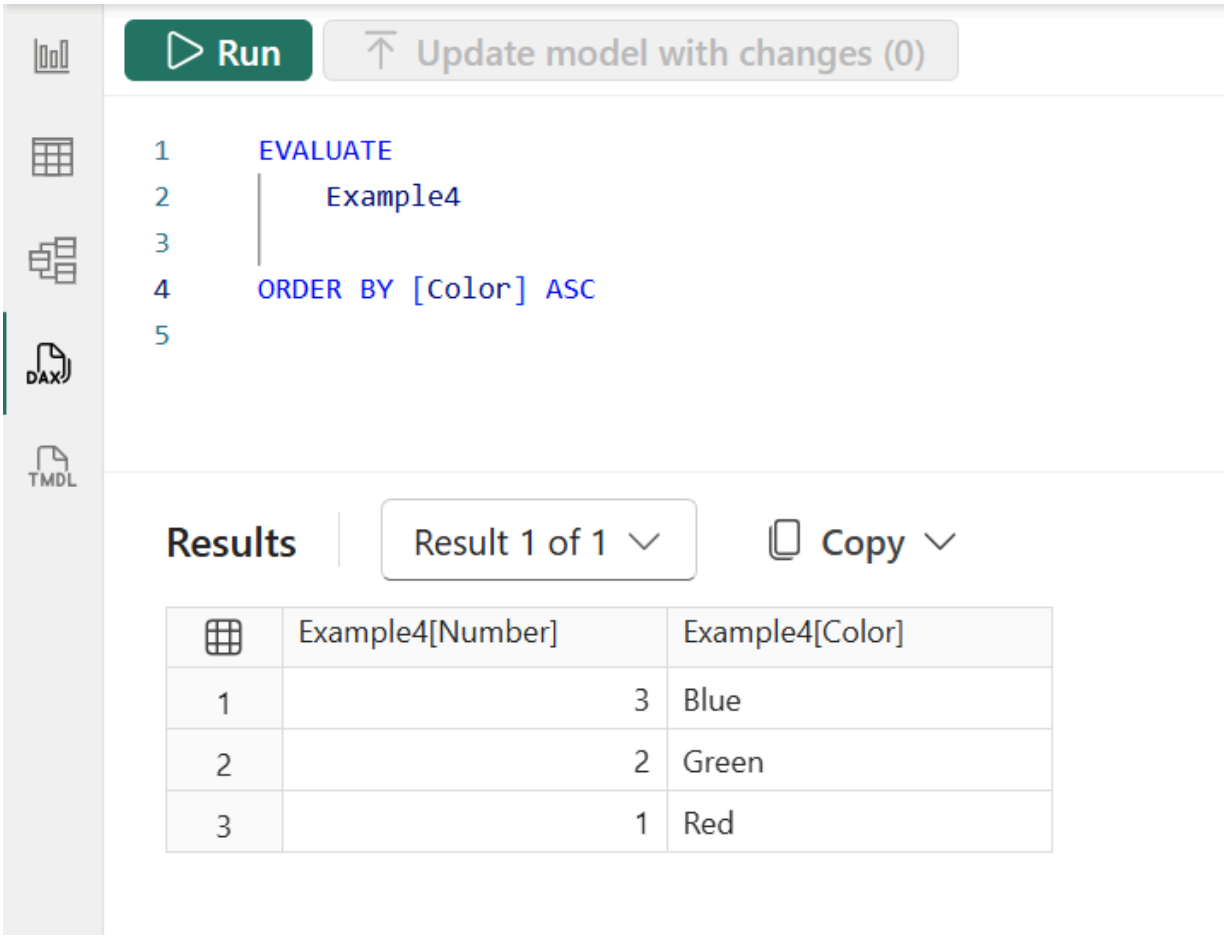


Figure 3.6: Running a DAX query in the DAX query view

Every visual in a Power BI report has a DAX query behind it to obtain the values that are necessary to populate the visual. This DAX query can be viewed by using **Performance Analyzer** and can be run directly in the DAX query view.

Another use of the DAX query view and DAX queries is to help you with debugging your model and identifying poor performance in measures.

User-defined functions

A final place where you can encounter DAX is in DAX **user-defined functions**, or DAX **UDFs** for short. A DAX UDF allows you to isolate pieces of DAX logic in a separate function with optional parameters, so you can apply it in multiple parts of your semantic model, in much the same way as a regular DAX function. You can use UDFs everywhere where you can use regular DAX: for

example, in calculated columns, measures, visual calculations, or even other UDFs.

UDFs are at least as powerful as measures, but by using their parameter options, you unleash an even greater power. You can define a function with zero or more parameters, and these parameters can be of any type: scalar, table, or even an expression type, which is a reference to a column, table, calendar, or measure (to learn more about calendars, see *Chapter 8*, *Alternative CalendarsCalendars*). By using references in your parameters, you can create really dynamic functions.

Chapter 10, *Exploring the Future: Forecasting and Future Values*,*13, Real-estate Investment Planning* discusses advanced applications of UDFs, taking forecasting to the next level. In this chapter, we'll look at defining a rather simple (and even a bit silly) DAX UDF, just to show how it can work.

DAX user-defined functions can be defined in the DAX query view, the TMDL view, or the model view of a Power BI model. Of these, the DAX query view is our preferred way. There is a new keyword in DAX to define these UDFs: `FUNCTION`. In the DAX query view, you can use the following code to define a simple function, `fnLength`:

```
DEFINE
FUNCTION fnLength = (x) => LEN(x)
```

This function takes a parameter, `x`, and returns its length. In the DAX query view, you can write this code and immediately evaluate it for a few expressions, to check whether the outcome is as expected, before actually adding the function to your semantic model.

For example, you can extend the preceding DAX code to include an `EVALUATE` statement, which asks to evaluate this new function for two static values, the number `42` and the textual value of `"Hello world!"`, as follows:

```
DEFINE
FUNCTION fnLength = (x) => LEN(x)

EVALUATE
{fnLength(42), fnLength("Hello world!")}
```

The result of this is a table with two rows, with the values 2 and 12.

NOTE

Note that the function that we have defined works for any data type of the parameter `x`: in the evaluation, we have called the function with a numeric value first and with a string value the second time. This is possible because we did not specify what type of parameter `x` is. This can lead to unexpected results. It is therefore best practice to specify the parameter type to build robust functions. When you do this, the function will try to transform the data type of the parameter value provided or return an error.

Alternatively, the function code may check for a parameter's data type and implement specific behavior based on what data type was provided.

Once you are happy with your DAX UDF, you can add it to your semantic model directly from the DAX query view. You can use the **Update model with changes** link at the top, or the pop-up message that appears, as shown here.

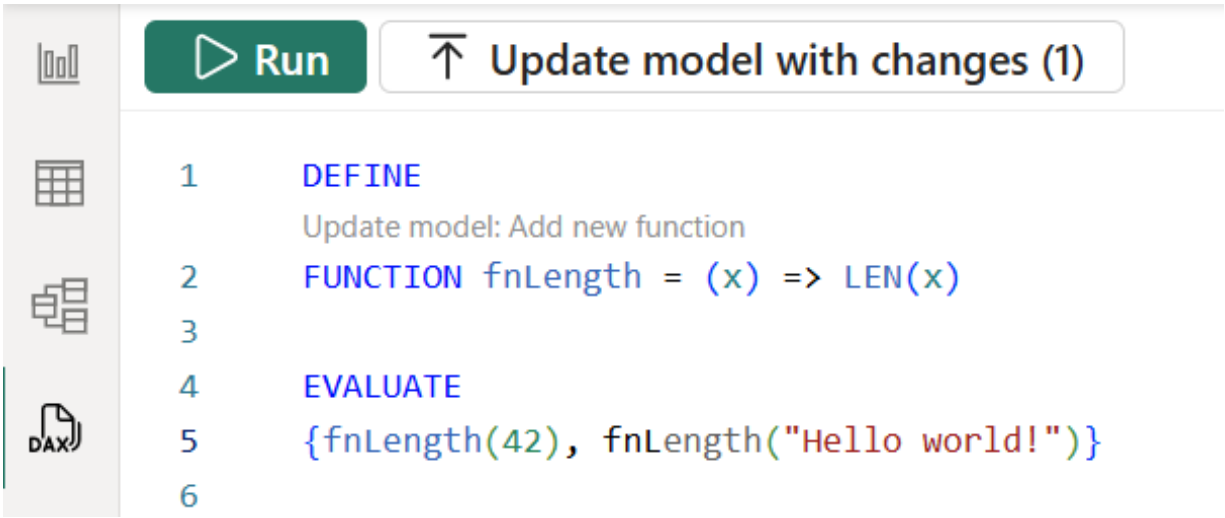


Figure 3.7: Adding a DAX user-defined function to your model in the DAX query view

Once you have added the UDF to your model, you can use it anywhere you can use regular DAX. For example, you can now create a measure that calculates the length of a selected value of the `Color` column in the calculated table `Example4`, with the help of this function:

```
Length of Color = fnLength(SELECTEDVALUE(Example4[Color]))
```

Showing this measure in a table visual together with the two columns from the `Example4` table gives this result:

Number	Color	Length of Color
1	Red	3
2	Green	5
3	Blue	4
Total		

Figure 3.8: A table visual shows the results of the `Length of Color` *measure*

Date tables

Most, if not all, Power BI models contain data that is related to dates. A **date table** (or calendar table, or whatever you like to call it) is therefore a common ingredient of a Power BI model. A date table has a special place in the model because of DAX time intelligence functions (more on these in *Chapter 4, Context and Filtering*).

A date table must have one row per day for an uninterrupted period of time. Typically, this period should be large enough to cover all the rows in your fact tables. It is advisable to start and end the date table on year start and end, respectively. The date table must have a date column, being the unique key of the table (you may choose the name of this column yourself). Other columns in the table are attributes for each day, such as year, month, quarter, weekday, and so on.

NOTE

Power BI models have an **Auto date/time** feature, which, when turned on, creates a hidden date table for each column in the model that has the date or date/time data type, supplemented with a year/month hierarchy.

If you have not yet done so, turn off this feature now! It is the cause of bloated models and performance problems. The value of these hidden tables, although convenient for users who do not want to bother with modeling, is non-existent for any experienced user. A Power BI report that delivers consistent results with a single selection of, say, year, needs a proper date table anyway.

Better still, in the Power BI Desktop options, turn off the **Auto date/time for new files** option to get rid of these tables forever.

Your table containing dates can be formally marked as such using the **Mark as date table** option. With this, the column containing dates is marked as the official date column:

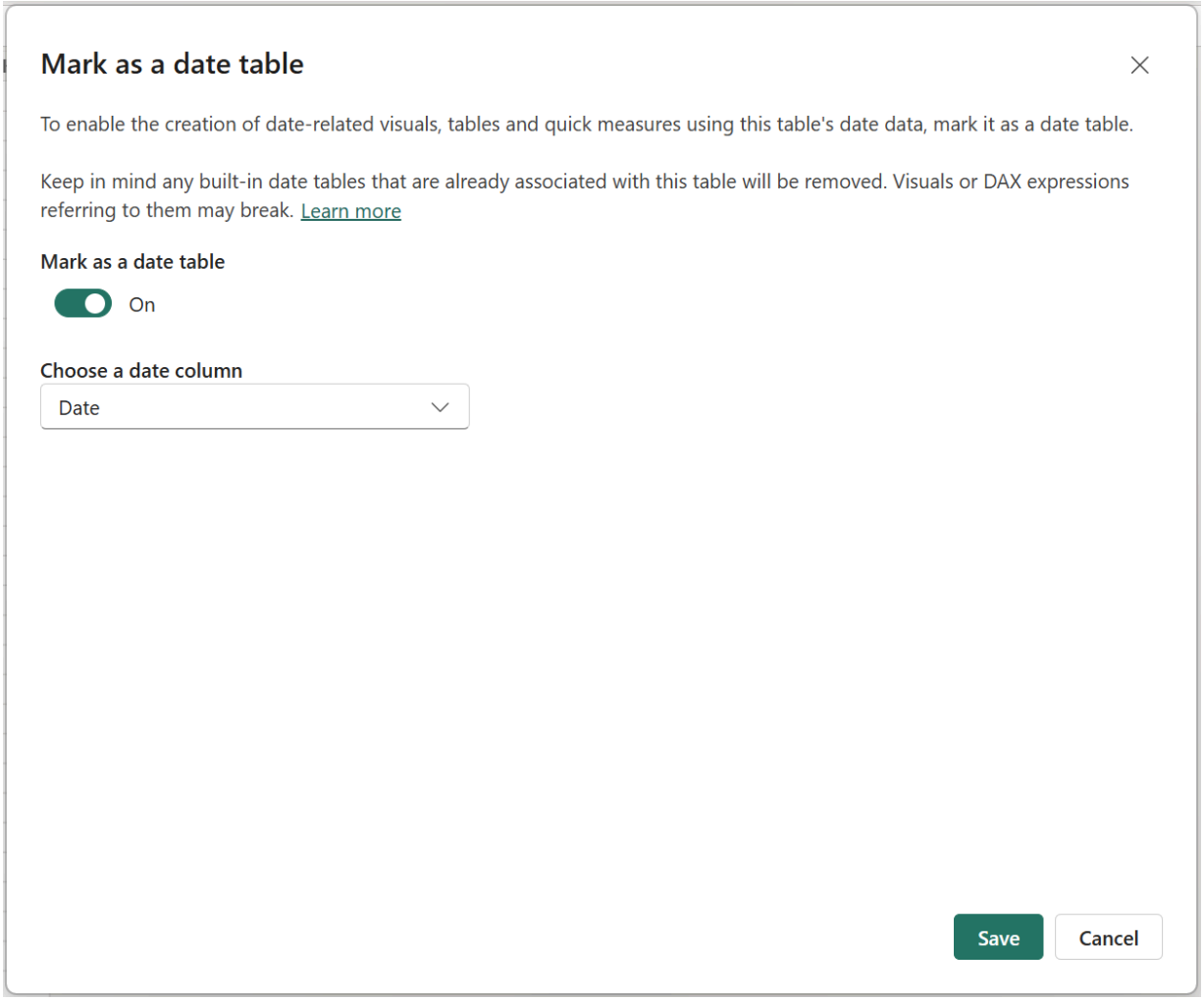


Figure 3.9: Marking a table as the date table

Creating a date table

Technically, a date table is not different from other tables. You may have calendar data available somewhere; in this case, you can just import the data into the Power BI model. There are also ways to create a date table just from some input parameters (for instance, which years the table should span) in Power Query. You will learn more in *Chapter 8*, *Alternative CalendarsCalendars*.

Here, we focus on how to create a date table as a calculated table based on a DAX statement. There are two DAX functions available that are specifically aimed at doing this: `CALENDAR` and `CALENDARAUTO`. Both functions return a one-column table containing dates.

`CALENDARAUTO` explores the whole model and finds the first and the last dates from any column with the `Date` or `Date/time` data type (calculated columns and columns in calculated tables excluded). The range of dates starts with the beginning of the year of the first date found and continues until the end of the year of the last date found. While this sounds convenient, you have to realize that when a model contains, for instance, birth dates or strange outliers such as December 31, 2199, a huge table spanning decades or even centuries will be created.

The better option is therefore `CALENDAR`. This function takes two arguments, the first and last dates of the table to be created:

```
CALENDAR(
    DATE(2025, 1, 1),
    DATE(2027, 12, 31)
)
```

As the result of this function is a single `Date` column, you will need to add more columns to create a proper date table. This can be done with calculated columns, but you can do it in one formula using the `ADDCOLUMNS` function, which does just that, adding columns:

```
Date =
ADDCOLUMNS(
    CALENDAR(
        DATE(2025, 1, 1),
        DATE(2027, 12, 31)
    ),
    "Year", YEAR([Date]),
    "Month", FORMAT([Date], "mmmm", "En-US"),
    "MonthNr", MONTH([Date]),
    "Year/Month", FORMAT([Date], "yyyy-mm")
)
```

In the preceding formula, `ADDCOLUMNS` takes the result of the `CALENDAR` function and adds columns to it. You have to provide both a name for each column and an expression that provides the corresponding value. The formula provides a nice example of the use of the `FORMAT` function, which can be used to apply a variety of formats based on, in this case, a date value, while optionally providing the locale to be used as well.

The result of the formula is as follows:

Date	Year	Month	MonthNr	Year/Month
01/01/2025	2025	January	1	2025-01
02/01/2025	2025	January	1	2025-01
03/01/2025	2025	January	1	2025-01
04/01/2025	2025	January	1	2025-01
05/01/2025	2025	January	1	2025-01
06/01/2025	2025	January	1	2025-01
07/01/2025	2025	January	1	2025-01

Figure 3.10: A date table created with a DAX formula

In a real-world model, the start and end dates of the date table need to be dynamic to reflect the loading of new data. You could, for instance, find the latest date in the `fSales` table using `MAX(fSales[OrderDate])` and use that value as the end date of the date table. You could also find the last day of the year of the last order date in the fact table using DAX. We will not elaborate on this any further here.

Best practices in DAX

When working with DAX, you will benefit from following some best practices. Applying these will help you create fast models, make it easier to maintain your models in the long term, and allow you to better support others who create Power BI reports or other output from your models.

Think in terms of DAX measures primarily

If we did not make ourselves clear enough previously, your main DAX tool should be DAX measures. These are highly dynamic, do not make the model larger, and there is no calculation that you cannot do through a measure.

As a rule of thumb, calculated columns and calculated tables are a no-go until you have very good arguments for using them!

Once you encounter duplicates of DAX code in your measures, you can start thinking about using calculation groups or functions, depending on the

situation. This makes your model easier to maintain because you don't duplicate code.

If your model starts to contain measures that only have a single use in a visual, start using visual calculations: these are especially useful if you are creating DAX code to use conditional formatting on your visuals. Instead of creating measures that you only use once, you can create a visual calculation for this.

Build explicit measures

We recommend creating explicit DAX measures instead of using numeric columns from (fact) tables directly in visual reports. There are several reasons for this:

- A measure is created by the Power BI model anyway when you use a column in a report, and it is easy to do this yourself.
- An explicit measure can be given its own clear name, such as `Total sales` instead of `Amount` or, in Power Pivot in Excel, `Sum of Amount`.
- The measure can also be given its own output format that is applied everywhere the measure is used. For instance, you may choose to show `Total sales` without a currency sign, without decimals, but with thousands separators. This can be a different format than the one derived from the data type.
- Explicit measures can be used as building blocks for more complex calculations (more on this shortly). Implicit measures are either not available or are not convenient to use because they cannot be changed.
- When you want to use calculation groups, implicit measures will be discouraged. A calculation item cannot be applied to an implicit measure; therefore, the use of implicit measures is discouraged if you want to create calculation groups anyway.

Not using numeric columns from fact tables directly has the added benefit that you do not risk using incorrect aggregations. As the `Average Price` measure discussed earlier shows, it is easy to make mistakes just by adding a column to a visual.

Use base measures as building blocks

In a DAX statement, measures can be called in order for the results of those measures to be used in the calculation. Building your set of DAX measures using basic measures (the simplest aggregations of numeric columns in fact tables) as building blocks helps to gradually create more complex calculations.

Using base measures saves you from having to think about how to calculate basic results over and over again. We see many people doing this. Moreover, base measures allow for easily adapting business logic. More often than not, some additional business logic shows up during a later stage of developing a Power BI solution. For instance, at first, you may be told, "*Sales is the sum of all invoice amounts*." But when the first results of the Power BI model become visible, you will hear, "*Oh, wait, invoices of type X do not count in sales and should be excluded*." With a base measure, all you would have to do is exclude type X invoices in the base measure's formula. Without this, you would need to go through all your sales-related measures to redo the logic.

Hide model elements

You may design a Power BI model, thinking that you are the only one creating the reports as well. But in practice, other people may start to use your model for building their own reports. For both them and you, it is good to hide elements of the model that would obscure the useful tables, columns, and measures:

- Foreign key columns of a relationship should be hidden: the same values are available as a primary key, which will correctly filter the other side.
- Technical (key) columns with no use for reporting should be hidden.
- We recommend hiding the fact tables: all foreign key columns should be hidden, and numeric columns should not be used directly but through explicit measures. You may have other columns in your fact tables; consider moving them to an appropriate filter table or removing them completely. (Some columns in a fact table may only be used to filter without being exposed to a user; they can stay in the fact table.)
- DAX measures that are only used as intermediate calculations for more complex DAX measures should be hidden.

Strategically hiding elements of the Power BI model will avoid confusion and reduce the number of questions that you, as the model designer, will get

because "*The model doesn't work.*"

Do not mix data and measures — use measure tables instead

A DAX measure always has a home table, which is the table under which the measure is shown to the model designer. More importantly, it is where the report designer sees the measure in the model's **Fields** pane when creating a Power BI report. We see many people placing measures under the fact table that contains the column being aggregated. While this is doable for simple, straightforward measures, such as basic aggregations, we advise against this practice for the following reasons:

- More sophisticated measures will aggregate columns from different tables, creating ambiguity around which table is the correct home table
- More importantly, as with calculated columns, if you ever need to delete a table and recreate it, you will lose all the measures under that table

What we do recommend is to store all measures in one or more dedicated **measure tables**. These are tables that do not contain data, but only host measures. A measure table is one of the exceptions to the *no calculated tables* rule. The simplest way to create a measure table is as a calculated table with the following formula:

```
Results = ROW("ZZ", "OK")
```

This creates a table, called `Results`, containing one column, `ZZ`, with one row of data. The value in the `ZZ` column in that single row is the text `"OK"`. The single column is needed, as a table without at least one column is not a table; but when you hide that column, Power BI will recognize this as a measure table and place it at the top of the **Fields** pane. This makes measures easy to find. The column is called `ZZ` to have it appear at the bottom of the list of measures in the table. This column, and the value in it, are never used, so you can replace `"OK"` with anything you like. The following is the result:

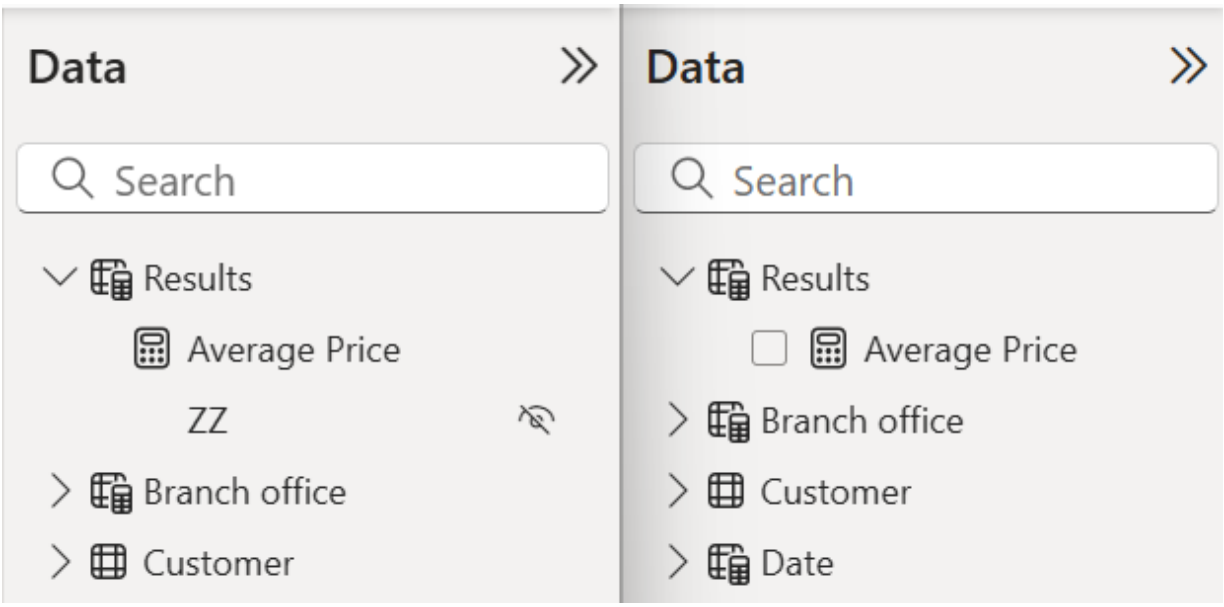


Figure 3.11: A measure table in Power BI Desktop's Table view (left) and Report view (right)

You can create a measure table in Power Query as well, for instance, through the **Enter Data** option. This works the same way: hide the data column and add a measure to have the table move to the top of the **Fields** pane.

There is a minor difference in the icon used for the measure table: when the table is imported through Power Query, a specific measure table icon is used, but when you create it as a calculated table, the generic icon for calculated tables is used:

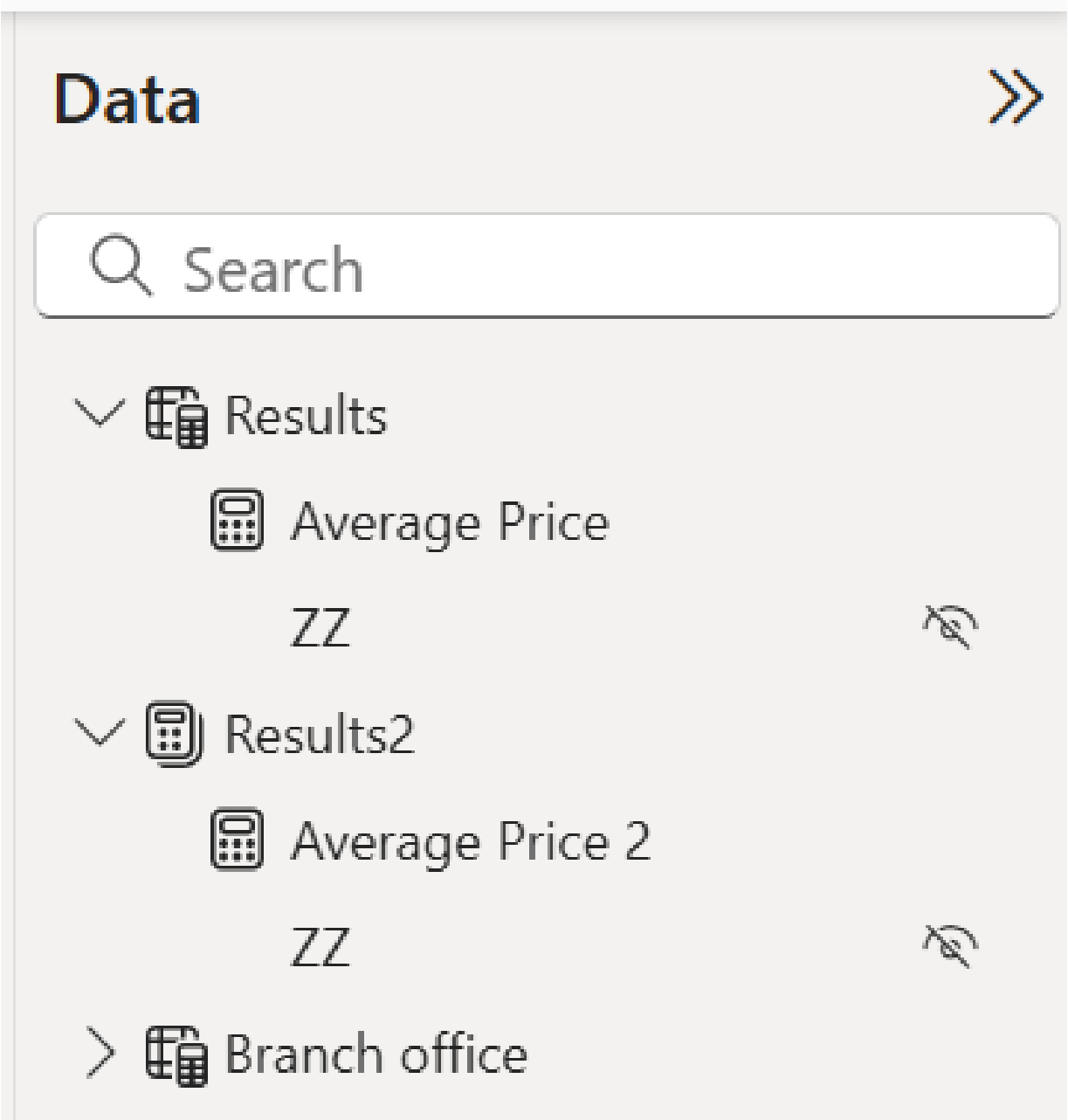


Figure 3.12: Calculated (top) and imported (bottom) measure tables

For complex models, you can use display folders to group measures. You can even decide to have multiple measure tables. For example, we have sometimes used a separate measure table for all basic measures that only serve as building blocks for higher-level calculations. By doing this, all building block measures can be hidden or made visible at once (you only need to hide the ZZ column by hand).

Table types

It is advisable to make a clear distinction between the types of tables we have discussed in this and the previous chapter. In addition to the three types already discussed, there is yet another table type, the **control table**:

- **Fact tables** contain the main data to aggregate, but no columns used in reports. Fact tables are hidden, and we give them a name with an `f` prefix.
- **Filter tables** (or dimension tables, if you will) contain all attributes on which to filter results from the model.
- **Measure tables** contain no data, but only DAX measures, and are found at the top of the **Fields** pane.
- **Control tables** are small tables used to drive specific report behavior, such as the selection of the time period reported. You will learn more about control tables in *Chapter 6*, *Dynamically Changing Visualizations*.

You don't need to distinguish between these table types by giving them specific names. We are not a fan, for instance, of adding a `dim` prefix to a filter table name: `dimCustomer`, `dimCalendar`, and so on. This is not very friendly to users of your model, who are left to wonder what `dim` could mean. You should present elements of the Power BI model in a way that speaks for itself as much as possible. Hiding fact tables, using measure tables, and giving filter tables descriptive names creates a situation where the **Fields** pane contains available (calculated) results at the top, and all attributes to filter those results below, nicely grouped into logical sets (that you, as the model designer, know to be tables).

Summary

In this chapter, you have seen the different uses of DAX in Power BI models: calculated columns, calculated tables, measures, visual calculations, security rules, field parameters, calculation groups, queries, and user-defined functions. The main takeaway is that DAX measures are (or should be) the primary way of generating valuable results from a model, and indeed, for the majority of the remainder of this book, we will focus on DAX measures. However, we will combine them with field parameters, calculation groups, and user-defined functions to enhance their flexibility even more. We have

given you some best practices for working with DAX: avoid calculated columns, use explicit DAX measures, create simple DAX measures, and use these as building blocks for more advanced calculations, use measure tables, and hide elements of a model that could confuse the report designer (even if that is yourself).

The next chapter deals with probably the most important concepts to understand when working with DAX: context and filtering. After that, we will be ready to explore advanced DAX business cases in the remaining chapters of this book.

4

Context and Filtering

Join our book community on Discord

https://packt.link/EarlyAccessCommunity

The single most important concept to understand when writing DAX calculations is **context**. Context is what separates DAX, as a data analysis language, from Excel functions or SQL queries – or Power Query scripts, for that matter. While all of these only return different results when the data changes (with some exceptions, such as when using parameters), a single DAX formula can provide many different results depending on where and how you use it: the context.

DAX context is also the key to achieving advanced results with DAX. After you have overcome typical beginner's mistakes, such as not knowing what DAX functions to use, incorrect syntax, or forgetting parentheses, issues with context are the most common problem when working with DAX. We will even go as far as to state the following:

Every problem in DAX comes from context, and every solution is found by closely examining context.

This statement is seldom negated!

In this chapter, we discuss foundational topics surrounding context that are a necessity for understanding all the content in the remaining chapters of this book. This is what this chapter covers:

- Introduction to DAX context
- DAX filtering: using `CALCULATE`
- Time intelligence
- Changing relationship behavior
- Table functions in DAX

- Filtering with table functions
- DAX variables
- DAX queries

This chapter will be relatively light on examples, as the remaining chapters demonstrate all aspects of DAX context in depth.

The Power BI model

The examples in this chapter are taken from a simple Power BI model. The model consists of one fact table, `fSales`, with some filter tables:

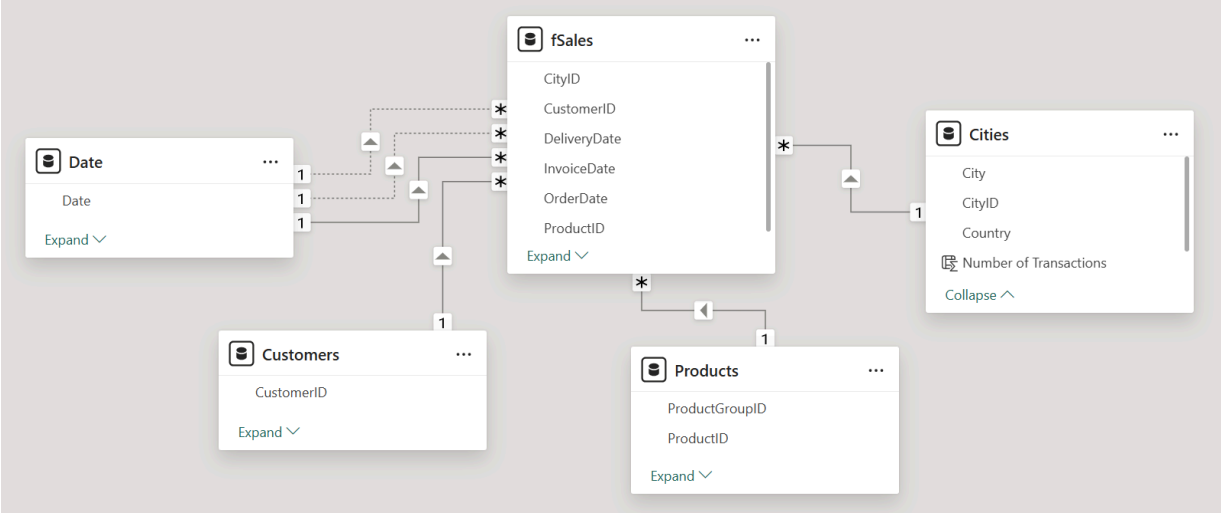


Figure 4.1: The sample Power BI model

NOTE

The model file for this chapter, `Context and Filtering.pbix`, can be downloaded from https://github.com/PacktPublishing/Extreme-DAX-Second-Edition/tree/main/Ch04.

Introduction to DAX context

The generic term for DAX context is **evaluation context**: it is the context in which a DAX formula is evaluated, leading to a specific result. We like to distinguish between three types of context:

- Row context

- Query context
- Filter context

In most Power BI documentation and publications, only two types of context are identified: *row* and *filter context*. The term *query context* has been used in relation to Power Pivot in Excel since way before Power BI came into being, and we have kept using it. In our experience from DAX courses, distinguishing between query context and filter context helps people understand more complex scenarios. Note that query context is not directly related to DAX queries, as discussed in *Chapter 3*, *Using DAX*.

Let's look at each context type in more detail.

Row context

Row context is, in some sense, the simplest context in use. This is the type of context you work with when creating calculated columns. Row context is context that works across a specific table, in a row-by-row fashion. A DAX formula is evaluated for each row in the table, and the row context provides information about the values of each column in the table. That means that in row context, you can directly reference a column and get a specific value in return, which you can use for a calculation.

The result of the calculation is, typically, specific to each row. This is because the values that are used in the calculation can be different in each row. For instance, a calculated column in the `fSales` table to compute margin as the difference between sales and costs would look like this:

```
Margin = fSales[SalesAmount] – fSales[Costs]
```

You can immediately spot that this formula is used for a calculated column by the direct reference to the `SalesAmount` and `Costs` columns. This can only be done within a row context, and this is what makes this context type different from other types. (In simple calculated column formulas, the `fSales` table reference is often omitted.)

Note that this direct reference only works for column values in the current row (the row for which the formula is currently evaluated): to retrieve values from other rows, you will need to take a completely different approach. This is a fundamental difference from calculations in Excel, where it is quite common

to take a value from *the row above this one*. It is easy to understand why, when you realize that there is no strict order between the rows in a table in a Power BI model.

There are only a few DAX functions specifically designed to work in a row context. If the table containing the calculated column is related to another table, in each row, you can retrieve the corresponding value from a column in the other table using the `RELATED` function. The formula that follows leverages a relationship between the `fSales[CityID]` and `Cities[Country]` columns to retrieve the year for each row:

```
Country = RELATED(Cities[Country])
```

The result is a `Country` column in the `fSales` table:

CityID	ProductID	UnitAmount	SalesAmount	Country
472	280	1	1.3053	United States
174	280	1	1.3053	Germany
174	280	1	1.3053	Germany
597	280	1	1.3053	United States
41	280	1	1.374	Canada
41	280	1	1.374	Canada
539	280	1	2.1755	United States
310	280	1	2.1755	United States
193	280	1	2.1755	France
38	280	1	2.1755	Australia

Figure 4.2: Adding a Country calculated column (some columns removed for readability)

There is one restriction required for `RELATED` to work: the relationship must have unique values in the "other" table (that is, it must be a one-to-one or many-to-one relationship to the related table) – in this example, the `Cities` table. After all, the formula must result in a single value.

When the cardinality of the relationship is reversed, you can use the `RELATEDTABLE` function. For example, if you wanted to add a calculated

column to the `Cities` table with the number of sales transactions for each city, the following formula would do the trick:

```
Number of Transactions = COUNTROWS(RELATEDTABLE(fSales))
```

The `RELATEDTABLE` function results, for each row in `Cities`, in the set of rows in `fSales` that are related to that row. As this is a table, it cannot be used directly as values in the calculated column; in this case, we use `COUNTROWS` to simply count the number of rows in that table.

Although `RELATEDTABLE` is meant for use in a row context, it is fundamentally different from `RELATED` in that it uses a different context type under the hood.

> **NOTE**
>
> As discussed in *Chapter 3, Using DAX*, we discourage the use of calculated columns. This does not mean that you will not have to deal with row context. Row context plays an important role in DAX table functions as well. More on that later in this chapter.

We'll move on to the other context types now. There are some peculiarities about row context that can be made clear after we have discussed query and filter context.

Query context

You work with **query context** when you use DAX measures. Like row context before, query context is what makes a DAX measure return a specific result. The difference, of course, is that we are not working within a single table. In short, query context refers to the set of rows in a Power BI model that are selected and over which the DAX formula is evaluated. It helps to distinguish between two separate, although closely related, elements in the query context:

- A **selection** refers to the set of rows in each table in the model that are selected in a specific context.
- A **filter** is an instruction to select one or more values in one specific column in the model. This causes rows to be selected.

In a query context, the filters come from elements in a (Power BI) report. These come in various sorts: slicers, filters in the filter pane, labels in a visual, or selected items in another visual. Each of these forms a specific instruction for a selection within a column; for instance, in *Figure 4.3*, the slicer induces a filter on the `Year` column: *Year equals 2026*. Furthermore, the value of `2.5bn` is returned for the month of February, because the month label in the visual creates an additional selection on the `Month` column: *Month equals February*.

There can be many filters on different columns, and there may even be multiple filters on the same column. Together, the filters determine which rows are selected in each table: all rows that satisfy every filter instruction.

> **NOTE**
>
> There is a thorough discussion of visual filters in *Chapter 9, Working with Auto-Exist*.

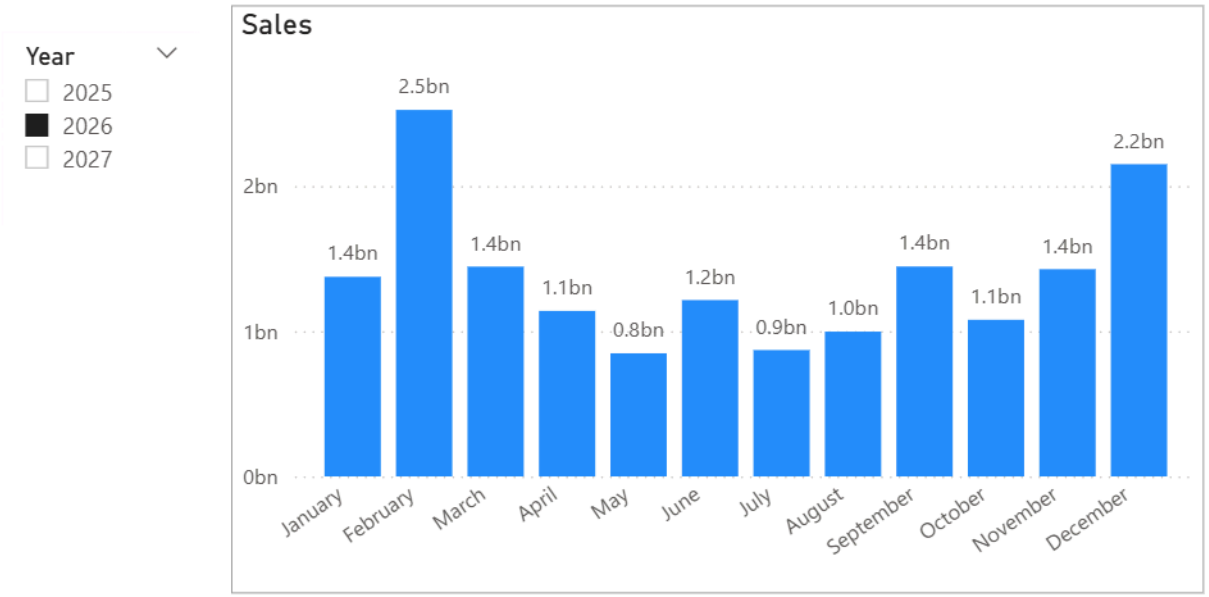


Figure 4.3: A simple Power BI report

In a query context, relationships between tables play an important role: that of **filter propagation**. This means that a filter on a column in one table is propagated by the relationship to the other table, in the relationship's cross-filter direction:

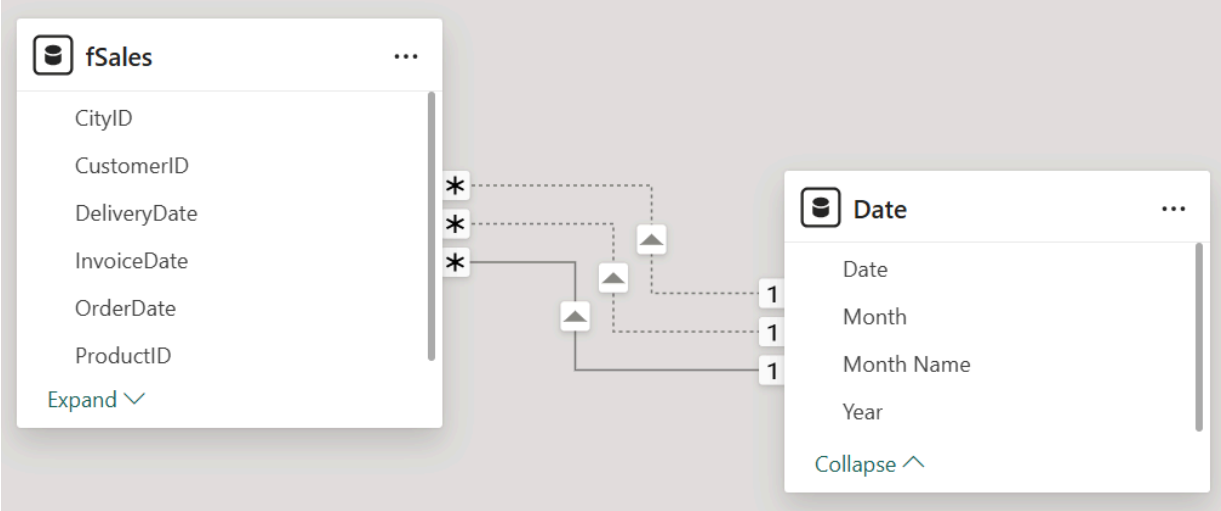


Figure 4.4: Filter propagation by a relationship happens in the direction of the arrow

Strictly speaking, it is not the filter itself that is propagated, but rather the effects of the filter: in the related table, only rows that correspond to rows that satisfy the filter rule are selected. This is exactly the behavior that is needed: when a slicer is set on the year 2026, as in *Figure 4.3*, we want to see results for 2026, which means that all calculations should only be done on rows in the fact table that correspond to dates in 2026.

Because of the nature of the query context, you cannot directly use columns in a formula as you can in a row context. The following formula, as a measure, is not accepted by the DAX editor:

```
Report Year = 'Date'[Year]
```

It results in an error message: **A single value for column 'Year' in table 'Date' cannot be determined**. The reason is that, conceptually, the selection could contain multiple values. This is true even when the column contains only one unique value, or when the table contains only one row.

Filter context

Filter context looks similar to query context, with one important exception: filter context is a context changed by DAX code, for example, by adding or changing filters in a query context. The central DAX function that is used for this is `CALCULATE` (or its sibling, `CALCULATETABLE`). The way changing context

with CALCULATE works is discussed in depth in the *DAX filtering: using CALCULATE* section later in this chapter.

What makes filter context difficult is that you cannot determine the filters in the context from the visual, like you can with query context. Working with filter context requires a level of abstract thinking and meticulously keeping track of which filters are active in a specific situation. For the same reason, measures that create and use filter context can be confusing to the report users and should therefore be applied with care, by using self-explanatory measure names, for example.

Being able to change context opens up a plethora of possibilities not available with merely row context and query context. It allows us to obtain results that do not correspond with the query context, and that can be used to provide advanced insights, such as comparing sales for a product with sales of all products, comparing this year's sales to last year's, extrapolating trends into the future, and so on. In fact, none of the scenarios discussed in the remaining chapters of this book are possible without a filter context.

The general approach to creating sophisticated insights with DAX can be described as follows:

1. Study the (possible) query contexts in which your calculation will be evaluated.
2. Determine what filter context is needed to return the required result.
3. Determine how to go from query context to filter context.

To master DAX, you should acquaint yourself with this way of thinking, which is fundamentally different from retrieving data with SQL, programming, or doing calculations in Excel.

Detecting filters

Filters in an evaluation context cause the selection of rows in the tables of a model. When you consider the effect of this on a single column, several things can happen. It could be that no selection is made, such that all values in the column are in the context. It could also be that a subset of values is selected. This can be caused by a filter on that column, in which case we say that the column is filtered *directly*, or it can be caused by a filter on another column in the same table or a filter in another table, which is propagated by a

relationship. No matter where that filter comes from, we say that the column is *indirectly* filtered or *cross-filtered*.

DAX contains a number of functions to detect the filters in the context and their effect. Each function takes a column reference, say column A, as an argument:

- `ISFILTERED`: Returns `True` when there is a filter directly on column A.
- `ISCROSSFILTERED`: Returns `True` when there is a filter on any column in the model that causes a selection in column A.
- `HASONEFILTER`: Returns `True` when a (direct) filter on column A selects *exactly one* value.
- `HASONEVALUE`: Returns `True` when a filter on any column in the model causes a selection of *exactly one* value in column A or, by coincidence, column A contains a single value to begin with.
- `FILTERS`: Returns a table with the selection in column A that is the result of the *direct* filters on that column.
- `ISINSCOPE`: Returns `True` when *exactly one* value is selected in column A, caused by a filter on column A, which is used as a grouping column or label in the visual. This function is designed to detect the current drill-down level in a visual that allows for drill-down.
- `ISATLEVEL`: A similar function to `ISINSCOPE`, but `ISATLEVEL` is only available in visual calculations. When you want to detect filters in visual calculations, `ISATLEVEL` is recommended to ensure that it works as expected in the context of the visual matrix, in which visual calculations are defined.

These functions can be of help if you want to investigate what the context looks like. They can also be used to implement specific DAX measure behavior, although there are pitfalls along the way. You can find some examples in *Chapter 5*, *Security with DAX*.

Comparing query and filter context to row context

Now that we have covered query and filter context, we can look at row context from a different perspective. As an example, suppose you create a calculated column in the `fSales` table with the following formula:

```
TotalTax = SUM(fSales[Tax])
```

You will find that in the resulting `TotalTax` column, each row contains the same value. The `SUM` function calculated the total of *all* rows in the table, even though we are in row context for a single row. To DAX beginners, this is often a surprising find. Let's look at another example – this time, a calculated column in the `Date` table:

```
TotalShipping = SUM(fSales[ShippingCosts])
```

Again, you will find that same result in every row, even though there is a relationship between the `fSales` and `Date` tables. Shouldn't the relationship cause the calculation to return the total shipping costs for each day separately?

These examples teach us something about the nature of row context. The `TotalShipping` example suggests that in a row context, relationships do not propagate selections. This is a useful rule of thumb to work with, but the reality is a bit more subtle. In a row context, DAX specifically allows for using column values from the same table; outside of that, nothing is selected or filtered. In a calculated column, there are no filters on any column in the table. As a consequence, relationships have nothing to propagate. This means that when you refer to another table, as the `TotalShipping` calculation does, you work with the complete table. Even when you refer to the table in which the calculated column is, as in the `TotalTax` calculation, you are looking at all rows.

As a consequence, if you are in a row context but need a relationship to propagate, you must find a way to *change the row context to a filter context*. And for that, you must use the `CALCULATE` function.

DAX filtering: using CALCULATE

Changing context using DAX code is one of the most powerful features of DAX. The DAX `CALCULATE` function, which is used for *context transformation*, is arguably the most important DAX function. By specifying filter expressions in `CALCULATE`, you can control the subsets of rows your formula works on. This can be done by adding or replacing filters, but also by removing filters from

the context. As relationships play an important role in context by propagating filters, activating or inactivating relationships, or changing their filter propagation behavior, is a form of context transformation as well.

Let's start with a sample DAX measure:

```
SalesLargeUnitAmount =
CALCULATE(
    SUM(fSales[SalesAmount]),
    fSales[UnitAmount] > 25
)
```

This measure returns the sales on transactions in which more than 25 units were sold. The first argument of `CALCULATE` is the calculation to be performed, in this case, the sum of the `SalesAmount` column in `fSales`. All other arguments – and there can be many – are filter arguments.

NOTE

DAX allows you to write a formula without explicitly using the `CALCULATE` function when the calculation is in a separate measure. For instance, this is a basic `Sales` measure:

```
Sales = SUM(fSales[SalesAmount])
```

The sample measure we saw previously can also be rewritten as follows:

```
SalesLargeUnitAmount =
    [Sales] (fSales[UnitAmount] > 25)
```

We advise against this syntax, as formulas using an explicit `CALCULATE` are more readable, especially in more complex formulas.

To understand how `CALCULATE` works, you should keep in mind that the function performs four basic steps in order:

1. The existing context (either row or query context, or another filter context) is changed into a filter context.
2. Existing filters, if any, on columns (or entire tables) referred to in the filter arguments are removed.
3. New filters are added.

4. The expression in the first argument is evaluated in the new filter context.

Several specific DAX functions can be used in filter arguments to change this behavior, but let us first focus on these steps, working with the `SalesLargeUnitAmount` measure as an example.

Step 1: Setting up a filter context

The first thing to do for `CALCULATE` is to create an environment in which filters can be changed. When we start in a query or filter context, we already have that environment, so there is nothing to do. So, for our `SalesLargeUnitAmount` measure, this step is trivial. When we start in a row context, however, the differences are huge.

The transition from row context to filter context is achieved by creating a filter on each column in the table that prescribes the value in that column to be the column's value in the current row (remember that a row context is always about a single row). The result is a filter context where the current row is selected. In addition to this, if there are relationships from this table to other tables, these relationships now have filters to propagate, and therefore, we can expect subsets of rows to be selected in those other tables as well.

For example, we can change the formula for the `TotalTax` calculated column used earlier by wrapping it in `CALCULATE` (note that we use `CALCULATE` without any filter argument here):

```
TotalTax2 = CALCULATE(SUM(fSales[Tax]))
```

For each row, the formula changes the row context into a filter context. The `SUM` function now works on the selected rows, which is only the current row. In other words, the result is just the value of the `Tax` column in the row itself:

Tax	TotalTax	TotalTax2
330.74	170,860,557.27	330.74
183.74	170,860,557.27	183.74
330.74	170,860,557.27	330.74
661.48	170,860,557.27	661.48
330.74	170,860,557.27	330.74
330.74	170,860,557.27	330.74
771.72	170,860,557.27	771.72
183.74	170,860,557.27	183.74

Figure 4.5: The effect of CALCULATE in a calculated column

In the case of the `TotalShipping` calculated column, in the `Date` table, the same thing happens:

```
TotalShipping = CALCULATE(SUM(fSales[ShippingCosts]))
```

The row context in the `Date` table is transformed into a filter context that has filters on each column of the table. This time, the relationship between the `fSales` and `Date` tables can propagate the selection, causing the set of rows in `fSales` to be selected that correspond to the current row in `Date`:

Date	Year	Month	Month Name	TotalShipping
Wednesday, 13 May 2026	2026	5	May	8,647.77
Thursday, 14 May 2026	2026	5	May	9,238.92
Friday, 15 May 2026	2026	5	May	9,817.61
Saturday, 16 May 2026	2026	5	May	5,244.29
Sunday, 17 May 2026	2026	5	May	6,854.08
Monday, 18 May 2026	2026	5	May	6,929.10
Tuesday, 19 May 2026	2026	5	May	8,878.40
Wednesday, 20 May 2026	2026	5	May	10,027.47
Thursday, 21 May 2026	2026	5	May	6,904.50
Friday, 22 May 2026	2026	5	May	4,931.97

Figure 4.6: CALCULATE causes relationships to propagate filters

The result of adding `CALCULATE` is that we now have a correct `TotalShipping` amount per day.

NOTE

You may wonder how realistic it is to use `CALCULATE` without filter arguments, but you do this more often than you may think. Each time you call a measure within a formula, `CALCULATE` is implicitly put around it, as in this example:

```
SalesByCustomer =
    DIVIDE([Sales], [Number of Customers])
```

In this formula, both the calculation for `Sales` and for `Number of Customers` are done in a filter context. Calling a measure is a common way to go from row context to filter context, which is something you often need to do.

Now that the initial filter context is in place, `CALCULATE` can proceed to the next step.

Step 2: Removing existing filters

The second step in the working order of CALCULATE is to remove filters from the new filter context. The process is straightforward: each column that is referenced in one of the filter arguments is checked for filters. If a filter exists on that column, it is removed.

In our example SalesLargeUnitAmount measure, the single filter argument is as follows:

```
fSales[UnitAmount] > 25
```

This filter argument leads CALCULATE to remove any existing filter on the fSales[UnitAmount] column.

What is sometimes forgotten is that columns that are not mentioned in filter arguments keep their filters (when they exist). As you do not have full control over what the original context looks like, you should take care in going over the different scenarios your measure may be used in. You may need to remove more filters than you initially expect. You will find an example of the impact that other filters can have after we have covered all four steps.

Sometimes, the behavior of CALCULATE in step 2 is not what you want. This behavior can be changed by using the KEEPFILTERS function. This function causes CALCULATE to skip step 2 for the filter argument you apply KEEPFILTERS to, as in this example:

```
SalesLargeUnitAmount KeepFilters =
CALCULATE(
    [Sales],
    KEEPFILTERS(fSales[UnitAmount] > 25)
)
```

In this formula, whenever there is an existing filter on the UnitAmount column, it is kept, *and* a new filter is added in step 3.

Step 3: Applying new filters

The third step that is performed by CALCULATE is applying new filters. As in step 2, the function goes over its filter arguments and takes these as instructions for creating new filters. In the example, the fSales[UnitAmount] > 25 filter argument causes a new filter to be created on the UnitAmount column, selecting all values larger than 25.

In understanding `CALCULATE`, it is very helpful to remember that steps 2 and 3 are applied in that order. The order of the filter arguments themselves is irrelevant. As a simple example, consider the following DAX statement:

```
Sales373_374 =
CALCULATE(
    [Sales],
    Products[ProductID] = 373,
    Products[ProductID] = 374
)
```

Many beginners in DAX expect this formula to return sales for product 374, reasoning that the filter is first set on `ProductID` 373, and then on 374. In reality, this measure will always return blank, because two filters on `ProductID` are added, which require the column to equal both 373 and 374. There are no rows in the `Products` table that satisfy these rules, and the `Sales` measure will therefore return a blank value (assuming there is a relationship propagating filters from the `Products` table to the `fSales` table).

You may consider this a trivial example; however, in a slightly different form, it fools many:

```
Sales373OrWhat =
CALCULATE(
    [Sales],
    Products[ProductID] = 373,
    ALL(Products)
)
```

As for the previous example, the correct way to understand what happens is to realize that filters are removed for each filter argument before new filters are added. The result is sales for `ProductID` 373.

Step 4: Evaluating the expression to calculate

The last step in the working order for `CALCULATE` is easy: after setting up a filter context, removing filters, and adding new filters, the expression in the first argument can be evaluated in the new context. This is the proof of the pudding, of course, as this is where we can really see what the effect of all the context transformations is.

As an example of how filter manipulation can be tricky, take the following matrix visual.

Group	Sales373	SalesRearWheel525	Sales
⊞ **Oil filters**			**190,215,158.25**
⊞ **Other**			**11,497,355,884.13**
⊟ **Rear wheel**	**735,209,424.43**	**735,209,424.43**	**2,391,842,110.38**
228	735,209,424.43		21,543,668.83
239	735,209,424.43		1,606,503,554.30
373	735,209,424.43	735,209,424.43	735,209,424.43
466	735,209,424.43		5,569,759.39
467	735,209,424.43		23,015,703.44
⊞ **Transport**			**415,907,579.32**
Total	**735,209,424.43**	**735,209,424.43**	**14,495,320,732.08**

Figure 4.7: Output for example measures

In this matrix, we show information about products using the `Group` and `ProductID` columns as labels. We focus specifically on product 373 in the `Rear wheel` product group. The name of this product, which is in the `Product` column, is `REAR WHEEL STEEL #525`. We want to be able to compare each product's sales with product 373's sales. You could think of this as product 373 being the most strategic product for our company, and we want to express each product's sales as a percentage of the sales of product 373.

To accomplish a comparison, we need a calculation that returns sales for product 373 in each row of the visual. We try this using two measures. This is the first:

```
Sales373 =
CALCULATE(
    [Sales],
    Products[ProductID] = 373
)
```

This is the second:

```
SalesRearWheel525 =
CALCULATE(
    [Sales],
```

```
    Products[Product] = "REAR WHEEL STEEL #525"
)
```

The `Sales` measure is in the visual, so you can better see what is happening. In most of the rows in the visual, two filters exist in the query context: one on the `Group` column and one on the `ProductID` column. The exceptions are the subtotal rows (with a filter on `Group` only) and the grand total row (with no filters).

Clearly, the two measures using `CALCULATE` return different results. Why this difference? As `Sales373` references the `ProductID` column in the filter argument, any existing filter on that column is removed (in step 2) before the new filter is added (in step 3). On the visual's row for product 239, for instance, the filter *ProductID equals 239* is removed, and the filter *ProductID equals 373* is added. As a result, the calculation returns sales for product 373.

Something else happens in the `SalesRearWheel525` measure. Here, the filter argument references the `Product` column, so any existing filter on the `Product` column is removed (step 2). After that, the new filter is added (step 3). Looking at product 239 again, the query context contains filters on `Group` and `ProductID`. The measure does not remove these, but adds the new filter on the `Product` column. The selection in the resulting filter context is all the rows in the `Product` table that satisfy all three filters; in other words, no rows are selected, and the result is blank, unless the three filters happen to point to the same product. This is true only for product 373 itself, which is why that is the only row in the visual that shows a result.

The same reasoning explains why the `Sales373` measure does not return results in groups other than `Rear wheel`: when a filter on `Group` selects another group, the combination with `ProductID` 373 (the newly added filter) leads to an empty selection on the `Product` table.

Removing filters with ALL functions

Both measures from the previous section have the same problem, of course, depending on the context. To create a measure that always returns the sales for product 373, regardless of what selection of products is made in the query context, we must get rid of any filter that may get in the way.

It is important to have precise control over which filters are removed. To enable this, a category of DAX functions is available that we call the `ALL` functions. The differences between these functions are in which filters are removed:

- `ALL`: This function can take one or more column references or a table reference as arguments. It removes filters from the columns specified, or from all columns in the table referenced. If you really want to, you can use `ALL` without arguments to remove all filters from the entire Power BI model. Here are some examples:

  ```
  ALL(Cities[Country])
  ALL(Cities[Country], Cities[State])
  ALL(Cities)
  ALL()
  ```

  When using `ALL` on a table, any filters on columns in related tables are also removed. For instance, `ALL(fSales)` would remove filters from the `Cities` table as well when there is a many-to-one relationship between `fSales` and `Cities`. See also `ALLCROSSFILTERED`.

- `ALLEXCEPT`: This function can be used as an alternative to `ALL` with many column arguments. Instead of mentioning all columns you want to remove filters from, you specify a table and the columns in that table that should keep their filters. Filters from all other columns in the table are removed. Here is an example:

  ```
  ALLEXCEPT(Cities, Cities[Country])
  ```

- `ALLNOBLANKROW`: When you use `ALL`, the resulting context includes all values from the specified columns. This includes values from a possible blank row that is added to the table because of an incomplete relationship (see *Chapter 2*, *Model Design*; these values are necessarily blank). If you do not want these blank values to be included in the context, you should use `ALLNOBLANKROW` instead of `ALL`. This function takes one argument, either a column or a table. Here is an example:

  ```
  ALLNOBLANKROW(Cities[Country])
  ```

- `ALLSELECTED`: This is a special `ALL` function, as it is the only function that is aware of the origin of filters. `ALLSELECTED` removes filters, but only when these filters originate from labels within the visual where the measure is used. External filters, such as those coming from slicers, page filters, or other visuals, are left untouched. This function is used to create measures that aggregate selected items within a visual, for instance, to compute a percentage that always adds up to 100% in the total for the visual. The function takes a table, one or more columns, or nothing as arguments. Here is an example:

```
ALLSELECTED(Cities[Country])
```

- `ALLCROSSFILTERED`: This function was introduced for use in composite Power BI models, or models that include a mix of DirectQuery and import tables, or different DirectQuery connections. Relationships between tables of different origin in such a model are limited and do not provide the standard behavior: `ALL(fSales)` would not remove filters from the `Cities` table when `fSales` is a DirectQuery table, and `Cities` is imported. The `ALLCROSSFILTERED` function takes a table reference as an argument and will remove filters from both that table and related tables, even when relationships are limited. Here is an example:

```
ALLCROSSFILTERED(fSales)
```

NOTE

There is an alternative DAX function you can use to remove filters within a `CALCULATE` statement: `REMOVEFILTERS`. This function takes one or more columns or an entire table as an argument, as i this instance:

```
CALCULATE(
    [Sales],
    REMOVEFILTERS(Cities)
)
```

This function was introduced as an easier-to-understand alternative to `ALL`. We prefer the shorter `ALL`, and never use `REMOVEFILTERS`.

By carefully choosing one or more `ALL` functions, you can make `CALCULATE` do exactly what you want. Remember that we wanted to create a measure that always returns sales for product 373; in other words, we know exactly what we want the filter context to look like. We do not have control over what filters exist in the query context we start with, but we do have control over what filters are removed. Consider this updated measure:

```
SalesRearWheel525_ALL =
CALCULATE(
    [Sales],
    Products[Product] = "REAR WHEEL STEEL #525",
    ALL(Products)
)
```

With this formula, any existing filter from the `Products` table is removed before adding the filter on the `Product` column. The difference is clear:

Group	Sales373	SalesRearWheel525	SalesRearWheel525_ALL	Sales
⊞ **Oil filters**			**735,209,424.43**	**190,215,158.25**
⊞ **Other**			**735,209,424.43**	**11,497,355,884.13**
⊟ **Rear wheel**	**735,209,424.43**	**735,209,424.43**	**735,209,424.43**	**2,391,842,110.38**
228	735,209,424.43		735,209,424.43	21,543,668.83
239	735,209,424.43		735,209,424.43	1,606,503,554.30
373	735,209,424.43	735,209,424.43	735,209,424.43	735,209,424.43
466	735,209,424.43		735,209,424.43	5,569,759.39
467	735,209,424.43		735,209,424.43	23,015,703.44
⊞ **Transport**			**735,209,424.43**	**415,907,579.32**
Total	**735,209,424.43**	**735,209,424.43**	**735,209,424.43**	**14,495,320,732.08**

Figure 4.8: Using ALL

Not only is the result for product 373, `REAR WHEEL STEEL #525`, returned for the other products, but the result is even returned when the query context filters other product groups. With this, we have the essential building block for expressing any product's sales relative to product 373's sales, with a simple division:

```
Sales% = DIVIDE([Sales], [SalesRearWheel525_ALL]
```

With a combination of filter arguments and `ALL` functions, many powerful DAX measures can be created. Some filters are more difficult to create and

specify than others, however, and among these are filters that deal with the calendar. This is why DAX contains a special category of functions for this purpose, which we discuss in the next section.

Time intelligence

There are probably hardly any Power BI models that do not include some analysis over time. We want to compare current results with those of last year, for instance. Many other calendar-related insights may also be needed, such as year-to-date results, rolling totals, or growth rates from any other past period of time. Working with calendars is usually quite difficult. For example, the Gregorian calendar is quite messy: most years have 365 days, but some have 366, and months can have anywhere between 28 and 31 days. And then there are even more calendars with other challenges. In *Chapter 8*, *Alternative Calendars*, we will explore this further.

These calendar complexities notwithstanding, calendar-based analysis is simply about filtering to change context. Consider the following year-to-date sales chart:

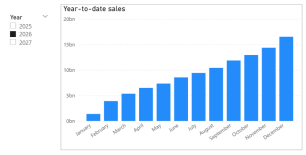


Figure 4.9: A year-to-date chart

By definition of year-to-date, what you see in, say, the `August` column, is the total sales in the period of January 1, 2026, up until August 31, 2026. However, the query context for this column contains a filter on `Year` (2026) and `Month` (August), causing a selection of August 1 to August 31, 2026. Clearly, the context must be transformed along the way to be able to return the year-to-date sales.

In the calculation for year-to-date sales, you would therefore expect the use of `CALCULATE` with some filter arguments:

```
SalesYTD =
CALCULATE(
    [Sales],
```

```
    ... (some filter argument)
)
```

Indeed, DAX time intelligence functions implement filter arguments in `CALCULATE` to deal with the complexities of the calendar. The year-to-date filter is provided by the `DATESYTD` function:

```
SalesYTD =
CALCULATE(
    [Sales],
    DATESYTD('Date'[Date])
)
```

The `DATESYTD` function works on the basis of the query context on the `Date` table. Its recipe looks like this:

1. Retrieve the last date in the context.
2. Determine the year this date falls in, and the first day of that year.
3. Create a filter on the `Date[Date]` column, selecting everything from the first day of the year to the last date in the context.

NOTE

With the new context, `CALCULATE` can do its job evaluating, in our example, the `Sales` measure. But wait: the `DATESYTD` filter argument references the `Date` column, but in the chart in *Figure 4.9*, there are filters on the `Year` and `Month` columns! This is a very common situation, and therefore, `DATESYTD` takes one more step, as follows.

1. Add an implicit `ALL('Date')` filter argument.

This last step is common to all time intelligence functions, and saves us from having to add explicit `ALL` functions all the time.

NOTE

The implicit `ALL('Date')` clause is only added when you have formally marked your table with dates as the Power BI model's `Date` table, or when the relationships from fact tables to your `Date` table are created on columns with the `Date` data type. Although the time intelligence functions can therefore work correctly without a formal

> `Date` table declaration, we strongly recommend using this declaration anyway.

As mentioned, time intelligence is a very common need. For this reason, several time intelligence functions offer shorter, easier-to-use versions as well. The `DATESYTD` function, used in combination with `CALCULATE`, can be replaced by the `TOTALYTD` function:

```
SalesYTD_short =
TOTALYTD(
    [Sales],
    'Date'[Date]
)
```

This formula is exactly the same, although syntactically different. While that may be an advantage, the downside is that, to many beginners in DAX, this function looks like it calculates a year-to-date total. In reality, the only thing `TOTALYTD` does is change the context. It may return a year-to-date average or year-to-date anything; it all depends on the measure or expression used as the first argument.

> **NOTE**
>
> For the sake of completeness, it is possible to use `DATESYTD` and `TOTALYTD` for fiscal years that start at another date than January 1. `DATESYTD` allows for a second argument, which it should be able to derive a day in the year from, such as 8/31 or 2020/9/30; this is taken as the last day of the year. For reasons we have never really understood, in `TOTALYTD`, this argument is the fourth argument, which means that you must include a third argument. The concept of skippable parameters has not reached this function yet, unfortunately. The third argument requires an additional filter. You can safely use `ALL('Date'[Date])` here, as that is implicitly added anyway.

Next to `DATESYTD`, the time intelligence functions used the most are the following:

- `SAMEPERIODLASTYEAR`: As the name says, the function takes the current context and moves it back exactly one year. This is, of course, what you need to compute year-on-year growth numbers. Surprisingly, there is no shortcut version of the function. The sales for last year could be calculated with the following:

```
SalesLY =
CALCULATE(
    [Sales],
    SAMEPERIODLASTYEAR('Date'[Date])
)
```

- `DATESINPERIOD`: This function can be used to return a period starting at (or ending on) a certain reference date, and with a specified length expressed in units of days, weeks, months, quarters, or years. This is the function you need to compute rolling totals. For example, the 12-month rolling total of sales (thus looking back 12 months) is calculated using the following formula. Here, `MAX('Date'[Date])` is used to retrieve the last day in the context as the reference date:

```
SalesRollingTotal =
CALCULATE(
    [Sales],
    DATESINPERIOD(
          'Date'[Date],
          MAX('Date'[Date]),
          -12, MONTH
   )
)
```

When working with time intelligence functions, it is important to keep in mind that each time intelligence function transforms the context on the `Date` table – nothing else. This means that any table in the model that is not related to the `Date` table will not be impacted by this context transformation. It also means that you will get unexpected results when your `Date` table is "short." For example, if your `Date` table starts with the year 2026 and you use the `SAMEPERIODLASTYEAR` function in a context that selects dates in February 2026, the result will be an empty context on the calendar. That will lead to a blank result for the measure, even if the fact table you aggregate over does have dates in 2025 or earlier.

Changing relationship behavior

In *Chapter 2*, *Model Design*, you learned that there can be multiple relationships between tables, but only one of those relationships can be active. The same is true for relationship paths between tables: a Power BI model allows for only one active path between any two tables in the model. This is, of course, only useful if the inactive relationships can be made active if you need to. You can do this by using the `USERELATIONSHIP` function.

The `USERELATIONSHIP` function is used by including it as a filter argument in `CALCULATE`. This may seem surprising, but it really makes sense. A relationship between a fact table and a filter table ensures that a selection in the filter table propagates to the fact table. Activating another relationship means that that relationship now propagates the selection to the fact table, with the effect that different rows in the fact table are selected. In other words, activating another relationship means changing the context for the calculation. And changing context is what `CALCULATE` is for.

`USERELATIONSHIP` takes two arguments, which are references to the columns between which the relationship is defined that must be activated. For example, if the active relationship between the `fSales` and `Date` tables is on `fSales[OrderDate]`, but you want to use the relationship on `fSales[InvoiceDate]` instead, this is the formula to do that:

```
TotalInvoiced =
CALCULATE(
    [Sales],
    USERELATIONSHIP(fSales[InvoiceDate], 'Date'[Date])
)
```

It should be clear that the meaning of the calculated results changes when you use another relationship. While the `Sales` measure returns the amount ordered, the `TotalInvoiced` measure returns the amount invoiced. The former would be used for revenue analysis, while the latter may help in cashflow analysis (in which calculations on actual payments would be a crucial addition). This depends, of course, on the business definitions in the organization of what sales actually are.

Another way of changing relationship behavior is to change the filter propagation behavior of the active relationship. The DAX function for this is

`CROSSFILTER`, which is, again, used in a filter argument in `CALCULATE`.

Like `USERELATIONSHIP`, `CROSSFILTER` takes the two columns involved in a relationship as arguments. In a third argument, you can set the filter propagation direction, or cross-filter type, of the relationship. There are five cross-filter types that can be used:

- `OneWay`: Propagate filters in the default direction – from the table with the primary (unique) key to the table holding the foreign (non-unique) key
- `Both`: Propagate filters in both directions
- `None`: Do not propagate filters at all
- `OneWay_LeftFiltersRight`: Propagate filters in one direction, from the column in the first argument to the column in the second argument
- `OneWay_RightFiltersLeft`: Propagate filters in one direction, from the column in the second argument to the column in the first argument

To give an example of the use of `CROSSFILTER`, suppose you want to know to how many states products were sold. The model contains relationships between the `fSales`, `Cities`, and `Date` tables:

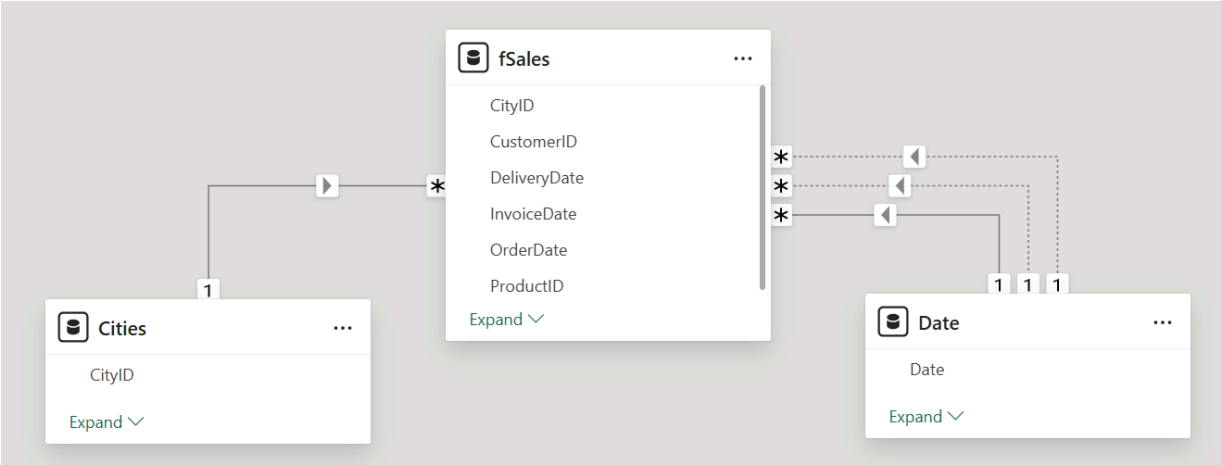


Figure 4.10: Model diagram for sales by city

From the model diagram, notice that we could select a month, and all sales transactions in that month would be selected through the active relationship. We cannot directly count the number of states, however: the relationship between `Cities` and `fSales` only propagates filters from `Cities` to `fSales`,

while we need filter propagation in the other direction. In that case, the selected rows in the `fSales` table would lead to the corresponding rows in the `Cities` table being selected, and we could then count the number of states.

Clearly, the filter propagation direction of the relationship must be changed. The DAX formula is this:

```
StatesSoldTo =
CALCULATE(
    DISTINCTCOUNT(Cities[State]),
    CROSSFILTER(fSales[CityID], Cities[CityID], Both)
)
```

With the `DISTINCTCOUNT` function, we count the unique values in the `State` column. But before that is done, `CROSSFILTER` causes a selection of rows in the `Cities` table based on which rows are selected in the `fSales` table.

Table functions in DAX

There is a lot you can do with basic aggregation functions such as `SUM` and `AVERAGE`, in combination with DAX filtering using `CALCULATE`. But the DAX language goes beyond that. This section is about table functions, which open up an ocean of more advanced calculations in DAX. In the remaining chapters of this book, you will find that many of the business scenarios discussed involve DAX table functions.

Table aggregations

To start, let's look closely at a simple aggregation in DAX:

```
Sales1 = SUM(fSales[SalesAmount])
```

The `SUM` function in this formula traverses the `fSales` table and retrieves the value in the `SalesAmount` column from each row. All these values are summed up to provide the end result.

NOTE

Because of the special way a Power BI model encodes and stores data (see *Chapter 2*, *Model Design*), this may not be what happens

> technically. Logically, however, this is what SUM does, and that is what we are interested in here.

Now, suppose the SalesAmount column is a calculated column, created with the following formula:

```
SalesAmount = fSales[UnitAmount] * fSales[SalesPrice]
```

As both the UnitAmount and SalesPrice columns are in the fSales table as well, a valuable question to ask is: can we calculate sales without using the SalesAmount column? After all, we strongly prefer not to have calculated columns in the model. The calculation we are looking for should again traverse the fSales table, but instead of retrieving the value in the SalesAmount column, it should take the values from the UnitAmount and SalesPrice columns and multiply one by the other. The DAX function to use here is SUMX:

```
Sales2 =
SUMX(
    fSales,
    fSales[UnitAmount] * fSales[SalesPrice]
)
```

While the SUM function only accepts a column reference as its argument, SUMX needs to be provided with a table, fSales in the preceding example, and in the second argument, an expression to be calculated for each row in the table. We call SUMX a *table aggregation function*; you may also encounter the name *iterator*, as SUMX iterates over the table in its first argument.

Most basic aggregation functions have a table aggregation equivalent, such as SUMX, AVERAGEX, MINX, MAXX, COUNTX, COUNTAX, PRODUCTX, and CONCATENATEX. As is obvious from this list, table aggregation functions are recognized by the *X* at the end of the function name. Lesser-known table aggregation functions include statistical functions such as MEDIANX, PERCENTILEX, and STDEVX (the last two functions come in two flavors, which we will not elaborate on here).

A simple table aggregation function is COUNTROWS, which returns the number of rows in a table and has no non-table equivalent. The RANKX function is the

table equivalent of the `RANK.EQ` function.

> **NOTE**
>
> This is somewhat confusing, as there also exists a `RANK` function, which seems the obvious candidate for the non-table equivalent of `RANKX`. However, the `RANK` function is a window function and has a more complex syntax than a simple column-based function such as `RANK.EQ`. We will not elaborate on this here.

The preceding example is helpful if you want to eliminate calculated columns. But the real power of table aggregation functions lies in the fact that you can use any table you want as the first argument. For example, suppose you want to create a measure that returns the average sales per city. For this, the calculation should not iterate over the `fSales` table, which contains individual sales transactions, but over the `Cities` table:

```
SalesPerCity =
AVERAGEX(
    Cities,
    [Sales]
)
```

We will discuss what happens exactly in this calculation, but let us first further elaborate on the tables that can be used. Not only can any table in the model be used in a table aggregation function, but you can even create your own specific tables to work with. We call these **virtual tables** (as *calculated tables* is already taken by the Power BI model).

Using virtual tables

In the previous section, you saw a formula to calculate the average sales per city. Now, suppose we want to compute the average sales per state. With the same approach as the `SalesPerCity` measure, we would need a table with one row for each state to iterate over. This table is not readily available in the model, as `State` is just a column in the `Cities` table. We must therefore construct this table ourselves, and although in this simple case we could add a `State` table to the model (as a calculated table, for instance), the preferred

way to do it is by creating a *virtual table*. This table will only exist during the evaluation of the measure.

There is a broad set of DAX functions available to create virtual tables. The general complexity of working with these functions is that their result is a table. This means that you cannot simply view the results by creating a visual, as you would do for a measure. It is, however, possible to directly view the results of the created table in a DAX query in the DAX query view, since the required output for a DAX query is a table. Creating the virtual table in DAX query view is very helpful for understanding what happens exactly, since working with DAX table functions requires a certain level of abstract thinking.

To start with our state challenge, the `VALUES` function takes a column reference as its argument and returns a table with unique values from that column, as in this example:

```
VALUES(Cities[State])
```

This table expression returns a table with the unique `State` values. An equivalent function is `DISTINCT`, which also returns unique values from a column; the difference is that `DISTINCT` does not include a blank value in the blank row coming from an incomplete relationship (see *Figure 2.5* in *Chapter 2*, *Model Design*). It is up to you to decide whether or not that blank value should be in the result.

The formula for sales per state is as follows:

```
SalesPerState =
AVERAGEX(
    VALUES(Cities[State]),
    [Sales]
)
```

There is a whole group of DAX functions returning tables, which we will not list here comprehensively. The most commonly used functions are as follows:

- `SUMMARIZE`: Although this is a very versatile and complex function that is able to generate complete pivot table-like results, this is not the way `SUMMARIZE` is used within DAX measures. This function is very useful as an extension to `VALUES`: while `VALUES` returns the unique values in one column, `SUMMARIZE` can return the unique combinations of values in

more than one column. For example, the unique combinations of `CityID` and `ProductID` values from the `fSales` table can be retrieved with the following (note that you have to provide the table itself in the first argument):

```
SUMMARIZE(fSales, fSales[CityID], fSales[ProductID])
```

- `ADDCOLUMNS` and `SELECTCOLUMNS`: These two functions are very similar. `ADDCOLUMNS` takes a table as its first argument, and allows you to add columns to that table. You have to give the new column a name and to define an expression that needs to be evaluated for each row of the original table. `SELECTCOLUMNS` also takes a table as a first argument, and with the rest of the arguments, you can either specify which columns of the table you want to have returned, or directly add new columns to this table. For example, if you would like to extend your virtual table from the previous example with a column that returns the sales on transactions in which more than 25 units were sold, you can use the following statement, which uses a measure that has been defined earlier:

```
ADDCOLUMNS(
    SUMMARIZE(fSales, fSales[CityID], fSales[ProductID]),
    "Sales Large Unit Amounts", [SalesLargeUnitAmount]
)
```

- `FILTER`: This function has two arguments. The first is a table (either an existing table in the model or the result of another table function), and the second is an expression that is evaluated for each row in the table. The expression should result in either `true` or `false`, and `FILTER` includes only the `true` rows in the result. For instance, the following expression returns cities in Germany:

```
FILTER(Cities, Cities[Country] = "Germany")
```

- `TOPN`: Like `FILTER`, `TOPN` returns a subset of the rows of a table. The highest or lowest rows of a table are returned, based on some criteria. You provide the number of rows, the table from which to take the rows, the value on which to rank each row, and whether you want them to be sorted from high to low or low to high. For instance, the following is used to create a table with the 15 customers who have the highest sales:

```
TOPN(15, Customers, [Sales], DESC)
```

- `CROSSJOIN`: These functions create a single table from two input tables. `CROSSJOIN` returns a table that simply contains a row for each combination of rows from the input tables. The following example returns a table with all combinations of products and cities, containing all the columns from the `Cities` and `Products` tables:

```
CROSSJOIN(Cities, Products)
```

- `GENERATE`: Like `CROSSJOIN`, this function returns a table that is a combination of input tables. In this case, the second argument is a table expression that is evaluated for each row in the table in the first argument. If this expression happens to return an empty table for a specific row, that row is not included in the result. Alternatively, you can use `GENERATEALL`, which does include such a row, but with blank values in the columns of the table expression. For example, the following expression returns a table with cities and products that have been sold to customers in those cities, again, including all the columns from the `Cities` and `Products` tables:

```
GENERATE(Cities, FILTER(Products, [Sales] > 0))
```

You will encounter a few other table functions in the remaining chapters of this book, which we will introduce when we need them.

Context in table functions

The preceding `GENERATE` example may be intuitively clear, but if you look closer, it is really a quite complicated example. To understand what an expression like this one does, it is important to know how DAX context works in table functions. Let's use the `GENERATE` example in a complete measure formula:

```
AvgUnitAmount1 =
AVERAGEX(
    GENERATE(
        Cities,
        FILTER(
            Products,
            [Sales] > 10000
        )
    ),
```

```
    AVERAGE(fSales[UnitAmount])
)
```

The purpose of this measure is to calculate the average number of products sold per sales transaction (`UnitAmount`) for all sales transactions on products in cities where those products sold more than 10,000. Can you spot the errors in this formula?

When we use this measure in a Power BI visual, it is evaluated in a query context. This context can be anything; it may contain one or more filters on columns in the Power BI model.

The `AVERAGEX` function has two arguments, and these arguments are themselves evaluated in different contexts:

- The first argument, a table expression, is evaluated in the same context as the `AVERAGEX` function itself
- The second argument, a scalar expression, is evaluated *in row context* for every row in the table in the first argument

You may have already noticed this from the `Sales2` measure discussed earlier, which uses direct column references in the second argument of `SUMX`. The row context here provides the possibility of directly using columns in the table to calculate with. In fact, the row context is stacked on top of the query context, while in the row context in a calculated column, there are no filters at all; in this case, the filters from the query context are still there.

NOTE

Common errors made when using virtual tables are related to row context in table aggregation functions. A simple example is shown here:

```
ThisDoesntWork =
SUMX(
    VALUES(fSales[UnitAmount]),
    fSales[Tax]
)
```

While this formula uses two columns from the `fSales` table, the `Tax` column cannot be directly referenced as the row context is not in the

> `fSales` table, but in the result of the `VALUES` function. This result is a table with only one column, `fSales[UnitAmount]`; only this column can be used directly.

The `ADDCOLUMNS`, `SELECTCOLUMNS`, `FILTER`, `TOPN`, and `GENERATE` table functions discussed previously work in the same way: the table argument is evaluated in the context in which the function is called; the other argument is evaluated in a row context on this table in addition to the original context (in other words, we can directly use column values from the table *and* we have the filter from the original evaluation context). In the case of `GENERATE`, this means that we have a table expression evaluated in a row context.

For the formula for `AvgUnitAmount1` shown previously, we have a whole stack of contexts that come into play. Let's break them down step by step:

1. `AVERAGEX`: Evaluated in query context.
2. `GENERATE`: Evaluated in the same context as `AVERAGEX`.
3. `Cities`: Still evaluated in the same context.
4. `FILTER`: Evaluated in row context in the `Cities` table.
5. `Products`: Evaluated in the same context as `FILTER`.
6. `[Sales]`: As this is a call to another measure, an implicit `CALCULATE` causes a filter context to be created. For each call, we are at one row in `Cities` and one row in `Products`. In the filter context, filters on each column in the `Cities` and `Products` tables are added. The result is thus the sales for that product, in that city.
7. `AVERAGE`: This is evaluated in row context in the table returned by `GENERATE`, which is a subset of combinations of cities and products.

So where is the error in this formula?

It is in the last step: although this step is evaluated for the correct combinations of cities and products, it is evaluated in row context. This means that only the filters already present in the query context have an impact on the `AVERAGE` calculation. The current city and product do not impact the calculation, as there are no (additional) filters on the `Cities` and `Products`

tables to select the current city and product. The way to solve this problem is to turn the row context into a filter context, just like what was done in step 6. We can do this by adding `CALCULATE`:

```
AvgUnitAmount2 =
AVERAGEX(
    GENERATE(
        Cities,
        FILTER(
            Products,
            [Sales] > 10000
        )
    ),
    CALCULATE(AVERAGE(fSales[UnitAmount]))
)
```

NOTE

In most cases, it would be better to put the average unit amount calculation in a separate measure, as this will probably be needed elsewhere. You then have to write the logic of the calculation only once, and the call to this measure would take care of the row context transition automatically.

In designing more complex measures in DAX, it is crucial to meticulously keep track of context and context transformations.

There is another, mathematical error in this formula: we calculate the average over the city/product combinations of the average unit amount. This is not necessarily equal to the average unit amount of all sales transactions for these city/product combinations. To solve this, we need to use a different approach, but let us focus on performance first.

Performance considerations using table functions

There are a couple of things to keep in mind when using virtual tables in DAX. The overall goal is always to provide results as fast as possible, so performance should always be a consideration.

First, it is important to realize that virtual tables need to be constructed in memory by the DAX engine. This means that the larger the virtual table, the more memory is needed and the higher the risk of lower performance. You may even run into **Not enough memory to perform this calculation** errors. For

this reason, you should ask yourself whether the table you use could be made smaller: specifically, do you need all the columns included in the table?

In the `AvgUnitAmount2` measure shown earlier, this is clearly not the case. The virtual table created by `GENERATE` contains all columns from both the `Cities` and `Products` tables. But for the result of the calculation, only the unique keys in the tables are really needed: they determine which rows in `fSales` are selected, and thus determine the value of the `Sales` measure. An optimized version would be the following:

```
AvgUnitAmount3 =
AVERAGEX(
    GENERATE(
        VALUES(Cities[CityID]),
        FILTER(
            VALUES(Products[ProductID]),
            [Sales] > 10000
        )
    ),
    CALCULATE(AVERAGE(fSales[UnitAmount]))
)
```

The second thing to consider is the number of rows in the virtual table. While this is obviously also a factor in the total size of the table, it determines the number of iterations in a table aggregation as well.

For instance, if the purchasing price of a product is stored in the `Products` table, you could calculate the total purchasing amount based on the `fSales` table:

```
TotalPurchased1 =
SUMX(
    fSales,
    fSales[UnitAmount] * RELATED(Products[PurchasePrice])
)
```

Instead, you could optimize the number of iterations by letting `SUMX` iterate over the `Products` table instead of the `fSales` table. Or even better, iterate over the unique `PurchasePrice` values:

```
TotalPurchased2 =
SUMX(
    VALUES(Products[PurchasePrice]),
```

```
    Products[PurchasePrice] * CALCULATE(SUM(fSales[UnitAmount]))
)
```

In this variant, all transactions corresponding to a `PurchasePrice` value are aggregated at once. Mind the `CALCULATE` here to go from row context to filter context and filter the correct sales transactions.

NOTE

Memory usage and the number of iterations is a reason why the `CROSSJOIN` function is often not an attractive function to use in DAX measures. The number of rows returned by `CROSSJOIN` can be huge, which can easily lead to memory problems. When you include `CROSSJOIN` in a table aggregation, such as the following (A and B being two arbitrary table expressions):

```
SUMX(
    CROSSJOIN(A, B),
    [Sales]
)
```

It is often more efficient to not use `CROSSJOIN` at all:

```
SUMX(
    A,
    SUMX(
        B,
        [Sales]
    )
)
```

A yet more efficient approach is not to iterate at all, but to use virtual tables for filtering instead. This is the topic of the next section.

Filtering with table functions

One of the most profound insights when working with DAX is the deep connection between tables and filtering. In this section, you will learn what this connection is, and how to leverage it.

Using CALCULATETABLE

As we discussed earlier in this chapter, table expressions used in table aggregation functions such as `SUMX` are evaluated in the same context as the

table aggregation function itself. This is not always what you want: sometimes, you need a different context. DAX provides a function for precisely this purpose: `CALCULATETABLE`.

Like its cousin, `CALCULATE`, `CALCULATETABLE` changes context before evaluating an expression. In `CALCULATE`, this expression must return a scalar value; in `CALCULATETABLE`, it must be a table expression. Apart from that, the function works with the same four-step process:

1. Set up a filter context.
2. Remove existing filters from columns or tables referred to in the filter arguments.
3. Add new filters as specified in the filter arguments.
4. Evaluate the table expression in the first argument.

Often, `CALCULATE` and `CALCULATETABLE` can be used interchangeably. Take, for instance, the following formula:

```
AveragePerCity_Canada1 =
AVERAGEX(
    CALCULATETABLE(
        VALUES(Cities[CityID]),
        Cities[Country] = "Canada"
    ),
    [Sales]
)
```

You can rewrite this measure as a straightforward `CALCULATE` formula:

```
AveragePerCity_Canada2 =
CALCULATE(
    AVERAGEX(
        VALUES(Cities[CityID]),
        [Sales]
    ),
    Cities[Country] = "Canada"
)
```

Both calculations return the average sales per city for cities in Canada (depending on the query context, of course). But be warned: there are technical differences between the two. In the first formula, the `Cities[Country] = "Canada"` filter is applied to the evaluation of the `Cities` table, while in the second formula, the filter is applied to both the `Cities` table and the `Sales` measure evaluations. While this difference does

not impact the results of the measure in this case, you may use more advanced measures instead of `Sales` that are impacted by this difference.

NOTE

Like `CALCULATE`, `CALCULATETABLE` creates a filter context. When used in a calculated column, in each row new filters are added to select that row. When evaluating a related table in the new context, the relationship now has filters to propagate and the related table will be filtered to only the rows that are linked to the current row. This is why the `RELATEDTABLE` function, which is used to retrieve the related part of another table, is nothing more than the `CALCULATETABLE` function without filter arguments.

Using `CALCULATETABLE`, it is possible to add filters to table evaluations. Interestingly, you can use tables to add filters as well.

Filters and tables

Now that we have covered table functions, it is time to go back and take a new look at filters. Earlier, we introduced filters in query and filter context as *instructions to make selections* within columns in the Power BI model, such as *Cities[Country] must equal France or Germany*. You can view this an instruction of the values that should be selected in the `Country` column; or, taking yet another perspective, view it as a single-column table with two rows, containing `"France"` and `"Germany"`.

In fact, this is exactly how filters work, and how the `CALCULATE` function works: by adding tables that define which values in columns are selected, possibly replacing existing tables that implement a filter. The foundational principle is this:

Every filter is a table, and every table can be used as a filter.

This principle means that any simple filter argument in `CALCULATE` can be rewritten as a table. Consider, for instance, the following formula:

```
SalesFranceGermany =
CALCULATE(
    [Sales],
```

```
    Cities[Country] IN {"France", "Germany"}
)
```

The filter argument here is the equivalent of the following table expression:

```
FILTER(
    ALL(Cities[Country]),
    Cities[Country] IN {"France", "Germany"}
)
```

So, the meaning of this filter is literally: of all the `Country` values, select only `"France"` and `"Germany"`.

NOTE

This explains why a formula such as the following one works:

```
CALCULATE(
    [Sales],
    Cities[Country] = "France"
    || Cities[Country] = "Germany"
)
```

Although this is not the simplest filter argument, it is just the equivalent of this:

```
FILTER(
    ALL(Cities[Country]),
    Cities[Country] = "France"
    || Cities[Country] = "Germany"
)
```

Most of the DAX functions we introduced as filter functions can be used as table functions, as you can already see from the preceding `FILTER` expression, which uses `ALL` as a table function. Indeed many `ALL` functions can be used as such: `ALL(Cities[Country])` is a one-column table containing all unique countries, and `ALL(Cities[Country], Cities[State])` is a two-column table containing all unique combinations of country and state that are found in the `Cities` table.

NOTE

Not all functions used in filter arguments are table functions: `USERELATIONSHIP` and `CROSSFILTER` change relationship behavior

> and do not create tables. `KEEPFILTERS` changes the behavior of `CALCULATE`, but cannot be used to create a table. `REMOVEFILTERS`, which functions like `ALL` in a filter argument, cannot be used to create a table.

Even the time intelligence functions are table functions when we exclude the shortcut versions such as `TOTALYTD`. Each creates a one-column table containing the dates in the specified period. This means that you could use these functions in table aggregation functions, for example, to calculate the year-to-date average sales per day:

```
AverageSalesPerDay_YTD =
AVERAGEX(
    DATESYTD('Date'[Date]),
    [Sales]
)
```

The more exciting use of the *filter is table* principle is that you can use any (virtual) table as a filter within a `CALCULATE` function. As an easy example, suppose you want to have a measure that returns the total sales in the country or countries your selected cities are in. If you are sure that in the query context for this calculation, the `Country` column is filtered, then the formula is not difficult:

```
SalesWholeCountry1 =
CALCULATE(
    [Sales],
    ALLEXCEPT(Cities, Cities[Country])
)
```

This calculation removes all the filters from the `Cities` table, except for from the `Country` column. So, if the query context contains the filters *City is Atlanta* and *Country is United States*, the resulting filter context has only the filter *Country is United States* left.

However, if the query context has only the filter *City is Atlanta* to begin with, we are in trouble: removing this filter gives us the whole world! To solve this, we need to "reinject" a *Country is United States* filter into the context, and it should be derived from the `City` filter. This can be done with a table filter:

```
SalesWholeCountry2 =
CALCULATE(
    [Sales],
    ALL(Cities),
    VALUES(Cities[Country])
)
```

To see what is happening here, it is important to realize that filter arguments are evaluated in the same context as the `CALCULATE` table itself. In a query context where only one city is selected, the `VALUES(Cities[Country])` expression returns a single-column table with the country that city is in. This is exactly the filter that we need to implement the required calculation.

As another example, the following formula calculates the sales amount of the 10,000 largest customers:

```
SalesLargestCustomers1 =
CALCULATE(
    [Sales],
    TOPN(10000, ALL(Customers), [Sales], DESC)
)
```

Note that we use `ALL(Customers)` here to be sure that we don't have filters in the query context limiting the customers we are considering. The `TOPN` function returns the 10,000 largest customers (as a subset of the `Customers` table, including all columns – you may want to eliminate unneeded columns here!). This table is then used as a filter.

This formula makes it clear why using table filters should be preferred over using table aggregations. Consider the table aggregation alternative for this measure:

```
SalesLargestCustomers2 =
SUMX(
    TOPN(10000, ALL(Customers), [Sales], DESC),
    [Sales]
)
```

Now ask yourself: how many times is the `Sales` measure called in the evaluation of this calculation? It depends, of course, on the number of customers we have in our `Customers` table. Let's assume that we have 60,000 customers. The `TOPN` function must call `Sales` for each customer to determine who the largest customers are. After that is done, the

`SalesLargestCustomers1` measure only needs one additional call to `Sales`: the `TOPN` table is applied as a filter, creating the context for the largest customers. The `SalesLargestCustomers2` measure, however, traverses the `TOPN` table and calls `Sales` for each row. In other words, this measure calls `Sales` 70,000 times in total, whereas the `SalesLargestCustomers1` measure only calls it 60,001 times! The DAX engine may optimize things here, but the difference will be significant.

Using tables as filters becomes even more powerful when you realize that, unlike filters in a query context, table filters can have multiple columns. This means that we now have a solution for our problematic `AvgUnitAmount3` measure from earlier in this chapter:

```
AvgUnitAmount4 =
CALCULATE(
    AVERAGE(fSales[UnitAmount]),
    GENERATE(
        VALUES(Cities[CityID]),
        FILTER(
            VALUES(Products[ProductID]),
            [Sales] > 10000
        )
    )
)
```

Instead of the average of averages that came with the table aggregation over the `GENERATE` expression, this time, we use `GENERATE` to provide a filter that selects combinations of cities and products that have sales over 10,000. We then calculate the average unit amount for all sales transactions that correspond to the selected city/product combinations. Not only do we now have the correct average calculation, but we have also eliminated iterating over the `GENERATE` table as well, which will help in further improving the performance of this measure.

Using TREATAS

There is one important constraint when using table filters: the tables must effectively filter the tables over which the calculation is done. Just adding a table as a filter that has nothing to do whatsoever with the rest of the model will not do anything.

For instance, the following formula does not return sales for the United Kingdom:

```
UKSales_wrong =
CALCULATE(
    [Sales],
    ROW("Country", "United Kingdom")
)
```

Why does this not work? The Power BI model has no way to determine that with the arbitrary table created with the `ROW` function, which has a column named `Country`, it should filter the `Cities` table, which contains a `Country` column as well. You could say, *Well, the name is the same so the DAX engine could assume that this is what the formula is meant to do*, but that would lead to very unpredictable results in models that may have the same column names in many different tables.

To be able to be used as a filter, the DAX engine should be able to identify that the virtual table is connected to a table, or some columns, from the model. This connection is called **lineage**, and in short, it means that when creating a virtual table, DAX keeps track of what the original columns were where the columns in the virtual table originated from. Let's look at the `GENERATE` expression once again:

```
    GENERATE(
        VALUES(Cities[CityID]),
        FILTER(
            VALUES(Products[ProductID]),
            [Sales] > 10000
        )
```

It is clear that `Cities[CityID]` is a column in the model, and as `VALUES` takes the unique values from that column, `VALUES(Cities[CityID])` has lineage to it. In the same way, `VALUES(Products[ProductID])` has lineage to the `ProductID` column in the `Products` table. `GENERATE` creates a table containing combinations of values from the two `VALUES` expressions, so each column in the resulting table has lineage to the respective model column.

Most table functions retain the lineage of the columns they provide. Some functions, however, allow for forming tables in a strange way, which can be problematic when it comes to lineage. For instance, the `UNION` function allows

for combining tables by taking rows from two source tables that may have conflicting lineage. If so, columns in the result table do not have lineage to any existing column in the model.

In some cases, you want the lineage of the virtual table to be different than it is by default. DAX offers a solution through the `TREATAS` function, which forces the model to consider the columns in a table to have a specific lineage.

`TREATAS` is another example of a function that is exclusively used in `CALCULATE`, or `CALCULATETABLE`, filter arguments. The following formula correctly calculates UK sales (although there are easier ways to accomplish this):

```
UKSales_correct =
CALCULATE(
    [Sales],
    TREATAS(
          ROW("Country", "United Kingdom"),
          Cities[Country]
    )
)
```

Note that `TREATAS` does not require the name of the column to be the same. We could have named the column in the `ROW` expression anything we wanted. `TREATAS` works with multi-column tables as well; in this case, you should specify a model column for each column in the table.

DAX variables

DAX table functions and filtering greatly enhance the complexity of calculations that can be done with DAX. The flip side is that formulas can become quite long. More importantly, with all the different contexts in play, it can be problematic to obtain correct results.

DAX variables make life much easier when designing advanced DAX code. The name is somewhat odd, as the purpose of DAX variables is that you can evaluate something once, and use it later in other circumstances (other contexts, typically) without worrying about the evaluation of the variable. In other words, a DAX variable is used as a constant!

Another advantage is that the result of a variable can be used in multiple places in your DAX code, without the need to calculate it again. This way, you

can even gain some performance optimization by using variables.

A variable is declared using the `VAR` keyword. Multiple variables can be declared, and the declaration of a variable can use the value of another variable declared earlier. Declarations of variables are closed off by the `RETURN` keyword:

```
VAR ThisValue = 5
RETURN
...
```

It is good to know that DAX variables can be used in any expression in a DAX formula. A variable can contain a scalar value, but it can also be a table. The following (rather ridiculous) formula is correct DAX code:

```
VariableTest =
VAR Variable1 = 3
VAR Variable2 = Variable1 + 5
RETURN
CALCULATE(
    VAR Variable3 = MAX(fSales[UnitAmount])
    RETURN
    SUM(fSales[Tax]) + Variable2 + Variable3,
    VAR Variable4 = 4
    VAR TableVariable =
          FILTER(
                ALL(fSales[UnitAmount]),
                fSales[UnitAmount] = Variable4
          )
    RETURN
    TableVariable
)
```

The location of the variable declaration in the formula determines the context in which the variable is evaluated. For instance, the `Variable3` variable shown previously is evaluated in the filter context formed by applying the `TableVariable` table filter to the original query context for this measure. If `Variable3` had been declared right after `Variable2`, it would have been evaluated in the query context. Note that `Variable4` and `TableVariable` are used in a filter argument of the `CALCULATE` function; both are evaluated in the original query context.

Each variable has its own scope, which means that it cannot be used outside of the expression that it is declared in. In the preceding formula, `Variable3` cannot be used to define `TableVariable`; `Variable3` is the first argument of

`CALCULATE` and outside of that, it is unknown. Conversely, `Variable4` and `TableVariable` cannot be used in the first argument of `CALCULATE`. `Variable1` and `Variable2` are part of the expression of the whole formula and can be used anywhere.

DAX variables can help to simplify the flow of a calculation, but they can also make your formulas more readable, simply through the use of clear variable names. Let us revisit the `AvgUnitAmount4` measure once again:

```
AvgUnitAmount4 =
CALCULATE(
    AVERAGE(fSales[UnitAmount]),
    GENERATE(
        VALUES(Cities[CityID]),
        FILTER(
            VALUES(Products[ProductID]),
            [Sales] > 10000
        )
    )
)
```

Using variables, this formula can be simplified:

```
AvgUnitAmount5 =
VAR LargeCityProductCombinations =
    GENERATE(
        VALUES(Cities[CityID]),
        FILTER(
            VALUES(Products[ProductID]),
            [Sales] > 10000
        )
    )
RETURN

CALCULATE(
    AVERAGE(fSales[UnitAmount]),
    LargeCityProductCombinations
)
```

Always remember that DAX variables are constants! The following variant is not correct:

```
AvgUnitAmount_wrong =
VAR LargeProducts =
    FILTER(
            VALUES(Products[ProductID]),
            [Sales] > 10000
    )
VAR LargeCityProductCombinations =
    GENERATE(
        VALUES(Cities[CityID]),
```

```
        LargeProducts
    )
RETURN

CALCULATE(
    AVERAGE(fSales[UnitAmount]),
    LargeCityProductCombinations
)
```

By putting the `FILTER` expression in a variable, we have turned it into a constant table. However, in the `GENERATE` function, we want the product list to be determined anew for each city.

DAX queries

DAX queries always return one or more tables. Therefore, you can use DAX queries to view the results of virtual tables that you use in measures. As described in *Chapter 3*, *Using DAX*, you can use the DAX query view to write these queries and view the results.

To properly debug your measures, it is necessary to create context for the calculations in the form of sets of filters. You can therefore use `CALCULATETABLE` to create context and, for example, mimic the contexts you would have in a report visual.

The following query defines a measure that uses a variable and evaluates both the measure and the variable in a specific context:

```
DEFINE
MEASURE Results[AvgSalesTop15Customers] =
VAR Top15Customers =
    TOPN(15, VALUES(Customers[Customer]), [Sales], DESC)
RETURN
CALCULATE(
    [Sales],
    Top15Customers
)

EVALUATE
CALCULATETABLE(
        {[AvgSalesTop15Customers]},
        'Product'[Category] = "Furniture"
)

EVALUATE
CALCULATETABLE(
        TOPN(15, VALUES(Customers[Customer]), [Sales], DESC),
```

```
        'Product'[Category] = "Furniture"
)
```

Note here that we define the `AvgSalesTop15Customers` measure in the DAX query, which will allow us to either add this measure or update it from within the DAX query view. As you can see, this query contains two `EVALUATE` keywords. This means that the query returns two different result sets, which the DAX query view allows us to do. For both result sets, we use `CALCULATETABLE` to add a `Product[Category] = "Furniture"` filter to the filter context. The second result set gives insight into the value of the `Top15Customers` variable declared in the measure.

Another way to quickly create a set of different evaluation contexts is the use of the `SUMMARIZECOLUMNS` function:

```
DEFINE
VAR RegionEast = TREATAS({"East"}, Customer[Region])

EVALUATE
SUMMARIZECOLUMNS(
    'Product'[Category],
    RegionEast,
    "Top15Sales", [AvgSalesTop15Customers]
)
```

The `RegionEast` variable uses `TREATAS` to turn a simple table with the `East` value into a selection of the `Customer[Region]` column. `SUMMARIZECOLUMNS` accepts a set of column references (just the `Product[Category]` column, in this case) and accepts a number of tables as well, which are used as filters. The combination of columns and filters is used to create a set of filter contexts. It evaluates a set of calculations (the `AvgSalesTop15Customers` measure here) in each of the contexts created.

Summary

In this chapter, you have learned about row context, query context, and filter context and the role that contexts play in the evaluation of DAX formulas. We have discussed how contexts can be transformed using the `CALCULATE` function, by removing filters and adding filters to an existing context. In addition, we looked at time intelligence functions, which provide filters specifically tailored to working with calendars.

We then focused on DAX table functions, which give us the ability to aggregate over tables and the use of custom-made, virtual tables within DAX formulas. Using virtual tables provides a wealth of analytics capabilities on top of what is already possible using "standard" DAX functions and filtering. We talked about the deep connection between tables and filters, which allows for using any table as a filter. After that, we introduced DAX variables, which make it easier to implement complex logic in DAX and add to the readability of DAX code as well.

And, finally, DAX queries were discussed, which allows you to evaluate tables directly, making it easy to view results of virtual tables, and allowing to break down complex DAX code.

All of these are foundational concepts you need for exploring more advanced analyses with DAX. After this pivotal chapter, the remaining chapters of this book are all about applying all the concepts discussed to real-life business cases. Our aspiration is that by going through these cases, you will get to appreciate and understand the power of DAX even more, and that you are inspired to tackle your own business questions using DAX calculations.

Extreme DAX